DUBLIN CITY UNIVERSITY

SCHOOL OF MECHANICAL & MANUFACTURING ENGINEERING

&

SCHOOL OF ELECTRONIC ENGINEERING

PREDICTIVE MAINTENANCE USING PHYSICAL AI

MEng Thesis - ME521

Student Name	Student Number	Program of Study
Franciszek Fraszczyk	21395216	MEIM5

Supervisor

Dr. Derek Molloy

18th March, 2026

Plagiarism Declaration

I understand that the University regards breaches of academic integrity and plagiarism as grave and serious. I have read and understood the DCU Academic Integrity and Plagiarism Policy. I accept the penalties that may be imposed should I engage in practice or practices that breach this policy. I have identified and included the source of all facts, ideas, opinions, viewpoints of others in the assignment references. Direct quotations from books, journal articles, internet sources, module text, or any other source whatsoever are acknowledged and the sources cited are identified in the assignment references. I declare that those sections, which I now submit for assessment, that I have been required to write individually, are entirely my own work and have not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of my work. I declare that those sections, which I now submit for assessment, that I have been required to write as part of a group, are entirely the work of my group and have not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text. I have read and understood the DCU library referencing guidelines (available at: https://www.dcu.ie/library/citing-referencing) and/or recommended in the assignment guidelines and/or programme documentation. By submitting this material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. By submitting material for assessment online I confirm that I have read and understood the DCU Academic Integrity and Plagiarism Policy (available at: https://www.dcu.ie/system/files/sustainability_dcu_editor/2025-09/1-academic_integrity_policy_v5.1.pdf)

Signature: Franciszek Fraszczyk Date: 18/03/2026

Acknowledgements

I would like to express my sincere thanks to my academic supervisor, Derek Molloy. His guidance, professional insight and positive attitude throughout this research were invaluable to me and the technical aspects of this project.

I would also like to thank my family, friends and girlfriend for their continued support during the completion of this work. Their understanding and motivation were essential in helping me stay focused and committed.

Finally, I would like to acknowledge the effort my engineer colleague and friend, Tadhg Reid has put in for me. His assistance and technical discussions related to this project have provided me with a unique perspective during the development and testing phases.

Abstract

Unplanned downtime in industrial environments represents a significant operational and economic challenge. Predictive maintenance (PdM) offers a data driven alternative to reactive and preventive maintenance strategies by enabling early detection of anomalous machine behaviour. This research investigates the feasibility of implementing vibration based anomaly detection using Edge Artificial Intelligence (Edge AI) on a low cost embedded device. A complete end to end PdM system was designed, developed and experimentally validated. Vibration data was acquired from an experimental rig using a triaxial accelerometer operating at 200 Hz. Multiple approaches were investigated through an iterative development process, beginning with a lightweight regression baseline and progressing to a one dimensional convolutional neural network (1D-CNN). To address the practical challenges associated with the lack of labelled fault data, physically inspired anomaly augmentation techniques were developed to generate bearing-like fault signatures in the time series signal. The final quantised deep learning model was successfully deployed on an ESP32-S3 microcontroller using TensorFlow Lite Micro. This enabled real time inference with minimal memory usage and deterministic sampling behaviour. Experimental results demonstrated robust anomaly detection performance across multiple operating speeds, inside and outside the training regime. The model successfully identified anomalies associated with severe mechanical excitation from structural resonance and electrically induced imbalance faults. Remote monitoring functionality was implemented using lightweight MQTT communication, allowing for information to be transmitted using minimal bandwidth. The findings presented in this research confirm that practical PdM solutions can be realised using standalone Edge AI architectures. The work contributes a reproducible framework for embedded vibration monitoring in data scarce industrial settings and establishes a scalable foundation for future extensions towards multi sensor diagnostic and prognostic systems.

Table of Contents

Plagiarism Declaration ..... **i**
Acknowledgements ..... **ii**
Abstract ..... **iii**
Table of Contents ..... **iv**
Nomenclature ..... **v**
List of Figures ..... **vi**
List of Tables ..... **vii**
Chapter 1: Introduction **1**
1.1 Predictive Maintenance 1
1.2 Edge Artificial Intelligence 2
1.3 Research Questions 3
1.4 Engineering Contribution 6
1.5 Engineers Ireland Accreditation Criteria 8
1.6 Thesis Structure 10
Chapter 2: Literature Review **11**
2.1 Motivating Factors 11
2.2 Predictive Maintenance: Data-Driven Frameworks 12
2.3 Sensing Modalities for Data-Driven Predictive Maintenance 14
2.4 Classical Machine Learning for Predictive Maintenance 15
2.5 Deep Learning 17
2.5.1 Convolutional Neural Networks 17
2.5.2 Long Short-Term Memory 19
2.5.3 Supervised vs Unsupervised Learning 20
2.5.4 Data Requirements and Practical Limitations 20
2.5.5 Transfer Learning 22
2.6 Edge Artificial Intelligence 22
2.6.1 Latency, Bandwidth and Reliability 23
2.6.2 Constraints and Trade-Offs 23
2.7 Vibration Based Activity Recognition 24
2.8 Conclusion 26
2.9 Research Gaps 27
Chapter 3: Ethical Considerations **28**
3.1 Stakeholder Impact 28
3.2 Reliability & Human Oversight 29
3.3 Dual-Use Technology and Military Applications 29
3.4 Commercial Misuse 30
3.5 Transparency and Reproducibility 30
3.6 Data Confidentiality 30
3.7 Manufacturing Sustainability 31
3.8 Sustainability in Society and Economics 32
3.9 Case Study: Stuxnet 32

3.10 Summary of Design Decisions..... 33
Chapter 4: Theory - Data Acquisition System..... 34
4.1 Limitations of Cloud-Based Logging..... 34
4.2 Local Data Logging Architecture..... 35
4.3 USB CDC Data Logging..... 36
4.4 USB Logged Dataset..... 40
4.5 Repeated Sample Values..... 40
4.6 Validation of the Logging System..... 41
Chapter 5: Iterative Development of the System..... 42
Version 1A - Experimental Regression Model..... 43
5.1 Specific Theory & System Design..... 43
5.1.1 Regression Model for Anomaly Detection..... 44
5.2 Implementation & Methodology..... 48
5.2.1 Regression Model Training & Implementation..... 48
5.2.2 Embedded Deployment and On-device Validation..... 56
5.3 Experimental Results & Discussion..... 61
5.3.1 Regression Model Offline Performance..... 61
5.3.2 Embedded Regression Inference Performance..... 65
Version 1B - Experimental 1D-CNN Model..... 69
5.4 Specific Theory & System Design..... 69
5.4.1 Convolutional Neural Networks..... 69
5.4.2 Quantisation & Deployment..... 74
5.5 Implementation & Methodology..... 78
5.5.1 Experimental 1D-CNN Model Training..... 78
5.5.2 Embedded Experimental 1D-CNN Model..... 84
5.6 Experimental Results & Discussion..... 86
5.6.1 Experimental 1D-CNN Model Performance..... 86
5.6.2 Embedded Experimental 1D-CNN Model..... 89
Version 2A - Physical Anomalies..... 92
5.7 Specific Theory & System Design..... 92
5.7.1 Physical Anomaly Generation..... 92
5.8 Implementation & Methodology..... 94
5.8.1 Motor 1D-CNN Model - Physical Anomalies..... 94
5.8.1 Embedded Deployment: 1D-CNN - Physical Anomalies..... 100
5.9 Experimental Results & Discussion..... 102
5.9.1 1D-CNN Model - Physical Anomalies..... 102
5.9.2 Embedded Deployment: 1D-CNN Physical Anomalies..... 107
Version 2B - Lengthened Data..... 109
5.10 Specific Theory & System Design..... 109
5.11 Implementation & Methodology..... 112
5.12 Experimental Results & Discussion..... 116
5.12.1 1D-CNN Model - Lengthened Data..... 116
Version 2C - Augmented Anomalies..... 120
5.13 Specific Theory & System Design..... 120
5.13.1 Data Augmentation - Bearing Like Anomalies..... 120

5.14 Implementation & Methodology..... 124
5.14.1 Data Augmentation - Bearing Like Anomalies..... 124
5.14.2 Motor 1D-CNN - Augmented Data..... 127
5.15 Experimental Results & Discussion..... 128
5.15.1 Data Augmentation - Bearing Like Anomalies..... 128
5.15.2 Embedded Deployment: 1D-CNN - Augmented Anomalies..... 129
Version 3A - Multi-Speed Augmented CNN..... 131
5.16 Specific Theory & System Design..... 132
5.17 Implementation & Methodology..... 134
5.17.1 Motor 1D-CNN - Multiple Speeds..... 134
5.17.2 IoT Implementation..... 137
Chapter 6: Results & Discussion..... 142
6.1 1D-CNN Model - Multiple Speeds..... 142
6.1.1 Generated Anomalies..... 142
6.1.2 Quantisation..... 145
6.1.3 Model Deployment - LED verification..... 145
6.2 Robustness Across Operating Conditions..... 147
6.2.1 False Positives..... 149
6.2.2 Discussion of Results..... 149
6.3 IoT Results..... 150
6.4 Embedded Deployment Discussion..... 152
6.4.1 Model Deployment..... 152
6.4.2 Practical Deployment..... 153
6.5 Limitations of Final System..... 154
Chapter 7: Conclusions & Recommendations..... 155
7.1 Conclusions..... 155
7.2 Recommendations for Future Work..... 157
References..... 158
Appendix A - SPIFFS Data Logging..... 162
Appendix B - Python Logger..... 167
Appendix C - Augmented Anomaly Generation..... 168
Appendix D - FYP Risk Assessment..... 170
Appendix E - Gantt Charts..... 171
Appendix F - Experimental Rig..... 173
Appendix G - Platform Selection..... 176

Nomenclature

AI	Artificial Intelligence
CNN	Convolutional Neural Network
CSV	Comma Separated Values
DL	Deep learning
Edge AI	Artificial intelligence executed on edge devices
ESP32	Embedded Microcontroller Platform
FFT	Fast Fourier Transform
I^2C	Inter-Integrated Circuit Communication
IoT	Internet of Things
IIoT	Industrial Internet of Things
MCU	Microcontroller Unit
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
PdM	Predictive Maintenance
ROC	Receiver Operating Characteristic
RPM	Revolutions Per Minute
RMS	Root Mean Square
SPIFFS	Serial Peripheral Interface Flash System
TFLM	TensorFlow Lite for Microcontrollers

List of Figures

Figure 1.1. High level overview of the proposed embedded PdM System

Figure 1.2. Development workflow of the proposed PdM system.

Figure 2.1. Example of a simple CNN Model

Figure 4.1. Flow diagram of local data logging architecture.

Figure 4.2. Circuit design of data logging system iteration 4.

Figure 4.3. Circuit implementation of iteration 4, logging active (left) inactive (right).

Figure 5.1. Example acceleration time-series data illustrating high-energy transient events embedded within stationary background vibrations.

Figure 5.2. Trimmed Accelerometer Axes x (blue), y (green), z (yellow) in m/s^2 vs Time (us).

Figure 5.3. Triaxial acceleration time-series with statistical threshold anomaly detections.

Figure 5.4. Under labelling behaviour of the threshold based anomaly detection approach.

Figure 5.5. Decision Boundary Plot.

Figure 5.6 On-device visual alert mechanism triggered by induced tap.

Figure 5.7. Devices invariance to slow rotations and orientation.

Figure 5.8. Mapping of FP32 values in the range [-1,1] to discrete int8 quantised values.

Figure 5.9. Raw triaxial acceleration time-series used for windowed anomaly labelling.

Figure 5.10. Jerk magnitude computed from triaxial acceleration with threshold based anomaly candidates highlighted.

Figure 5.11. ROC Curve for Version 1B

Figure 5.12. Real time NeoPixel LED response during embedded anomaly detection (green = normal, red = anomaly).

Figure 5.13. Servos position on the accelerometer casing.

Figure 5.14. Servo on and off states.

Figure 5.15. Fully assembled rig with annotations.

Figure 5.16. Raw z-axis acceleration during motor operation showing visually distinct high amplitude transient disturbances.

Figure 5.17. Triaxial acceleration time series highlighting clear separation between normal motor operation and induced anomalous events.

Figure 5.18. Example of slight accelerometer mounting tilt.

Figure 5.19. Embedded system during normal operation with servo inactive (LED green).

Figure 5.20. Embedded system during normal operation with servo active (LED red).

Figure 5.21. Quasi periodic vibration signal with phase consistent segmentation point indicated.

Figure 5.22. Phase-consistent signal lengthening workflow.

Figure 5.23. Phase aligned segmentation point at index 2682 identified using the estimated frequency.

Figure 5.24. Effect of the Hann window on a FFT input segment.
Figure 5.25. One second comparison of original (top) and phase-lengthed (bottom) x-axis vibration signals demonstrating continuity and absence of stitching artefacts.
Figure 5.26. Ten second comparison of original (top) and lengthed (bottom) x-axis vibration signals showing preserved harmonic structure and amplitude.
Figure 5.27. Full length comparison between original (top) and lengthed (bottom) vibration datasets showing preserved global structure.
Figure 5.28. Crossfade region between two stitched segments.
Figure 5.29. Bearing-like anomaly injection workflow.
Figure 5.30. First 250 windows from the augmented dataset (z-axis), showing healthy samples (blue) and synthetically injected bearing-like anomalies (orange) across training, validation and testing sets.
Figure 5.31. Push button operated coil disconnector.
Figure 5.32. Full system integration inference pipeline workflow.
Figure 6.1. Example input windows for 180 μs step delay showing normal operation (left) and augmented anomalous operation (right) across the normalised x, y, z axes.
Figure 6.2. Example input windows for 240 μs step delay showing normal operation (left) and augmented anomalous operation (right) across the normalised x, y, z axes.
Figure 6.3. Example input windows for 320 μs step delay showing normal operation (left) and augmented anomalous operation (right) across the normalised x, y, z axes.
Figure 6.4. Example input windows for 180 μs step delay showing normal operation (left) and augmented anomalous operation (right) across the normalised x, y, z axes.
Figure 6.5. X-axis acceleration over time with electrical fault induced.
Figure 6.6. Normal operation
Figure 6.7. Resonance anomaly detection
Figure 6.8. Latched alarm state.
Figure 6.9. Alarm reset and return to normal operation.
Figure 7.1. Objectives and Achievements of each model iteration.
Figure A.1 Iteration 1 Circuit Design
Figure D.1. FYP Risk Assessment
Figure E.1. Semester 1 Gantt Chart
Figure E.2. Semester 2 Gantt Chart
Figure F.1. SOLIDWORKS Assembly of Experimental Rig
Figure F.2. SOLIDWORKS Drawing of Top Bracket
Figure F.3. SOLIDWORKS Drawing of Bottom Bracket

List of Tables

Table 1.1. Research Questions and Model Versions

Table 1.2. Colour scheme used for **Figure 1.1**

Table 4.1. Target Design Requirements

Table 4.2. Summary of Data Acquisition System Iterations (1-4)

Table 4.3. Sample dataset excerpt

Table 5.1. Regression model objectives.

Table 5.2. Table of extracted features.

Table 5.3. Confusion matrix for regression model

Table 5.4. Serial monitor output showing embedded feature values and predictions.

Table 5.5. 1D-CNN architecture.

Table 5.6. Classification report for Version 1B

Table 5.7. Embedded CNN inference probabilities and decision threshold behaviour.

Table 5.8 Serial monitor output.

Table 5.9. Embedded Normalisation diagnostics.

Table 5.10. Serial monitor output.

Table 5.11. Stepper motor operating speeds derived from step time parameters.

Table A.1. Sample of the dataset from iteration 1 of the data logging program.

Table G.1. Table detailing the strengths and weaknesses of different programming platforms.

Table G.2. Decision Matrix detailing the chosen platform.

Table G.3. Table of boards

Table G.4. Legend detailing colour patterns in Table C

Chapter 1: Introduction

1.1 Predictive Maintenance

Manufacturing environments depend on the continuous operation of mechanical assets such as motors or actuators. These machines are typically subject to constant load, vibration and/or friction, leading to degradation and eventual operational failure. The method in which maintenance is performed has a direct impact on production efficiency, operational economics and system reliability.

Traditional maintenance strategies such as reactive maintenance (fix when broken) and preventative (scheduled) maintenance are inefficient, time consuming and financially weak. Reactive maintenance only addresses faults after failure occurs, which leads to unplanned downtime, emergency repairs and potential secondary damage to nearby equipment. Preventive maintenance attempts to reduce these risks by servicing equipment at fixed time intervals. However, this approach is often inefficient since the actual health condition of the machine is not considered. Replacing components prematurely wastes production time and resources.

The fundamental problem that lies within manufacturing companies is that they do not know the real-time health condition of their machines during operation. Without data driven insights or early signs of mechanical failure, maintenance personnel are forced to operate based on a set schedule or relative assumptions. Unplanned downtime in manufacturing lines represents a heavy economic burden. A 2023 survey conducted by ABB reported that over two thirds of industrial businesses experience unplanned downtime at least once a month, with the average cost estimated at approximately $125,000 per hour [1]. Other industry analyses also highlight the effect of unplanned downtime, for example Aberdeen Research reported that 82% of companies experience at least one instance of unexpected downtime within a three year period [2].

Predictive maintenance (PdM) aims to support these companies by providing a system capable of continuously monitoring equipment condition and detecting early signs of anomalous behaviour. By identifying faults before catastrophic failure, maintenance can be performed when necessary, reducing downtime, extending asset life and improving factory throughput. As industrial systems become more data driven and automated, the development of reliable and deployable PdM solutions are increasingly more important.

1.2 Edge Artificial Intelligence

Recent advances in machine learning (ML) and deep learning (DL) have enabled the development of data driven condition monitoring systems, capable of identifying complex patterns in vibration signals. Many existing approaches rely on high performance cloud computing platforms, which introduce latency, cybersecurity concerns and increased infrastructure costs.

Edge Artificial Intelligence (Edge AI) offers solutions to these limitations. Sensing, data processing and inference can all be performed locally on embedded hardware at the asset. This approach reduces communication overheads, improves response times and enables monitoring solutions to be deployed in environments with limited connectivity or strict operational information security requirements.

Additionally, due to the computational demands of AI, cloud services are becoming increasingly more expensive. This motivates a shift within industries to invest in local intelligence in order to avoid these increasing costs.

The work in this research investigates the feasibility of implementing a low cost embedded system, capable of vibration based anomaly detection entirely at the edge. Vibration data is acquired using a triaxial accelerometer and processed directly on the embedded device. A deep learning model is trained using experimentally recorded vibration data and deployed in quantised form to enable real time inference.

The scope of this work is limited to the detection of anomalous vibration behaviour rather than full fault diagnostics or prognostic estimations. The objective is to demonstrate the feasibility of a deployable embedded anomaly detection system rather than to develop a complete industrial PdM platform. Importantly, the architecture is developed without the use of transfer learning techniques or other datasets that could be unobtainable, making this fully reproducible and accessible. As such, this work is designed to support future extensions such as remote data collection and more advanced diagnostic or prognostic algorithms.

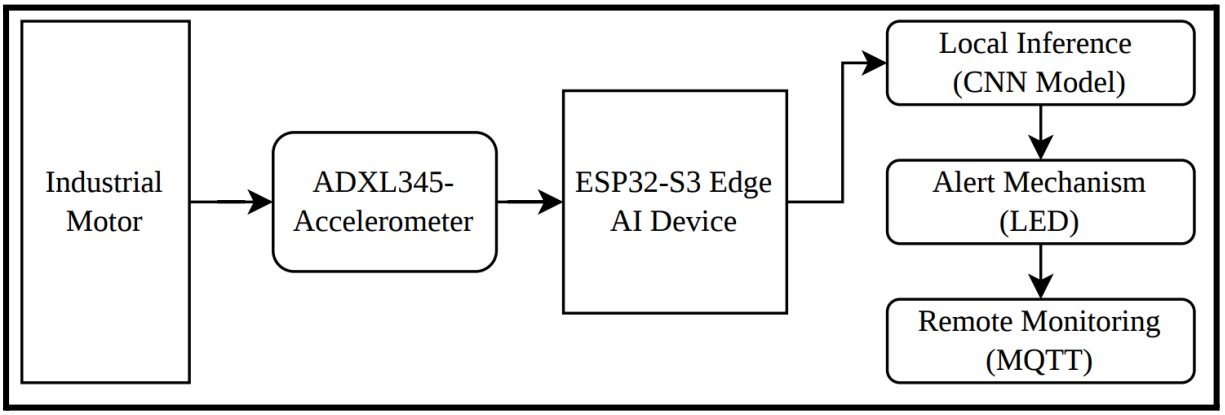


Figure 1.1. High level overview of the proposed embedded PdM System

1.3 Research Questions

There are several challenges identified in current industrial PdM practises, including limited availability of labelled fault data, reliance on cloud based analytics and difficulties associated with live deployment on embedded systems. Based on these challenges, this research investigates the feasibility of a fully embedded vibration based anomaly detection system.

To address the complexity of this problem in a structured form, the investigation is divided into four research questions. Each question focuses on a key technical uncertainty that must be addressed in order to deploy the practical standalone solution.

RQ0 - Data Acquisition

Can vibration data suitable for machine learning based anomaly detection be reliably collected using an ADXL345 accelerometer operating at 200Hz?

RQ1 - Embedded Anomaly Detection Feasibility

Can anomaly detection be performed on an embedded system using classical machine learning or deep learning methods?

RQ2 - Real Motor Operation Anomaly Detection

Can a trained 1D-CNN model detect anomalies directly from raw vibration signals generated by a running motor in real time?

RQ3 - Robust Embedded Predictive Maintenance System

Can a robust embedded anomaly detection system be achieved by combining mutli-speed datasets, synthetic data augmentation, and embedded deployment with network communication?

Table 1.1. *Research Questions and Model Versions*

Research Question	Model Versions
RQ0	Data Gathering
RQ1	Version 1A-1B
RQ2	Version 2A-2C
RQ3	Version 3A

To address these research questions, the development process is further divided into incremental experimental versions. Each sub version explores a specific methodological improvement or variation while remaining aligned with the corresponding research question. This structured approach allows the investigation of the project to progress iteratively while maintaining clear traceability between experiments and research objectives.

For example, RQ1 is explored through Version 1A and Version 1B, where Version 1A establishes a classical machine learning baseline using extracted vibration features. Version 1B on the other hand replaces manual feature engineering with a deep learning approach and tests whether it can be deployed on the ESP32-S3. This progression allows the feasibility of embedded anomaly detection to be evaluated prior to overly committing in one direction.

Similarly, RQ2 is investigated in three separate steps, each addressing a different aspect of model robustness when exposed to real motor vibration data. Version 2A introduces physically generated anomalies on a running motor, Version 2B increases the effective dataset size through signal lengthening, and Version 2C introduces synthetic fault augmentation.

Finally, RQ3 corresponds exclusively to Version 3A, which is the cumulation of all efforts and represents the full system implementation. In this stage, the model is trained using multi-speed datasets combined with augmented anomalies. The trained model is then deployed on the embedded device with integrated network communication.

This versioned structure enables the research questions to be addressed through steps of controlled experiments. The overall workflow diagram is presented below in **Figure 1.1** and illustrates the objective of each version.

Table 1.2. *Colour scheme used for* ***Figure 1.2.***

Research Question 0
Research Question 1
Research Question 2
Research Question 3

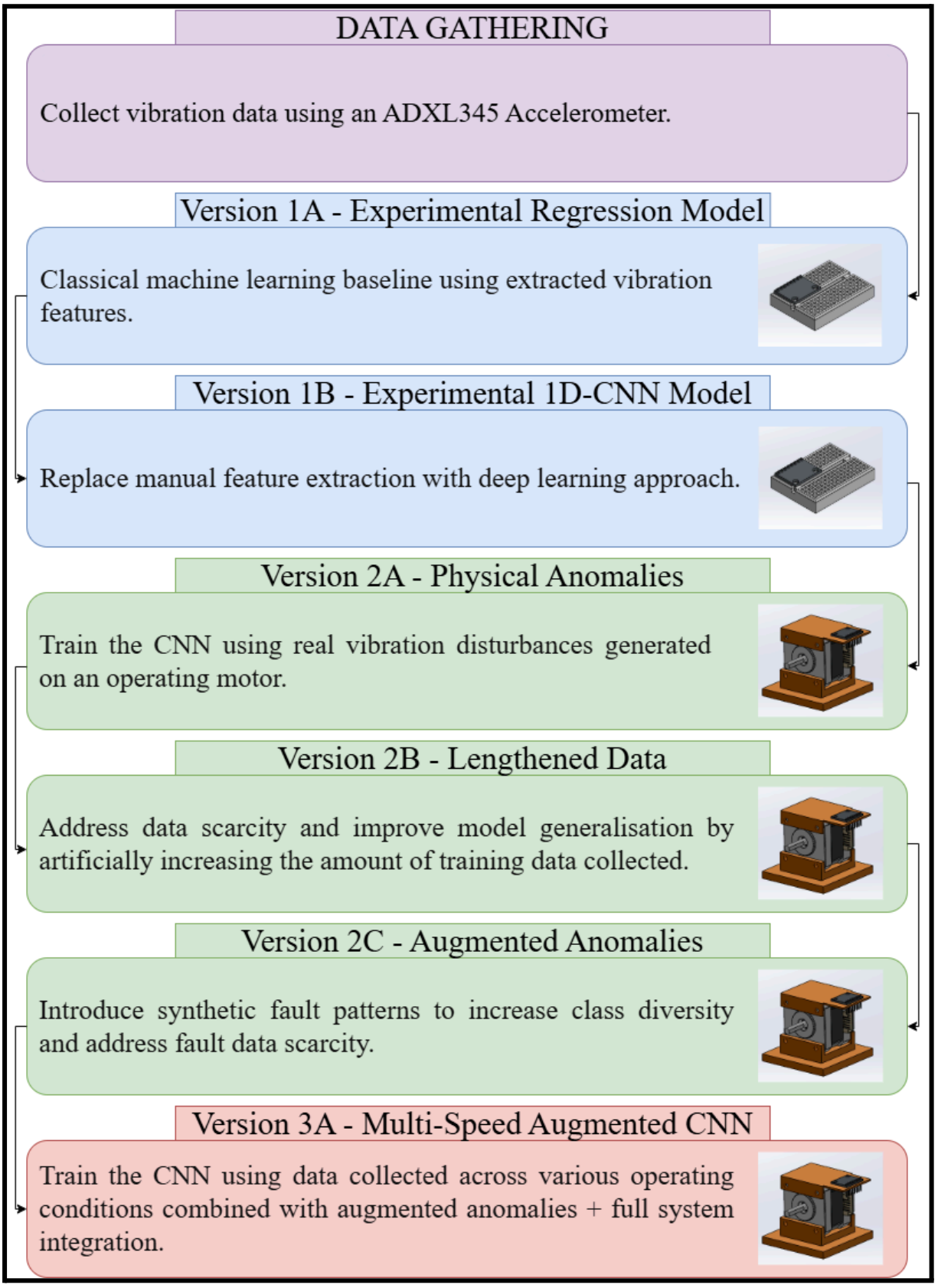


Figure 1.2. Development workflow of the proposed PdM system.

1.4 Engineering Contribution

The engineering contributions of this work are organised in order of their significance to the overall predictive maintenance system. The primary contribution is the realisation of the fully operational embedded anomaly detection system. This contribution demonstrates that vibration based PdM can be implemented at the edge. The remaining contributions support this central objective by addressing key practical challenges encountered during development. Each contribution is aligned with one or more of the research questions defined above.

1. Realisation and Testing of an Embedded Anomaly Detection System

(Related to RQ0, RQ1, RQ2)

A complete end-to-end anomaly detection pipeline was developed, implemented and deployed onto an ESP32-S3. The system collects vibration data from a triaxial accelerometer, performs preprocessing and executes a quantised 1D convolutional neural network directly on the embedded device. This demonstrates the feasibility of running deep learning vibration analysis on resource constrained hardware, enabling real time anomaly detection without reliance on cloud processing.

A fully functional experimental rig was developed to simulate vibration behaviour of a stepper motor. The rig enabled the generation of repeatable vibration datasets across multiple motor speeds and provided a controlled environment for testing the embedded anomaly detection system. This contribution directly supports **RQ0**, which investigates the reliability of vibration data acquisition. Furthermore, the end-to-end pipeline encompasses **RQ1** and **RQ2**, by evaluating the feasibility of anomaly detection on embedded hardware and by validating the different data augmentation methods used.

2. Investigation of Data Scarcity in Small-Scale Industrial Monitoring

(Related to RQ2, RQ3)

A key challenge addressed in this work is the lack of large, labelled vibration data in small-scale industrial environments. Multiple strategies were explored to address this issue, including signal lengthening and synthetic anomaly injection. Through experimental evaluation, this research demonstrates the encountered limitations of data lengthening and highlights the importance of preserving statistical consistency when augmenting time series vibration data.

These findings provide insight into the practical challenges associated with training machine learning models in data scarce environments. This investigation directly contributes to **RQ2** and **RQ3**, which question how robust embedded predictive maintenance systems can be developed when large labelled datasets are unavailable.

3. Development of Physically Inspired Data Augmentation for Bearing-like Faults

(Related to RQ2, RQ3)

A reproducible augmentation pipeline was implemented to generate faults similar to bearing failure in vibration data. The augmentation process models impulsive excitation followed by sinusoidal decay, producing fault signatures that resemble real mechanical defects. This approach enables the generation of diverse anomalous training samples while maintaining physically realistic signals. The methodology and implementation details are documented in this work, with the key code listings provided. As a result, the augmentation process can easily be reproduced or extended by other practitioners facing similar data scarcity challenges. Similarly to the second engineering contribution, this one supports **RQ2** and **RQ3**. This development explores the use of data augmentation to improve robustness in the deployed live system.

4. Integration of Remote Monitoring Through MQTT

(Related to RQ3)

The PdM system was extended beyond local operation by integrating Message Queuing Telemetry Transport (MQTT) communication. Detected anomalies can be transmitted from the embedded device to remote monitoring systems, enabling real time alerts and logging. This also lays down the foundation for future improvements. By modifying the system to also capture the preceding vibration history surrounding anomalies, more sophisticated diagnostic or prognostic algorithms could be developed. Therefore, the system presented in this work is not only a standalone anomaly detection device, but also a scalable foundation from which more advanced systems can be built. This contribution addresses **RQ3**, which investigates the development of a fully connected, robust embedded PdM system.

1.5 Engineers Ireland Accreditation Criteria

This thesis was developed to meet the expectations of a Master's level engineering investigation [3]. The criteria are set out by Engineers Ireland and they were met by demonstrating advanced engineering knowledge, complex problem solving, system design capability and professional awareness.

PO1. Knowledge and Understanding

- This research integrates vibration signal processing, machine learning, embedded systems and IoT communication into a single architecture. This demonstrates the ability to link multidisciplinary knowledge to achieve a final, complete product.

PO2. Problem Analysis

- The work addresses the challenges that are associated with implementing PdM in data scarce industrial environments. Experimental methods were developed and used to collect usable vibration datasets, mitigate limited fault data and validate anomaly detection performance. Additionally, the work investigates embedded deployment constraints and verifies the feasibility of such implementations.

PO3. Design

- A low cost, standalone Edge AI monitoring system was designed, developed and implemented. The system performs real-time vibration analysis using a quantised deep learning model deployed on a microcontroller. The entire system operates independently, with no need of transfer learning techniques or foreign datasets. This represents an innovative, reproducible and scalable approach to industrial condition monitoring.

PO4. Investigation

- A series of controlled experiments were conducted iteratively to evaluate model feasibility, model robustness and effectiveness of data augmentation strategies. Embedded deployment constraints were consistently investigated and validated.

PO5. Professional and Ethical Responsibilities

- Ethical and sustainable considerations influenced several key design decisions. For example, local data processing was used to reduce exposure of sensitive operational information. The development and use of PdM directly supports sustainable engineering practises as it helps extend asset lifespan and reduce waste.

PO6. Teamwork and Lifelong Learning

- The research was done primarily in an independent manner. Where appropriate, major design decisions and methodological choices were discussed with academic supervisors. The iterative nature of the work demonstrates the ability to plan, execute and refine complex engineering tasks in a systematic approach. Engagement with relevant literature and emerging Edge AI methodologies further reflects the development of engineering skills required for lifelong professional practice.

PO7. Communication and Technical Reporting

- The outcomes of this research are communicated through a structured technical document with supporting figures, tables and relevant code listings. Complex signal processing techniques, machine learning workflows and embedded system constraints are all presented in a transparent and reproducible manner. This demonstrates the ability to convey advanced engineering concepts to a technical and non-technical audience.

PO8. Engineering Management

- Practical project planning included experimental testing, dataset generation, embedded resource optimisation and deployment feasibility assessments. In addition to technical constraints, time management was critical in the successful completion of this work. As part of an integrated Master's programme, the thesis was carried out alongside other academic commitments such as exams, coursework and laboratory work. The iterative approach in this work supports effective project management, where technical objectives were achieved within fixed time boundaries.

1.6 Thesis Structure

The thesis is structured in an organised sequence of chapters that progressively develop and evaluate the proposed PdM system.

Chapter 2 presents a review of relevant literature in maintenance, vibration based monitoring, embedded machine learning and Edge AI. Existing approaches to anomaly detection, data requirements and deployment challenges are critically discussed in order to ground the engineering contributions of this work.

Chapter 3 outlines the ethical considerations connected with the development of data driven industrial monitoring systems. Topics include dual-use technology, cybersecurity, data privacy and responsible system deployment. These considerations influence several design decisions adopted in the PdM architecture.

The development of the system begins in Chapter 4, which focuses on the theoretical background and actual implementation of the vibration data acquisition system. This chapter describes the sensing principles and experimental rig design used to generate the vibration datasets for all implementations of the PdM system.

Chapter 5 forms the primary development chapter of the thesis. It presents the theoretical foundations, implementation methodology and experimental evaluation of each incremental system version. Versions 1 and 2 are fully theorised, developed and evaluated within this chapter, including model design, preprocessing techniques and real-time embedded deployment results. This structure is adopted to reflect the iterative workflow used during system prototyping.

The final system is separately evaluated in Chapter 6, which presents the results of Version 3A. This chapter evaluates the robustness of the complete embedded PdM architecture, including multi-speed dataset integration, synthetic anomaly augmentation and MQTT based remote monitoring.

Finally, Chapter 7 summarises the main findings of the research, discusses limitations of the proposed system and outlines opportunities for future research.

Chapter 2: Literature Review

2.1 Motivating Factors

Unplanned downtime remains as one of the most significant operational and financial challenges that modern industrial manufacturing systems face. Large-scale industry surveys consistently report that unexpected equipment failures can result in substantial financial losses due to the cost of lost production, labour inefficiencies and cascading secondary damage. At a more global scale than the ABB's *Value of Reliability* survey [1], a multi-year survey conducted by Siemens in 2024 estimates that unplanned downtime costs the world's 500 largest industrial organisations approximately 11% of their annual revenues, corresponding to total losses of roughly $1.4 trillion per year [4].

These industry-reported figures are supported by peer-reviewed economic analyses of different maintenance practices. A study in 2021, managed by Thomas and Weiss at the U.S National Institute of Standards and Technology (NIST) estimates that the total annual costs associated with inadequate maintenance in industrial manufacturing amount to an average of $222 billion, based on Monte Carlo analysis of survey data collected across multiple industrial sectors [5]. Collectively, these figures highlight that maintenance-related failures represent a significant economic burden, motivating the development of more effective maintenance strategies. Furthermore, as manufacturing systems continue to increase in complexity and autonomy, the tolerance for unpredictable faults has been reduced drastically, creating a strong industrial demand for intelligent, predictive maintenance strategies.

Historically, maintenance practices have evolved through several distinct paradigms. Reactive maintenance, often referred to as *run-to-failure*, represents the earliest approach and is characterised specifically by corrective action only after equipment failure has occurred. While simple to implement, this approach towards maintenance is economically inefficient and unsuitable for modern high throughput production environments [6]. As a response to the economically outdated method, preventive maintenance emerged, introducing scheduled servicing based on historical data and statistics such as mean time to failure (MTTF), mean time to detect (MTTD), mean time to repair (MTTR) and mean time between failures (MTBF) [7]. Although preventive maintenance reduces catastrophic failures, it often results in increased labour costs, unnecessary component replacement and does not adequately account for variations in operating conditions or machine usage [8]. These limitations have driven the continued evolution of maintenance strategies towards condition-based and predictive maintenance frameworks, where maintenance actions are influenced by the actual health state of equipment.

Statistically investigating reactive maintenance further validates the necessity of implementing newer, updated maintenance strategies. In the short term reactive maintenance appears to be the most cost effective due to the reduced inspection and monitoring overheads, since there is no need for additional sensors or labour to monitor the health of assets. Unplanned failures give rise to emergency repairs, expedited part procurement, overtime or specialised labour and potential collateral damage to other adjacent components. The aforementioned economic analysis by Thomas and Weiss [5] demonstrates that organisations relying predominantly on reactive maintenance experience 52.7% more unplanned downtime and 78.5% higher defect rates when compared to organisations that primarily employ preventative or predictive maintenance. In monetary terms, unplanned downtime alone contributes to approximately $18.4 billion annually to U.S. manufacturing losses, while total maintenance-related costs, including defects and lost sales, were estimated to exceed $222 billion on average [5]. This study further indicates that in the same comparison these firms on average experience approximately 73% greater sales losses due to maintenance-related delays when measured relative to shipment value. In this context, shipment value refers to maintenance-related costs expressed as a proportion of the total value of goods produced and sold [5]. Companies that transition away from reactive maintenance typically increase direct maintenance expenditure relative to shipments, but this increased cost is substantially offset by reduction in downtime, defects and lost sales.

2.2 Predictive Maintenance: Data-Driven Frameworks

Predictive maintenance represents a core pillar of Industry 4.0, integrating sensors, connectivity, data analytics, and artificial intelligence to anticipate points of failure preemptively [6], [8], [9]. In the predictive approach to maintenance, continuous streams of sensor data such as vibrations, temperature or current draw are analysed to detect deviations from normal operating behaviour. By identifying early indicators of mechanical degradation, predictive maintenance enables timely intervention on the equipment, extending asset life, reducing downtime and improving overall equipment effectiveness. Achouch *et al.* [6] emphasise that data-driven predictive maintenance has become progressively more viable due to advancements in sensor technology, increased data availability, and the existence of mature ML techniques capable of extracting complex patterns for dimensional time-series data.

Predictive maintenance approaches can be roughly categorised into two major frameworks: physics based and data driven [10]. This classification of the two major frameworks is commonly employed in the development of digital twin systems, where real time machine health and operational metrics are used to model and monitor the condition of physical assets. Although the referenced study focuses on the construction of digital twins, the underlying modelling methods are directly applicable to predictive maintenance, as both approaches aim to infer asset health and degradation from measured real-time system data.

Physics based PdM relies on explicit, handcrafted mathematical models, system dynamics and material properties in order to describe degradation trends. Such models can provide strong insights into the health condition of operating machinery, but are often difficult to construct for complex, mutli-component systems. As industrial assets become more specialised and operate under variable loads, purely physics based approaches become more impractical at scale [11].

In contrast, data driven PdM systems leverage learned and real-time sensor data to associate patterns with normal and faulty operations. ML and DL techniques are used to infer the relationship between measured signals and equipment health directly from the data. Diverse studies have shown that data driven PdM is particularly well suited to modern industrial environments, where high resolution sensor data is readily available and operating conditions are constantly adjusted [12].

Within the scope of PdM, it is critical to distinguish between diagnostic and prognostic decision making. Diagnostic approaches aim to detect, classify or localise faults once atypical behaviour has developed. Common diagnostic outputs include binary indicators, fault class labels or confidence scores based on deviation from normal operation. On the other hand, prognostic approaches seek to estimate the future evolution of degradation, often in the form of remaining useful life (RUL) predictions or time-to-failure estimates [13]. While the prognostic approach toward predictive maintenance offers greater long term planning capabilities, it also requires extensive historical failure data, consistent degradation trends and accurate dataset labelling.

The preceding conditions are rarely satisfied in early stage or small scale industrial deployments. As a result, many practical PdM implementations such as this research prioritise robust diagnostic capability as the critical operating point. This is particularly the case in scenarios where fault data is scarce and failures are rare by design. Diagnostic anomaly detection systems provide immediate value to the customer by enabling early fault awareness, rapid intervention and reduction of catastrophic failures. Furthermore, employing a diagnostic system lays down the fundamental groundwork for future prognostic extensions. The presented project is positioned as a data driven, diagnostic predictive maintenance system, focused primarily on real time anomaly detection from vibration data. The emphasis on diagnostic inference enables reliable deployment under limited data availability and strict real time constraints, while remaining compatible with future expansions towards prognostic modelling.

2.3 Sensing Modalities for Data-Driven Predictive Maintenance

A crucial design decision in any predictive maintenance system is the selection of sensing modalities used to characterise the health status of the asset. Commonly used sensor types in industrial environments include vibration, temperature, electrical current and oil or lubricant analysis [14]. Each modality gives insight into a different perspective on the machine's condition and is sensitive to different classes of faults. Among these, vibration analysis emerged as one of the most widely adopted and information rich sensing approaches, particularly for rotating machinery such as electric motors, pumps, gearboxes and bearings [14]. Mechanical faults such as imbalance, misalignment or bearing wear often manifest as subtle changes in vibration patterns prior to any catastrophic failures. Consequently, vibration-based monitoring provides a rich, consistent and attainable pipeline of information for early fault detection. Comprehensive reviews of PdM systems frequently identify vibration signals and waveforms as the primary source of diagnostic information due to their high sensitivity in regards to mechanical degradation [15].

In contrast, temperature and electrical current measurements often provide more indirect indicators of machine health. Temperature sensors are effective for detecting severe faults such as excessive friction or thermal overload, which typically occur after significant mechanical degradation has already developed. Consequently, as recommended by Mobley, temperature based monitoring alone is generally less suitable for early fault detection and is more appropriately employed as a complementary sensing modality rather than a primary diagnostic tool [16]. In addition, motor current signature analysis (MCSA) can reveal load variations, electrical faults or mechanical issues through changes in the motor current waveform [17]. The referenced study demonstrates that MCSA is a consistent and powerful diagnostic tool in industrial applications. However, motor current signals are influenced by control mechanisms, load conditions and current supply fluctuations, which complicates reliable signal acquisition. Furthermore, the sensing hardware, signal conditioning requirements, and computational overheads associated with high resolution current analysis introduce too much complexity. This renders MCSA impractical for real time deployment on resource-constrained edge devices.

From a data driven perspective, vibration signals are well suited to ML and DL approaches. Accelerometers provide high-resolution time-series information containing both time-domain and frequency-domain characteristics which can be utilised by learning algorithms. Additionally, vibration signals enable the application of more advanced signal analysis techniques to be used such as empirical mode decomposition (EMD) [18].

EMD and related techniques can provide valuable insights during offline testing and analysis, but their direct implementation on an embedded system is generally impractical. Classical EMD algorithms are computationally intensive, iterative and require dynamic memory allocation, making them poorly suited for real time embedded execution. As a result, such techniques are more appropriately used during exploratory analysis, anomaly label generation or offline training, rather than being used in the deployment pipelines.

In conclusion, vibration sensing is the chosen primary sensing method for the proposed predictive maintenance system. By focusing on accelerometer based vibration monitoring, the system is capable of capturing early mechanical degradation signals while remaining compatible with real time, embedded deployment. Accelerometers are low-cost, compact and capable of high-frequency sampling, making them ideal for edge based implementations where power, memory and computational resources are constrained. Furthermore, using vibration signals naturally aligns with deep learning anomaly detection approaches, where the model can learn normal and anomalous operating signals directly from raw sensor data. As such, vibration sensing provides a practical and technically justified foundation for the predictive maintenance system developed in this research.

2.4 Classical Machine Learning for Predictive Maintenance

Classical machine learning methods have been increasingly adopted for predictive maintenance applications. Early data driven PdM systems follow a structured pipeline consisting of signal acquisition, preprocessing, feature extraction, feature selection and finally classification or regression. Within this framework, some techniques such as regression models, support vector machines (SVMs), k-nearest neighbours (kNN), decision trees and clustering algorithms have demonstrated promising results in fault classification and anomaly detection [12] [15].

In vibration based PdM, classical ML approaches rely on a specialised workforce where feature engineers manually derive features from time-domain, frequency-domain, or time-frequency representations of the acquired signal. Typical features include root mean square (RMS), peak-to-peak amplitude, kurtosis, skewness, jerk or more computationally heavy features such as wavelet energy. The features are then carefully selected on a case by case basis in order to best capture known fault characteristics and are then used as inputs into ML classifiers or regressors [19]. When carefully selected and tuned, with redundant or weak features omitted, such feature-based pipelines can achieve high diagnostic accuracy in controlled environments. Empirical evidence suggests that classifier performance is highest when the number of input features remains relatively small, with classification accuracy deteriorating as feature dimensionality increases [20].

Despite the effectiveness of these systems, classical ML PdM systems possess inherent limitations that restrict their applicability in real world industrial applications. A primary challenge lies within the reliance on domain expertise for feature engineering. Feature sets must be specifically tailored to each machine as they have different fault types and operating conditions, making them difficult to generalise across different assets or production environments. Zhang *et al* [15] identify that a models ability for migration and generalisation is a key practical challenge, noting that PdM models should be transferable to other equipment in order to improve applicability and reduce unnecessary time consumption. Another limitation of classical ML approaches is their sensitivity to non-stationary operating conditions. Variations in load, speed and environmental conditions can alter vibration patterns, causing feature characteristics to shift overtime. As a result, models trained on historical data may fail to maintain reliable performance under changing conditions [21]. This issue is particularly problematic for an embedded system solution, where continuous retraining and manual feature adjustments are impractical.

From an embedded deployment perspective, classical ML pipelines introduce computational complexity due to the need for explicit feature computation prior to inference. Common signal and current processing techniques such as spectral analysis or the previously mentioned MCSA impose non-trivial computational overheads and memory requirements. While effective in cloud-based systems, these requirements can conflict with the real-time aspect of the system and the limited resources available on embedded edge devices. Therefore, there exists a trade-off between the richness of features and deployability when classical ML approaches are applied to edge-based PdM [22].

In conclusion, classical machine learning methods have played an important role in the evolution of data driven maintenance systems. ML models continue to provide valuable benchmarks for fault diagnostic tasks and have seen great results in assisting deep learning models using techniques such as weakly supervised deep learning. ML models are dependent on handcrafted features, have limited generalisation capability and are sensitive to varying operating conditions, which limits their suitability for scalable, real-time, embedded implementations. These limitations have motivated increased interest in deep learning approaches which aim to classify faults directly from raw sensor data, reducing the reliance on manual feature engineering.

2.5 Deep Learning

Unlike traditional ML pipelines, DL models enable end-to-end learning directly from raw sensor signals without the need for precise, manual feature engineering. By learning feature representations and decision boundaries, DL models reduce dependence on domain specific feature engineering, which is a core limiting factor associated with ML models. Furthermore, with sufficient data and training, DL models have improved robustness to noise, can work under non-stationary operating conditions and can recognise more complex fault signatures [23].

2.5.1 Convolutional Neural Networks

In vibration-based PdM, DL architectures have demonstrated their effectiveness due to their ability to model non-linear time-series data. Convolutional Neural Networks (CNNs) are particularly well suited to vibration-based PdM due to their ability to learn localised patterns directly from raw time-series data. In one-dimensional CNN (1D-CNN) architectures, vibration time series data is processed using convolutional kernels that are overlapped with the input signal at successive time intervals. At each position, the kernel output is computed as a sum of element-wise products between the kernel weights and the corresponding segment of the input (i.e., a sliding dot-product operation). During training, the kernel weights adapt and optimise so that different filters respond strongly to informative local patterns in the signal, such as transient spikes which might indicate mechanical degradation. These optimised kernels and convolution layers act as learned feature extractors. After convolution, a pooling layer (e.g., max pooling or average pooling) reduces the signal's temporal resolution by summarising each short time region into a single value. This provides a much more compact representation and decreases the model's sensitivity to small timing shifts, while preserving the most important responses produced by the filters. By further stacking multiple convolution layers, the neural network is able to form a hierarchical representation. With these additional convolution layers, the network can learn increasingly abstract patterns in the machine's behaviour at the cost of computational complexity [24].

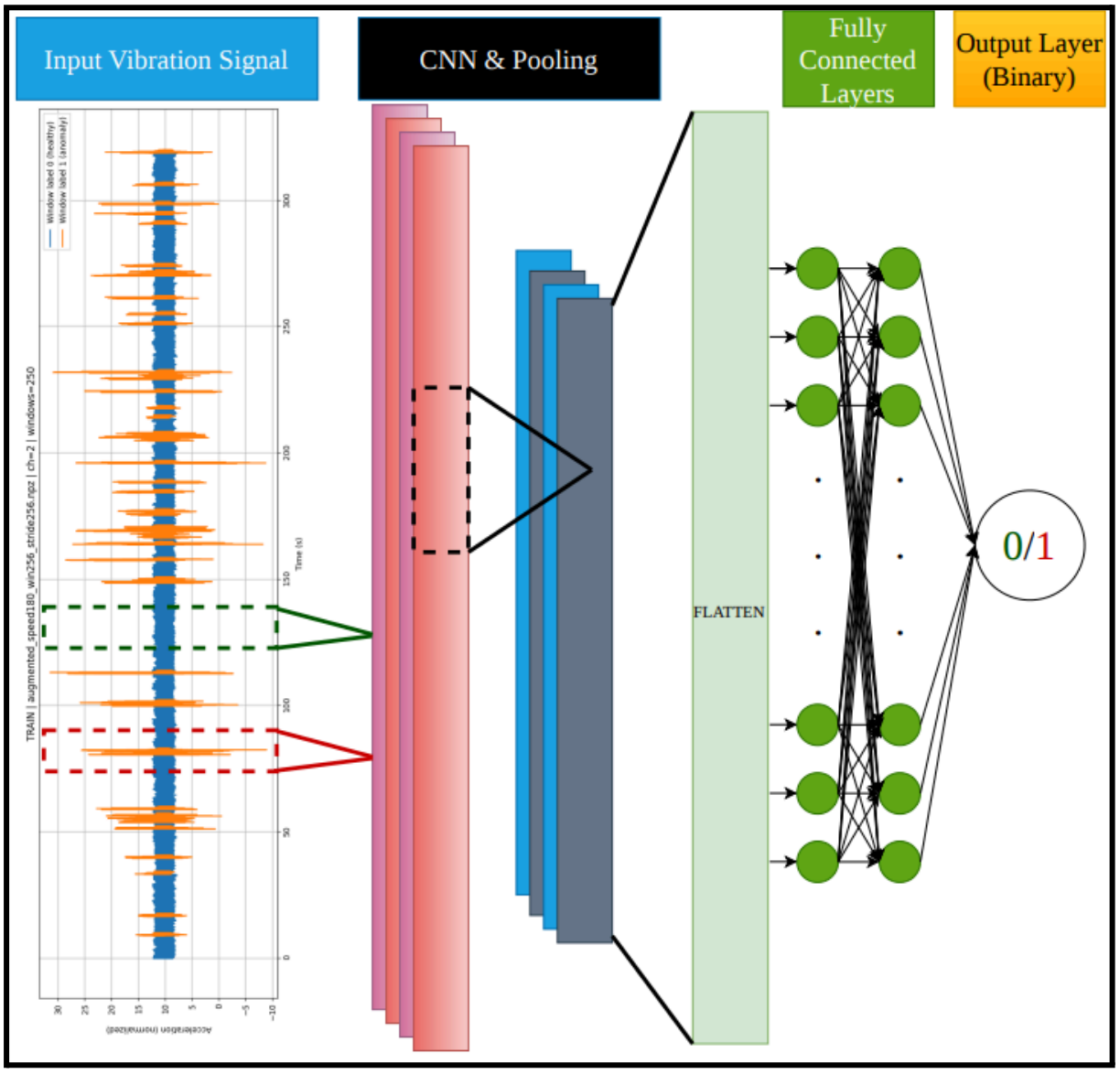


Figure 2.1 Example of a simple CNN Model

It is also important to inspect the model's complexity. When using CNNs, the model's weights and parameters are trained offline in Python and then deployed to the ESP32, where inference consists of only forward pass convolutions and activation operations. As a result, the inference engine does not perform gradient backpropagation during deployment. This means it is possible to control the size and complexity of the model by adjusting kernel dimensions, filter counts and layer depth. In contrast, recurrent architectures such as LSTMs introduce hidden state propagation during inference. At each timestep, internal gate computations and state updates must be performed, which increases memory requirements and computational overheads. These recurrent operations increase structural complexity and can potentially lead to oversizing of the model [25].

From a PdM design perspective, CNNs offer several advantages that are directly related to this research. First, CNN-based models eliminate the need for manual feature engineering, improving generalisation and reducing reliance on domain-specific feature tuning. Second, CNNs are computationally efficient with bounded and predictable complexity relative to recurrent architectures. These properties make CNNs particularly well suited for real-time inference on embedded systems, where memory and processing resources are a major constraint. Given that CNN-based approaches have demonstrated superior performance under noisy and variable operating conditions when compared to classical ML methods [15] [22], their use is well justified for the vibration-based anomaly detection task addressed in this project.

2.5.2 Long Short-Term Memory

Recurrent Neural Networks (RNNs) and in particular Long Short-Term Memory (LSTM) architectures have also been explored for PdM due to their ability to model temporal dependencies across extended periods of time. Unlike CNNs, which operate on fixed-size local receptive fields, LSTMs incorporate gated memory cells that regulate the flow of information over time. These gates are what enable LSTMs to retain relevant historical information while selectively forgetting redundant data. This method allows the neural network to capture slow degradation trends and any long lasting temporal correlations in vibration signals. As a result, LSTMs have been shown to perform well in applications involving fault progression and RUL estimations [26].

However, critical to this research, LSTM-based models introduce additional computational complexities and memory overheads, which can be problematic for embedded real-time deployment. Sequential, recurrent processing and internal state maintenance of the memory gates increases inference latency and complicates deterministic timing on resource-constrained hardware. For this reason, while LSTMs provide valuable modelling capabilities, their direct deployment on low-power edge devices requires careful optimisation and trade-offs making LSTM models a secondary choice in this research.

In the scope of this project, CNN-based architectures are therefore selected as the primary DL approach due to their favourable balance between diagnostic performance, computational efficiency, and real-time feasibility on embedded platforms. CNNs enable the effective learning of characteristic vibration patterns while remaining compatible with the constraints of edge-based deployment. Nevertheless, hybrid CNN-LSTM architectures remain a relevant extension of PdM, where the CNN layers perform local feature extraction and LSTM layers model longer-term temporal dependencies. Such hybrid architectures have been reported to improve performance in scenarios involving complex temporal fault evolution, and may be explored in future work once baseline CNN performance has been developed and verified [27].

2.5.3 Supervised vs Unsupervised Learning

Unsupervised and self-supervised learning approaches have been increasingly investigated for predictive maintenance because they reduce dependence on labelled fault data. In these frameworks, models are trained primarily on normal operating behaviour and then identify anomalies as deviations from the learned baseline. Common strategies include reconstruction-based methods, like autoencoders with one-class classification objectives. For example, deep one-class methods such as Deep SVDD learn a representation that encloses normal samples within a compact hypersphere, enabling anomaly detection without explicit fault labels [28]. More recently, self-supervised learning techniques for time-series data have gained attention. They allow models to learn useful patterns directly from unlabeled sensor datasets. These learned representations can then be used downstream for tasks such as anomaly detection with the reduced need of labelled data [29].

Despite the promising appeal, fully unsupervised PdM introduces practical challenges that are outside the scope of this research. Firstly, model evaluation and benchmarking becomes non-trivial without reliable ground-truth fault labels, often requiring synthetic anomaly injection or expert evaluations. This can severely weaken the validity of performance claims. Secondly, unsupervised anomaly scores typically require careful thresholding and drift management to avoid excessive false positive triggers under changing load, speed or sensor placement. Finally, several unsupervised pipelines rely on additional processing stages like clustering or adaptive thresholding, which can increase computational overheads and complicate the goal of deterministic real time deployment. Accordingly, this project focuses on supervised/weakly-supervised diagnostic anomaly detection using controlled normal/anomalous datasets to enable a transparent, bias-free evaluation of the system and reliable embedded deployment. That being said, unsupervised and self-supervised approaches remain a strong direction for future research, particularly to reduce the burden associated with labelling data.

2.5.4 Data Requirements and Practical Limitations

Despite the promising future directions and diagnostic performance demonstrated by deep learning models, their practical applications in predictive maintenance systems are fundamentally constrained by data availability and data quality. DL models normally require large volumes of labelled training data in order to generalise well over different assets, working conditions and varying loads. In the industrial world, this requirement presents a significant challenge, as fault events are rare by nature. Machines are specifically designed to operate reliably, and any possible faults are actively avoided through maintenance interventions. As a result, available datasets are often heavily imbalanced and dominated by normal operating conditions, with limited examples of anomalous behaviour [30].

In addition to class imbalance, industrial vibration datasets suffer from variability in operating conditions, sensor placement, machine configuration and noise from other machinery. These factors make the data non-stationary, which makes it difficult for models trained under one set of conditions to generalise reliably to unseen scenarios. While data augmentation, transfer learning and semi-supervised learning techniques have been proposed to mitigate these issues, they do not fully eliminate the underlying dependency on high quality, representative training data [22], [30]. Consequently, the effectiveness of DL based PdM systems is closely tied to careful data collection strategies, controlled anomaly generation and labelling, and robust validation procedures.

For this project, data limitations directly influence model selection and system design. Rather than reaching for large-scale fault classification or over-time prognostic modelling, the proposed system focuses solely on diagnostic anomaly detection, using short, high-frequency vibration windows collected under controlled normal and anomalous operating conditions. This methodology reduces the need for extensive fault labelling and allows the model to learn deviations from baseline behaviour, making it far more feasible in small-scale, early-stage deployments.

Besides data requirement limitations, there are further constraints that are particularly relevant to the deployment of deep learning models to embedded PdM systems. Many DL based PdM solutions reported in literature rely on cloud-centric architectures, where raw sensor data is streamed from sensors to remote servers for storage, processing and inference. While this approach provides access to scalable computational resources and memory, it also introduces unwanted downsides such as latency, network bandwidth requirements, reliability concerns and data privacy risks [31], [32]. The limitations associated with cloud-centric pipelines are especially problematic in time-critical maintenance applications, where rapid anomaly detection and local decision making are required to prevent catastrophic asset damage.

These challenges have motivated the development of edge artificial intelligence (Edge AI), where ML and DL models are deployed directly onto embedded devices located on or near the monitored asset. Edge AI introduces its own set of constraints, such as limited memory and reduced computational capacity, which necessitate careful model selection, design and optimisation. Therefore, the following section examines Edge AI architectures, their advantages over cloud-based systems, and the tradeoffs associated with deploying DL models on resource constrained platforms.

2.5.5 Transfer Learning

In response to limited data availability, transfer learning has emerged as a promising strategy for predictive maintenance applications. Transfer learning involves pre-training a neural network on a large source dataset and then fine-tuning the model on a smaller, task-specific basis [33]. In vibration based fault diagnosis, pre-trained CNN architectures can be adapted to new machinery using significantly fewer labelled samples, instead of working from scratch [22], [27]. Several studies have demonstrated that transfer learning improves generalisation across operating conditions and reduces the data burden found in industrial settings [30].

Despite these advantages, transfer learning introduces additional system complexity, including the requirement for compatible source datasets, architectural adaptation, and careful fine-tuning to avoid negative transfer effects. Furthermore, when targeting low-cost embedded deployment, large pre-trained models may exceed memory constraints unless heavily compressed or restructured. As a result, while transfer learning represents a valuable direction for scaling PdM systems across multiple assets and machines, the research focus is on developing a fully standalone diagnostic framework trained directly on locally collected data. The approach in this project is on prioritising architectural simplicity, deterministic deployments, and independence from external pre-trained models, making it more suitable for early-stage and small-scale industrial applications.

2.6 Edge Artificial Intelligence

As identified, the practical development of DL based PdM systems is constrained not only by data availability, but also by latency, bandwidth, reliability and privacy considerations. Edge AI directly addresses these limitations by relocating data processing and model inference from remote servers to embedded devices situated directly at the monitored asset. Walani and Doorsamy demonstrate that edge-based monitoring systems can achieve diagnostic performance comparable to cloud-based solutions while significantly reducing latency and network dependencies [32]. The shift from cloud-based solutions to Edge AI is especially beneficial for time-critical applications, such as this project, where immediate fault awareness is required and model accuracy needs to stay adequate.

2.6.1 Latency, Bandwidth and Reliability

For vibration based PdM, latency directly affects the effectiveness of fault detection. Latency refers to the total time delay between data acquisition at the sensor and the availability of a diagnostic decision. In cloud-based PdM pipelines, latency can arise from the sum of multiple different sources, such as sensor buffering, data transmission, internet routing delays, cloud server queuing, and remote model inference time. Mechanical anomalies such as bearing defects or misalignment can evolve rapidly, sometimes within milliseconds or seconds. If detection was to be delayed due to network latency, the faults may result in catastrophic or collateral damage. Edge AI serves as a possible solution, where it enables near-instantaneous inference by removing network related delays and by allowing anomalies to be detected in real time [31], [32].

Bandwidth limitations associated with high-frequency accelerometer data further motive edge deployment. Bandwidth refers to the maximum rate at which data can be transmitted over the network and directly constrains how much sensor data can be sent to the cloud within a given period of time. High-frequency accelerometer data generates large volumes of data due to high sampling rates and multi-axis measurements. This data intensity presents scalability challenges for cloud-based PdM architectures [15], [22]. Continuously streaming such high volumes of data to the cloud is impractical in industrial environments, particularly when multiple systems access the same network infrastructure. Moreover, bandwidth usage incurs operational costs associated with the services required. These costs scale rapidly when monitoring multiple assets, making cloud-only solutions economically inefficient. By processing data locally, edge-based systems help reduce bandwidth requirements and associated costs while still retaining high-level diagnostic outputs [31], [32].

Reliability is another critical factor to be considered when reviewing cloud-based PdM systems. Cloud-based solutions are vulnerable to network outages, latency spikes, or service disruptions, which can disable monitoring capabilities, leading to a system in which there is a single point of failure [11], [34]. Edge-based systems maintain operational continuity even during network failures, ensuring that critical diagnostic functions remain active at all times [31].

2.6.2 Constraints and Trade-Offs

Despite the advantages, Edge AI introduces a new set of constraints that must be carefully evaluated and managed. Embedded platforms such as the ESP32-S3 are typically limited in terms of memory, processing power and energy consumption, especially when compared to cloud connected computers. As a result, DL models, trained on powerful computers, when deployed at the edge must be specifically designed, optimistic and evaluated to meet strict real time performance requirements.

Model size, computational complexity and inference latency all become critical design parameters. Techniques such as quantisation, pruning, reduced numerical precision have all been proposed to enable efficient deployment of neural networks on microcontrollers [34], [35]. However, optimisations like these often involve tradeoffs between model accuracy and deployability, requiring careful evaluation during the design of the model.

Vermesan *et al*. demonstrate that modern Industrial Internet of Things (IIoT) devices are capable of executing real-time classification tasks with high accuracy while under heavy memory constraints, provided that the models are appropriately optimised [34]. These findings highlight that Edge AI is not a theoretical concept but a practical and increasingly more mature solution for real industrial applications.

2.7 Vibration Based Activity Recognition

Even though vibration-based predictive maintenance targets mechanical assets rather than humans, human activity recognition (HAR) provides a closely related state-of-the-art research stream. Both research topics involve wearable/embedded inertial sensors, high-rate time series signal data, and the need to infer states from noisy acceleration patterns. A representative example is the work done by Soro *et al*. [36]. The research presents an end-to-end deep learning pipeline for recognising and counting repetitions of 10 complex full body exercises using raw inertial signals from smartwatches.

Soro *et al.* demonstrate that CNN-based models can operate directly on raw acceleration time series data without any manual feature engineering, and achieve near perfect recognition performance in real time [36]. This is directly relevant to vibration based PdM because the same core signal properties apply here. The informative patterns are often local and transient, signal quality varies due to placement and external noise, and robust inference requires careful windowing, smoothing, and validation across subjects or in this case assets. The paper therefore functions as a state-of-the-art piece showcasing how deep networks can learn discriminative representations from raw data streams, which further supports the decision choices made in this project to use CNNs on raw vibration windows rather than handcrafted condition indicators.

A major strength of this study is the complete pipeline framing consisting of raw sensor streams being segmented into overlapping windows, passing through a CNN classifier, and then being stabilised via majority voting and smoothing to reduce misclassifications [36]. This is aligned with industrial vibration PdM practice, where inference is typically performed over fixed length windows and outputs are post processed to avoid false alarms triggered by short disturbances.

Another strength of this paper is the treatment of data acquisition and labelling. Data requirements have already been deemed a major practical barrier in generalisation and real deployment. The authors explicitly acknowledge that obtaining accurately labelled time-series sensor data is difficult. In the paper, the proposed method to gather quality data was a protocol where smartwatches were used to provide a vibration cue that marks repetition start events, enabling automatic annotation of exercise class and repetition boundaries [36]. This reinforces an important principle for PdM, where if real fault examples are rare by design, then controlled operating protocols (or controlled anomaly injection) can be used to generate high-quality, accurate labelled data.

Furthermore, the study provides concrete, engineering-relevant design choices that transfer well to embedded vibration monitoring. The authors discuss; (i) sampling irregularity and resampling, where signals were acquired at irregular intervals then had to be interpolated or resampled to match a static 100 Hz; (ii) the importance of window length selection, where multiple window sizes were evaluated for better generalisation in faster exercise sequences and; (iii) robust evaluation schemes that avoid optimistic leakage and data snooping by ensuring each participants data is entirely in train or test splits [36]. These design choices tackle challenges which are analogous to the ones found in vibration based PdM, where sampling jitter, window length, and leakage across runs can strongly influence the results.

Despite its strong performance, the work has several limitations when viewed through an industrial PdM lens. First, the pipeline is explicitly evaluated offline, the authors state that computation is performed offline on GPUs and that adaptation to on-device/online inference is left as future work [36]. Consequently, the paper does not address the embedded constraints which are central to this research, such as deterministic timing, memory constraints, quantisation effects, and sustained inference at fixed sample rates.

Second, the data collection step includes a constrained workout designed to simplify the labelling stage. While for their work, it yielded clean labels and high accuracy, the authors also show that generalisation to unconstrained exercises introduces failure modes, such as reduced repetition counting accuracy for certain movements [36]. This is an important caution for PdM, since models trained on clean or artificially structured data can degrade in performance when deployed into more realistic operating regimes, motivating robust validation and conservative claims of generalisation.

Finally, the task in the researched paper is closed-set classification of exercise type and repetition starts, whereas industrial PdM frequently involves open-set conditions of unseen fault modes, changing loads, and non-stationary sensing across assets. As a result, while the paper strongly supports CNN-based recognition from learned inertial data, it does not directly solve the PdM problem of detecting novel or emerging anomalies under varying operational conditions.

This thesis leverages the work of Soro *et al*. primarily as a state-of-the-art justification and methodological template rather than as a deployable embedded solution. The paper supports the use of CNNs to learn discriminative patterns directly from accelerometer streams, motivating the decision to avoid handcrafting features. The use of overlapping windows, majority voting, and smoothing provides a validated approach for improving robustness in streaming inference and supports real time anomaly decision logic. The constrained protocol used towards data collection demonstrates how engineering the data collection process can be as important as the deep learning model itself, aligning with this project's controlled data logging and anomaly generation strategy.

2.8 Conclusion

In essence, the literature reviewed highlights a clear progression from traditional maintenance strategies towards more intelligent, data-driven approaches, implemented by using ML or DL. Among the various sensing modalities explored in the literature, vibration monitoring emerged as one of the most informative and widely used approaches for detecting early signs of mechanical degradation in rotating machinery. Advances in sensor technology and ML have further enabled the development of PdM systems for anomaly detection.

Deep learning approaches, particularly CNNs, address many limitations found in classical machine learning. CNNs learn directly from raw vibration signals, however, their practical deployment introduces new challenges. Data scarcity, non stationary operating environments, and computational constraints associated with embedded systems all need to be addressed prior to deployment.

The emergence of Edge AI provides a promising pathway to overcome limitations associated with cloud-based PdM. Achieving reliable anomaly detection under constrained memory, processing and data availability remains an open engineering challenge. These limitations motivate the research presented in this thesis, which focuses on the development of a lightweight, embedded PdM system capable of performing vibration based anomaly detection using locally collected datasets.

2.9 Research Gaps

The key research gaps identified in the literature relate to the practical development of PdM systems. Many existing approaches rely on transfer learning from large historical datasets or the need to collect such data, which may not be available in small scale or startup industrial settings. This research addresses this gap by demonstrating a fully standalone PdM architecture that can be trained locally and deployed directly onto the asset.

Additionally, while numerous studies investigate individual components of PdM pipelines, such as signal processing or machine learning, few works demonstrate complete end-to-end system integration. By integrating sensing, signal processing, deep learning and lightweight IoT communication into a single embedded device, the research displays full system integration.

Finally, this work addresses the role of Edge AI to decentralise industrial monitoring from cloud dependant services. The proposed system minimises data transmissions, reduces network dependency and lowers operational costs. Collectively, this work contributes a practical and reproducible framework for implementing PdM in resource constrained applications, supporting a wider adoption of intelligent monitoring technologies.

Chapter 3: Ethical Considerations

Engineering solutions that combine sensing technology, machine learning and network connectivity introduce ethical responsibilities along with technical challenges. Predictive maintenance systems influence operational decisions, interact with sensitive industrial data and may be deployed in environments where reliability, safety and confidentiality are crucial. For these reasons, professional engineering bodies emphasise that technical competence alone is insufficient without ethical responsibility [37]. Accordingly, ethical considerations form an important step in the system design process in this paper.

This chapter examines the ethical implications that are associated with the development and deployment of the proposed PdM system. The discussion focuses on several key areas including potential dual-use, risks of commercial misuse, protection of operational data, sustainability benefits and the importance of transparency in AI driven systems. Importantly, confirmed by a popular case study, these considerations are not treated as theoretical concerns, instead they influence the design decisions within the project.

3.1 Stakeholder Impact

Maintenance engineers, machine operators and factory plant managers all benefit from improved visibility of machine health. The proposed PdM system enables proactive maintenance to improve job satisfaction and safer working conditions. By identifying early signs of degradation, factory floor workers will not be exposed to potential machine failures which could be unsafe.

Industrial organisations may benefit financially through reduced downtime, extended asset lifespan and improved production efficiency. However, reliance on automated monitoring systems may also introduce risks if the system outputs are misinterpreted or trusted without trained operator oversight.

Local communities may benefit from improved industrial reliability. Reduced catastrophic failures may lower noise pollution and material waste associated with damaged machinery.

Future engineers and researchers form another stakeholder group. By developing a reproducible and transparent framework, this project contributes to shared technical knowledge and reduces the duplication of research efforts.

3.2 Reliability & Human Oversight

Machine learning models inherently involve some degree of uncertainty [38]. PdM algorithms may generate false positives, or false negatives and both scenarios introduce ethical concerns.

A false positive occurs when the system incorrectly classifies normal machine behaviour as anomalous which could lead to unnecessary downtime, maintenance and financial costs. On the other hand, a false negative occurs when an actual fault is not detected by the system. An unnoticed fault could cascade into further damage and completely undermine the entire point of the PdM system. The ethical consideration here is that the system is presented as a solution to unplanned downtime or catastrophic damage, but if it doesn't work as intended, it could cause more damage or more unplanned downtime.

To mitigate these risks, the proposed PdM system is designed to be a decision support tool rather than an automatic maintenance controller. The system provides anomaly indicators and diagnostic probabilities, while the decision needs to be made by a trained human operator. This approach aligns with professional engineering practice by ensuring that automated systems support expert judgment and not replace it entirely.

3.3 Dual-Use Technology and Military Applications

The system designed in this research is for predictive maintenance of rotating machinery using vibration monitoring and Edge AI. The intended application of this system is industrial asset monitoring for civilian use in environments such as manufacturing facilities, small-scale industrial plants and laboratory equipment. However, like many sensing and diagnostic technologies, the system may be considered a dual-use technology [39].

Condition monitoring systems could be theoretically applied to maintain equipment in military settings. For example, PdM could be used to monitor the motors and actuators within unmanned aerial vehicles (UAV) such as drones or other defense devices. While the presented work is not designed for military purposes and contains no features specifically intended to aid in warfare, the possibility of secondary use cannot be disregarded [40]. The system presented in this research focuses solely on industrial reliability, safety and sustainability, and any application in military contexts would fall outside of the intended scope.

3.4 Commercial Misuse

Another ethical concern relates to the misuse of diagnostic information for financial gain. PdM systems produce information regarding the current health state of machines, and these outputs influence decisions regarding maintenance [38]. For example, third party maintenance companies could misuse this system by falsely claiming that a machine requires servicing or replacement when that is not the case. Such situations could allow companies to justify unnecessary repairs or component replacements while shifting the blame to the monitoring systems.

To mitigate these risks, the proposed PdM system should include traceable model outputs and clear interpretation guidelines.

3.5 Transparency and Reproducibility

A modern ethical consideration surrounding the use of AI systems is the concept of the “black box” problem, where model decisions cannot be easily interpreted or reproduced. Lack of transparency can reduce trust in AI driven systems, particularly in safety critical applications.

This project addresses the black box problem by providing detailed documentation of the system architecture, data processing pipeline and model development methodology. The experimental setup, signal processing steps, and machine learning models are all described in sufficient detail to allow the work to be reproduced. By ensuring this level of transparency and reproducibility, the system avoids being deemed as an opaque decision making tool, and instead its a verifiable engineering solution.

Providing access to the methodology and implementation details also supports sustainable engineering practice. When systems are documented and reproducible, future researchers and engineers can build on this existing work rather than duplicating development efforts. This approach promotes more efficient use of research resources and encourages collaboration within the field.

3.6 Data Confidentiality

PdM systems contain sensitive operational information. Continuous monitoring of machines may indirectly expose data related to production schedules, machine usage, and overall factory throughput. If such information was obtained by competitors, it could provide insight into operational strategies and production capacity. This ethical issue is particularly relevant in modern day industrial environments where production efficiency and scheduling are very optimised. Consequently, any deployment of live PdM systems should consider secure data handling practises, controlled access to data and protection of operational information.

This issue led to a key design choice in this project. The use of Edge AI means that data processing and inference occur locally on an embedded device rather than in a remote cloud environment. Vibration signals are analysed locally, and raw sensor data does not need to be transmitted or stored on external servers. This significantly reduces the exposure of sensitive operational data. Additionally, anomaly prediction values, alarm states and timestamps transmitted over MQTT are protected behind a username and password system.

3.7 Manufacturing Sustainability

PdM technology can positively contribute to sustainable engineering practices. By detecting anomalous operations in machinery at an early stage, maintenance can be performed before catastrophic damage has occurred. This reduces material waste associated with premature component replacement and lowers the environmental footprint of industrial operations.

In addition, the proposed monitoring system is designed as a solution that can, in theory, be installed onto a wide range of existing assets. This promotes sustainable principles by allowing for the continued use of legacy machinery, rather than encouraging equipment replacement. The portability and standalone nature of the embedded device also allows it to be reused across different assets as monitoring requirements change. This level of flexibility contributes to the lifecycle of old and new industrial assets.

The embedded architecture used in this project also supports sustainability objectives. Edge based inference reduces dependence on cloud computing, which lowers communication energy requirements. This is particularly relevant in modern days, since data centres represent a significant and growing component of global electricity consumption. Reports from the International Energy Agency (IEA) indicate that data centres account for approximately 1-1.5% of the worldwide electricity demand [41]. By processing vibration signals locally and only transmitting essential diagnostic information, the proposed system contributes to much more energy efficient monitoring architectures.

3.8 Sustainability in Society and Economics

Widespread adoption of PdM technologies may influence workforce dynamics within industrial environments. While improving the automation within the industry can enhance safety and efficiency, it may also reduce reliance on traditional manual inspection roles. Therefore, when deploying such systems, workforce reskilling should also be considered.

Economically, low cost embedded monitoring solutions may improve competitiveness for small and medium sized companies. Previously, data driven maintenance strategies were only accessible to larger organisations given the requirement of high performance cloud computing platforms. Now, this solution is far more accessible which supports lost term economic sustainability.

Therefore, the proposed system aligns with United Nations Sustainable Development goals (SDGs). By improving industrial reliability and enabling efficient use of mechanical assets, the system contributes to SDG 9: Industry, Innovation and Infrastructure. Reduction of material waste and extension of asset lifespan supports SDG 12: Responsible Consumption and Production, while also contributing to SDG 13: Climate Action [42].

3.9 Case Study: Stuxnet

A known example of the aforementioned ethical considerations is the Stuxnet cyberattack. This attack targeted industrial control systems used in Iranian nuclear enrichment facilities. The malware was used by the hackers to modify the operating behaviour of centrifugal pumps by changing the rotational speed while simultaneously reporting normal operating conditions back to the operators. By falsifying the sensor data, Stuxnet was able to cause physical damage to equipment while avoiding immediate detection by the operators [43].

This attack is a prime example that industrial monitoring systems can be manipulated to conceal anomalous machine behaviour. It highlights the vulnerabilities in industrial control infrastructure and shows how operational data can be exploited to achieve malicious objectives. This case study illustrated the importance of ensuring that industrial monitoring technologies, such as the one presented in this research, must be designed with cybersecurity considerations and secure data handling mechanisms. Failure to protect such systems can lead to incorrect decisions, equipment damage or exposure of sensitive information.

3.10 Summary of Design Decisions

These ethical considerations discussed have directly influenced several design decisions made during the development of the predictive maintenance systems. The system architecture was structured in such a way that these concerns were minimised. These include:

- Adoption of Edge AI to minimise transmission of sensitive operational information,
- Implementation of authentication mechanisms in MQTT communication,
- Presentation of model outputs to aid in maintenance related decisions,
- Full documentation of system architecture and methodology to promote transparency and reproducibility.

Collectively, these design decisions demonstrate how the ethical concerns were used to make technical developments in the PdM system.

Chapter 4: Theory - Data Acquisition System

The data acquisition system is presented in the theory chapter because it forms the foundation upon which all subsequent experiments in this work are built. Every model version described in later chapters relies on the vibration datasets collected using the described sensing hardware, sampling strategy and logging architecture.

Reliable detection of anomalies in high-frequency vibration data is dependent on the integrity and timing consistency of the data acquisition pipeline. Vibration based PdM requires deterministic sampling at fixed intervals to preserve temporal structure, derivative-based features and enable time-frequency analysis. This section outlines the requirements of the proposed data acquisition system and justifies the decisions made to satisfy them.

The primary requirement of the data gathering system is the ability to collect tri-axial acceleration data at a fixed sampling rate of 200 Hz, corresponding to one sample every 5ms. At this frequency, any significant timing jitter, buffering delay, or network latency can distort the temporal structure of the signal and compromise the model.

4.1 Limitations of Cloud-Based Logging

Cloud based platforms such as ThingSpeak and Google Sheets API were both evaluated for remote data collection. Direct per-sample transmissions are completely unachievable. Using a standard account with ThingSpeak limits updates to one update every 15 seconds and the Google Sheets API has inherent latency constraints, relating to network stack processing on the ESP32, HTTP request headers, HTTPS transmission, server side processing, spreadsheet updating and response handling. In principle it would be possible to buffer the high frequency data locally and upload samples in periodic batches.

However, even with batching, such an approach introduces architectural inconsistency between training data acquisition and embedded deployment. While inference on the ESP32 operates at a strictly deterministic sampling rate of 200 Hz, cloud-based transmission relies on asynchronous network communication. Both cloud based methods would embed variable and non-deterministic delays into the dataset. Although the raw acceleration samples may be buffered at 200 Hz, the timing of their transmission becomes decoupled from the acquisition loop. Since temporal consistency is so critical, to ensure that the training dataset characteristics match those of those deployed inference environments, data acquisition was implemented entirely using local deterministic sampling routines.

4.2 Local Data Logging Architecture

To satisfy the requirement for deterministic high frequency sampling, the data logging system was designed to operate entirely on the microcontroller. Sampling is governed by a hardware timed loop operating at 200 Hz, ensuring consistent and repeatable temporal spacing between acceleration readings across all three axes.

Two local logging mechanisms were considered, SPIFFS (Serial Peripheral Interface Flash File System) for onboard storage and USB CDC Streaming for real-time data transfer to a host device (computer).

In both approaches, data acquisition is completely offline and decoupled from any external communication processes. Sampling, buffering and timestamp generation occur independently, ensuring that the strict temporal structure of the vibration signal is acquired. Furthermore, maintaining a fully local acquisition pipeline ensures consistency between training and deployment environments. The final embedded inference system operates under deterministic timing conditions governed by hardware timers. By collecting training data under the exact same regime, the statistical properties of the dataset remain aligned with those which will be used in the inference system. This method avoids any ambiguity or mismatches that could arise if training data were to be collected under network influence timing conditions. A clear detailed evaluation of the SPIFFS-based data logging (Iterations 1-3) is provided in **Appendix A**.

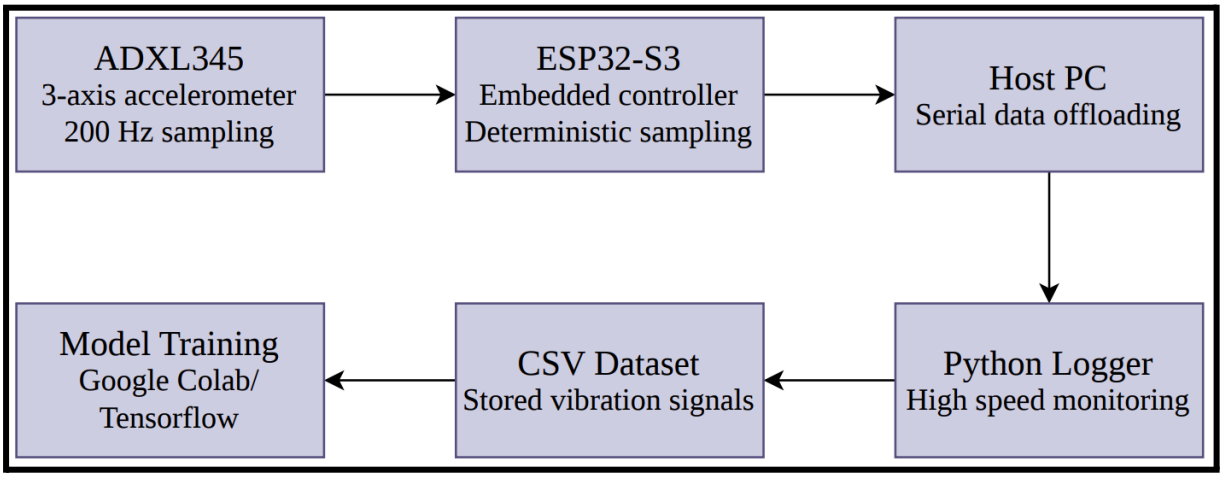


Figure 4.1. Flow diagram of local data logging architecture.

4.3 USB CDC Data Logging

The data acquisition system was developed iteratively, with each revision addressing limitations observed in earlier prototypes. Across all different versions, the measurement pipeline consisted of an ESP32-S3 sampling a triaxial ADXL345 accelerometer over I²C, with data exported as a CSV for future processing and model training.

Table 4.1. *Target Design Requirements*

1.	Deterministic sampling at a defined rate (200Hz -> 5,000 µs period)
2.	Monotonic timestamps suitable for windowing and spectral analysis
3.	Minimal timing jitter from non real-time tasks (Wi-Fi, HTTP server, file I/O)
4.	Clean CSV format compatible with Python

Iteration four was a complete overhaul of the data logging system. It replaced flash-based storage and network transfer entirely with a USB CDC (Communications Device Class) streaming approach. Instead of writing accelerometer reading samples to SPIFFS, the data was streamed directly over the ESP32-S3's native USB port to a host computer in real time.

Listing 4.1*. Build Flags*

```
build_flags =
  -DARDUINO_USB_CDC_ON_BOOT=1
  -DARDUINO_USB_MODE=0
```

USB CDC allows the microcontroller to act as a virtual serial device (USB stick), which enables high-throughput, low-latency data transmission without the need for intermediate storage. This approach eliminated any file size limitations and removed the cumulative performance degradation which was observed with the prolonged use of the on-device filesystem. Additionally, USB CDC provides a higher and more deterministic data rate than Wi-Fi based transmissions, allowing for sustained streaming of data at a constant 200 Hz with microsecond precision.

To support repeatable and user-controlled operation, a button-triggered start mechanism was implemented on GPIO4. This mechanism was configured with an internal pullup resistor and software debouncing. In the idle state, the system continuously monitored the GPIO4 input and remained in a “non-logging” mode until the button was pressed. Upon the button press, the logging state was latched, and all variables were reset. The reset ensured that each logging session began at a defined time origin and prevented partial data capture. The CSV header was transmitted once at the start of each logging session, guaranteeing consistent data formatting for downstream processing.

Visual state indicators were added for operation feedback. Two LEDs were used to represent the system's state. A red LED indicated the idle state, in which no data was being recorded, while a green LED indicated active logging.

To support reliable high-rate data capture on the host (PC) side, a dedicated Python-based logging script was developed to work in conjunction with the ESP32-S3’s USB CDC port. While typical serial monitors such as the one in PlatformIO or the Arduino IDE are suitable for debugging and low-rate data monitoring, they are not designed for sustained high-throughput data logging and can introduce buffering delays, dropped lines or UI-induced throttling. In contrast, the Python logger programme accessed the CDC interface directly by using the pyserial library, allowing for efficient, noninteractive reading of incoming data. The Python code can be found in **Appendix B**.

The script was written to open the assigned CDC port at the configured baud rate and continuously read new CSV rows using ser.readline(). Each valid line was decoded, stripped of control characters, and appended to a timestamped CSV file on the host machine. Line buffering was enabled to ensure that data was written incrementally to the file without excessive memory usage. Periodic flush operations were also performed at fixed intervals to prevent data corruption. Status lines and non-data lines were explicitly filtered out to prevent generating a dirty dataset. By offloading file I/O entirely to the host computer and using Python's efficient serial handling, this approach ensured that the full data stream produced by the ESP32 could be captured in real time without loss.

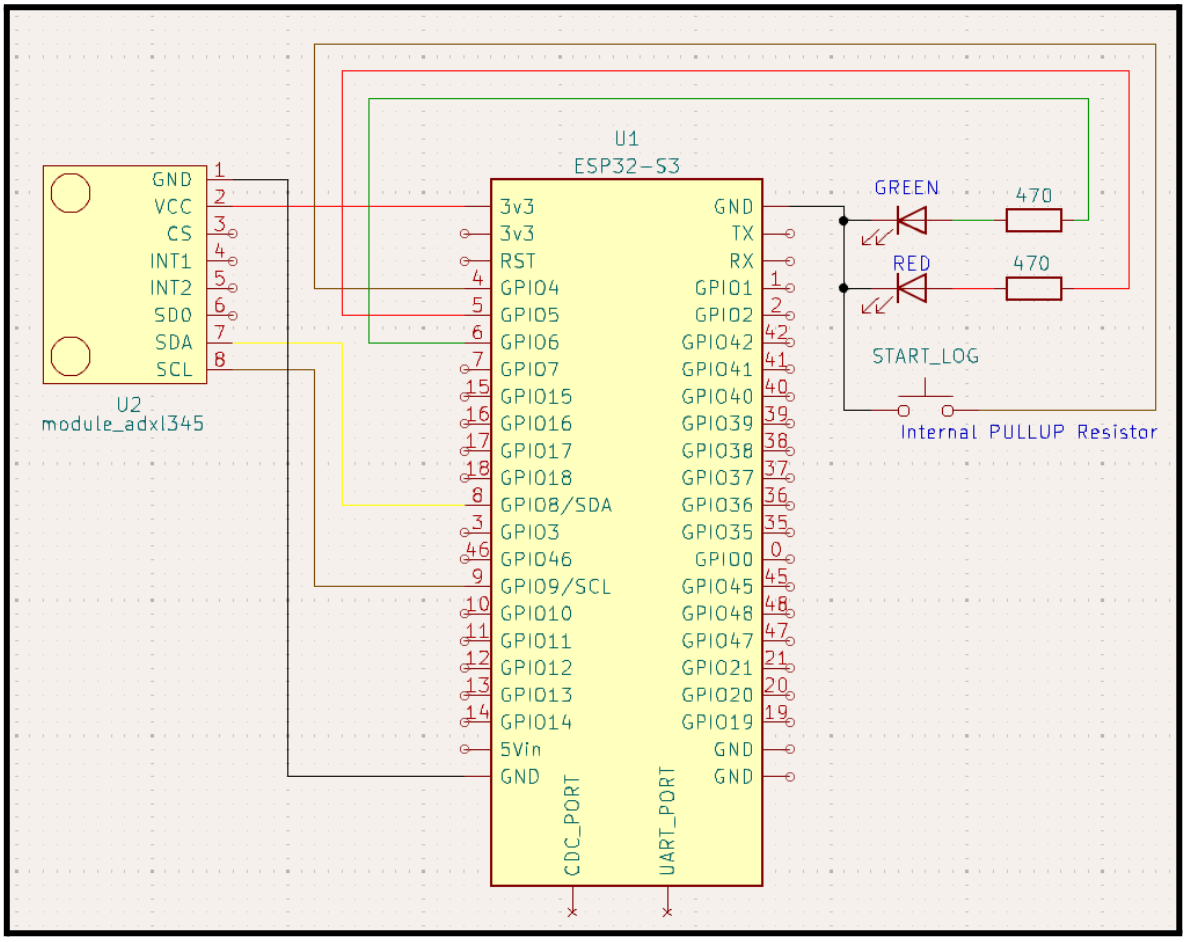


Figure 4.2 Circuit design of data logging system iteration 4.

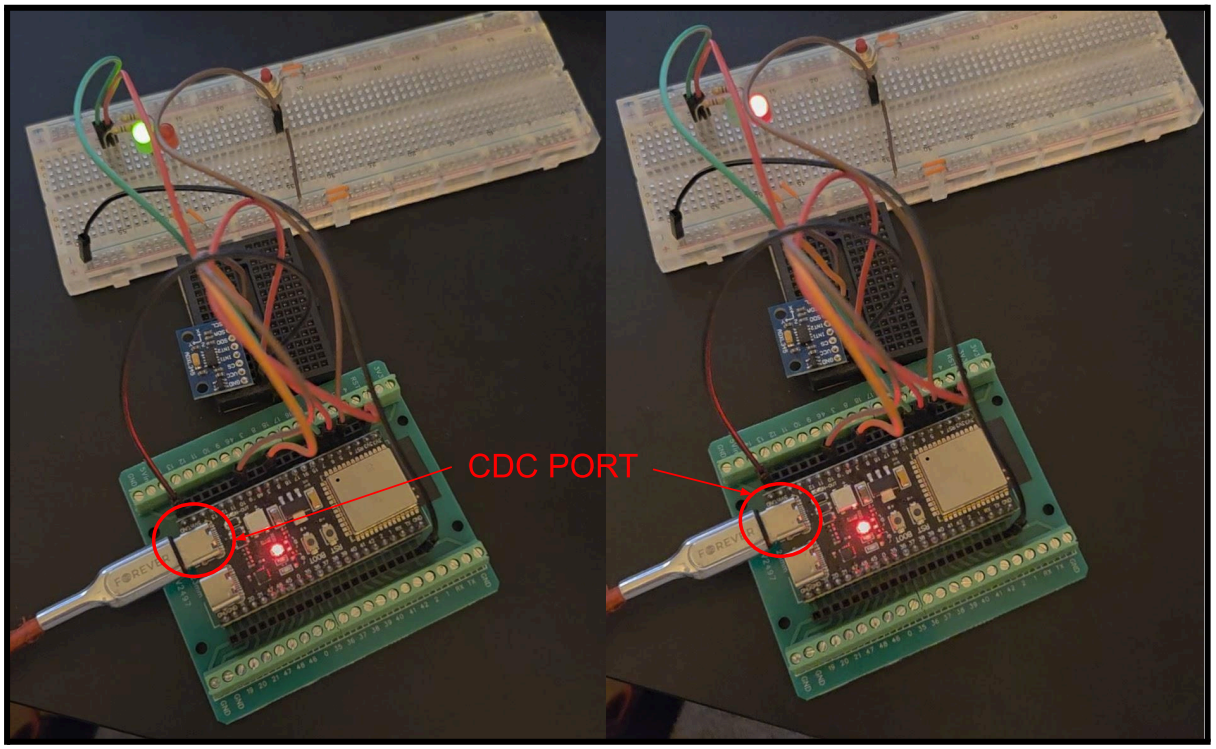


Figure 4.3 Circuit implementation of iteration 4, logging active (left) inactive (right).

Table 4.2. *Summary of Data Acquisition System Iterations (1-4)*

Iteration	Purpose	Key Improvements	Limitations
Event-driven SPIFFS logging coupled to servo motion.	Validate end-to-end functionality of the sensing pipeline.	Successful communication with the ADXL345.	Event-driven sampling produced irregular time intervals
		SPIFFS file generation.	Timestamps limited to millisecond resolution
		Basic timestamping.	Per sample SPIFFS writes introduced variable latency
		Data retrieval via Wi-Fi HTTP download.	Unsuitable for vibration analysis and fixed-window model training
Fixed-rate sampling with SPIFFS and Wi-Fi	Introduce deterministic time-driven sampling for signal processing.	Microsecond-resolution timestamps.	High per-sample filesystem overhead remained
		Fixed-rate scheduler targeting 200 Hz.	Continuous HTTP servicing latency.
		Decoupled sampling from servo logic.	Timing jitter reduced but not eliminated.
Buffered SPIFFS logging with throttled server handling.	Reduce I/O jitter and preserve deterministic sampling	Batch-buffered CSV writes in RAM, reduced flash access.	Sustained SPIFFS usage led to increasing filesystem latency overtime
		Throttled HTTP pooling, limited network interferences.	
		Added lateness handling to prevent long-term drift.	Memory management and flash housekeeping caused missed samples during long runs
USB CDC streaming with button-triggered logging and Python host logger	Eliminate storage and network bottlenecks while enabling repeatable experiments.	Removed on-device file I/O entirely.	No explicit anomaly injection or real-time labelling.
		Streamed data in real time over USB CDC at 200 Hz.	
		Introduced debounced start trigger and LED state indicators.	Anomalies identified retrospectively during post-processing.
		Enabled high-throughput host-side logging using Python	

4.4 USB Logged Dataset

The logged dataset consistent of the following fields:

- Time_us - Timestamp in microseconds
- log_val - Sample index counter
- x, y, z - Acceleration values in m/s^2

A representative dataset excerpt is shown below:

Table 4.3. *Sample dataset excerpt*

Time_us	log_val	x (m/s^2)	y (m/s^2)	z (m/s^2)
25083	1	-0.07845	0.627626	10.04201
30083	2	-0.07845	0.745305	9.963556
35083	3	-0.15691	0.784532	9.963556
40083	4	-0.03923	0.706079	10.04201

The accelerometer values are reported in SI units (m/s^2), as the event.acceleration function from the Adafruit library automatically converts the internal digital counts into SI units.

4.5 Repeated Sample Values

During the inspection of the logged dataset, repeated acceleration values were observed across consecutive samples (see z-axis). At first glance, this behaviour could be interpreted as a logging error or timing fault. However, detailed analysis of the ADXL345 dataset confirms that these repetitions arise from sensor quantisation rather than system malfunction [44].

The ADXL345 is a digital accelerometer in which each axis is represented by a fixed-width signed integer. In FULL_RES mode, the device operates with an effective 13-bit signed resolution as described in the datasheet. In this mode:

- Resolution increases with the selected g-range,
- Scale is maintained at approximately 3.9 mg/LSB,

Therefore the calibrated scale is:

$$3.9\ \text{mg/LSB} \approx 0.0383\ \text{m/s}^2/\text{LSB}$$

This implies that the smallest digitally representable acceleration change is approximately:

$$\Delta a_{min} \approx 0.0383 \mathrm{m/s^2}$$

If the true acceleration change is less than one Least Significant Bit (LSB), the digital output remains unchanged. Consequently, small, negligible vibration fluctuations below this threshold show up as repeated numerical values in the logged dataset.

For example, from the dataset:

$$x = -0.03923 \mathrm{m/s^2}$$

Dividing by the scale:

$$\frac{-0.03923}{0.0383} \approx -1.02$$

This corresponds to approximately 1 raw digital count. Similarly,

$$0.078 \mathrm{m/s^2} \approx 2 \text{ raw counts}$$

This confirms that the measured increments in the dataset directly correspond to the integer digital steps in the ADXL345. Importantly, inspection of the dataset confirms that at least one axis updates per sample. This further verifies that the system is functioning correctly and that repetitions are not caused by the coded logic or USB transmission faults.

4.6 Validation of the Logging System

- Timestamps increase consistently at 5000 μs intervals,
- Sample index increments monotonically,
- At least one axis changes values at each update,
- Quantised increments match the theoretical LSB magnitude,
- No missing samples or corrupt data fields were observed in the captured datasets.

The only implication this system has for machine learning is that very low amplitude changes below 1 LSB are not observable. However, because the quantisation step is small relative to the dynamic vibration amplitudes observed, the resolution is sufficient for fault detection purposes.

Chapter 5: Iterative Development of the System

The development of the proposed PdM system followed an iterative engineering process. Rather than designing the final system in a single step, each iteration was developed to evaluate a specific technical challenge identified by the Research Questions in Chapter 1. This approach allowed individual components of the system to be validated independently before integrating them into the final embedded solution.

Early iterations (Version 1) focus on establishing baseline anomaly detection under simplified conditions, while later versions introduce increasing levels of realism, including operation on a running motor. By incrementally increasing the complexity of the system, it becomes possible to isolate the effects of each change made and evaluate its contribution to the research questions.

To organise the progression, the experimental stages are labelled using a versioned naming convention consisting of a number and an alphabetical identifier. The numerical component corresponds to the primary development phase, while the alphabetical suffix denotes refinements within that phase. For example Version 1A and 1B investigate baseline detection techniques under controlled conditions. Then, Versions 2A-2C evaluate the behaviour of the system when applied to the real motor. The final stage, Version 3A, represents the complete implementation of the PdM system, integrating the most effective techniques identified throughout all the earlier experiments.

Each version presented in this chapter follows a consistent internal structure in order to clearly separate theory, implementation details and experimental results. For every system iteration, three subsections are provided.

The first subsection, **Specific Theory & System Design**, introduces the theoretical concepts and design decisions that are unique to that particular version of the system. The second subsection, **Implementation & Methodology**, describes how the proposed design was actually realised. This includes the configuration of the embedded system, data processing, model training and any experimental setup required to evaluate the version. The final subsection, **Experimental Results & Discussion**, presents the experimental outcomes for that version of the system. Rather than providing a full comparative analysis, this subsection focuses on the immediate observations, limitations and shortcomings identified in the test. The final analysis and results are reversed for Chapter 6.

By structuring each version of the system this way, the chapter presents a clear progression from theoretical concepts, to implementations, to evaluations, allowing the iterative development to be followed systematically.

Version 1A - Experimental Regression Model

The first experimental implementation of the PdM system investigates the feasibility of performing anomaly detection on an embedded device using lightweight classical machine learning. This stage directly addresses RQ1 - Embedded Anomaly Detection Feasibility, which asks whether anomaly detection can be achieved on an embedded system using classical machine learning.

5.1 Specific Theory & System Design

Earthquake detection systems represent one of the most fundamental applications of anomaly detection in time-series vibration data. At their core, these systems are used to identify rare, high energy transient events, embedded in long periods of stationary background vibrations caused by environmental noise, human activity or micro-tremors.

From a signal processing point of view, this makes earthquake detection a perfect example of event-based anomaly detection in time-series data, where the objective is not classification of fault types but rather reliable discrimination between normal and abnormal behaviour. The major assumption made is that normal ground vibrations are statistically stationary, whereas seismic events such as earthquakes manifest as abrupt deviations in amplitude, frequency or energy. Some earthquake detection algorithms therefore rely on simple, robust indicators such as sudden increases in acceleration magnitude or threshold exceedance.

These approaches prioritise low latency, computational simplicity and robustness, since earthquake detection systems must operate continuously and provide rapid alerts. Early earthquake detection systems did not require explicit model training of geological vibration structures, instead, they focused on detecting statistical irregularities in raw accelerometer or seismometer data [45].

This paradigm places earthquake detection as one of the simplest and most intuitive entry points into anomaly detection, making it an ideal theoretical baseline for more complex predictive maintenance problems. An important note is that this work adopts earthquake detection not as an end application, but as a conceptual and methodological blueprint for validating the first stage of anomaly detection systems.

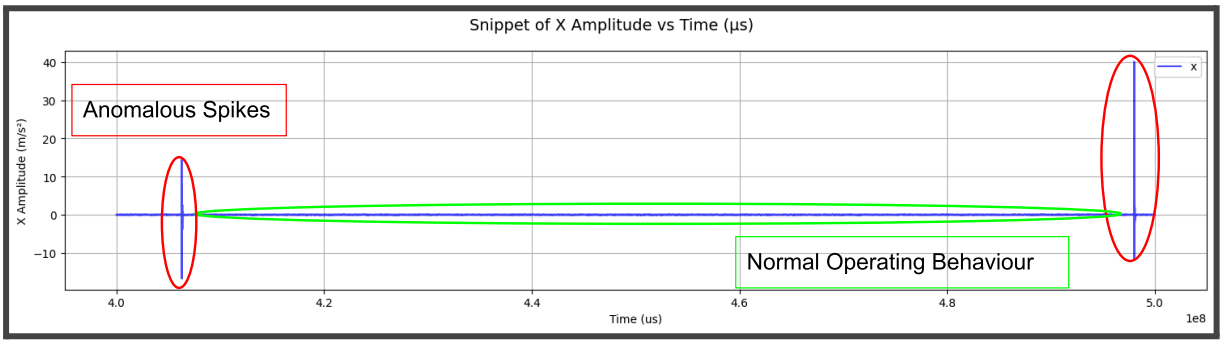


Figure 5.1 Example acceleration time-series data illustrating high-energy transient events embedded within stationary background vibrations.

5.1.1 Regression Model for Anomaly Detection

To establish a robust foundation for vibration-based anomaly detection, a regression model is introduced prior to the development of deep learning algorithms. This section presents the theoretical motivation and reasoning behind the proposed selection of logistic regression as an intermediate design stage prior to CNN-based models. The objective of the regression model is not to maximise classification accuracy or performance, but to validate the physical separability of anomalous vibration events under strict embedded hardware constraints.

Motivation for a Classical Regression Baseline

Before introducing complex deep learning models which require vast amounts of high quality data, it is necessary to determine whether anomalous vibration behaviour is separable from normal operating conditions. Ideally this separation should be possible using simple and interpretable decision boundaries to visualise how the model performs. From an engineering design perspective, this regression model classifier serves as a minimum viable intelligence layer, showcasing the feasibility of intelligent anomaly detection without introducing overly complex models.

In vibration datasets, many anomalies, particularly impulsive or transient events manifest as abrupt deviations in acceleration or jerk magnitude. If these deviations cannot be distinguished from background vibration using low-order statistical features, then a higher-capacity model such as the proposed CNN model is unlikely to perform reliably without being severely overfit. Consequently, this regression model acts as a sanity check, it confirms that the chosen sensing modality, sampling strategy, and preprocessing pipeline provides sufficient information for discrimination between normal and anomalous behaviour.

Emphasis needs to be put on the fact that the validity of this approach is tied exclusively to the controlled nature of the experimental environment at this stage in the research. In this setting the operating conditions, mounting configuration and source of disturbances are all constrained and controlled. This makes linear separability using hand crafted features more likely. In real-world deployment that the system will be tested in at a later stage, the system is exposed to a far wider range of operating regimes, load variations and environmental noise. Under such conditions, the assumptions required for linear separability are unlikely to uphold. For this reason, while regression is appropriate here as a feasibility and validation step, CNN models are pursued in subsequent stages to enable robust anomaly detection in less controlled environments.

Regression models offer significantly higher levels of interpretability when compared to deep learning models. Unlike DL models, where learned representations are distributed across a neural network, regression models provide regression coefficients which give direct insight into the relative importance of each input feature. In two dimensions, it is possible to visualise the decision boundary plot of the regression model. This level of interpretation is particularly valuable at early design stages, since it enables the physical relevance of extracted features to be assessed and validated before progressing to black-box models.

Feature Representation in Windows

Rather than operating directly on raw acceleration data in time series, the proposed regression model relies on features from the vibration signals which are computed over short, fixed-length time windows. This approach enables the extraction of physical indicators of normal and anomalous mechanical behaviour.

In this context, windowing serves two primary purposes. First, it converts continuous time-series data into discrete observations which are more suitable for supervised learning. Second, it aggregates local signal behaviour over a short period of time, which improves the models robustness to sensor noise or minor timing variations. Each window computed represents a snapshot of the system's behaviour at a given point in time.

The selected features which are fed as inputs into the model are derived from fundamental kinematic relationships. Acceleration magnitude captures overall vibration energy, while its time derivative (jerk), emphasises rapid changes which are associated with impulsive events. Sudden disturbances such as taps on the accelerometer produce disproportionately large jerk responses, relative to steady background vibrations and noise. As a result, jerk-based metrics are highly sensitive to transient anomalies while remaining computationally inexpensive to compute.

By restricting the feature set to a small number of meaningful quantities, the model avoids overparameterisation [20]. In this context, overparameterisation refers to the introduction of unnecessary or redundant features that increase the model's complexity relative to the available training data, leading to unstable decision boundaries. The choice of keeping minimal features also improves generalisation, which is defined here as the models ability to maintain reliable anomaly detection performance when exposed to unseen vibration data, minor variations in operating conditions or measurement noise.

Logistic Regression Model

Logistic regression models compute a weighted linear combination of the selected input features and map the result to a probability using a sigmoid activation function. A binary decision is then obtained by thresholding this computed probability.

Let the feature vector extracted from a single vibration window be defined as:

$$x = [x_1 \quad x_2 \quad \dots \quad x_i]$$

Where x_i represents the ith engineered features (such as jerk magnitude or peak-to-peak acceleration).

The model then computes a linear score z as:

$$z = w^T x + b$$

where,

$$w = [w_1, w_2, \dots, w_n]^T$$

is the vector of learned weights and b is the scalar bias term.

The linear score (z) is the passed through the logistic (sigmoid) function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

To produce a model-based probability estimate $p \in (0, 1)$ representing the likelihood that the input window corresponds to anomalous behaviour. y = 1 denotes the anomalous class, while y = 0 denotes normal operation. A final classification decision is obtained by applying a threshold τ to the predicted probability:

$$\hat{y} = \begin{cases} 1, & p \geq \tau \\ 0, & p < \tau \end{cases}$$

τ is typically chosen based on the desired tradeoff between false positives and false negatives. In safety relevant applications, such as vibration based anomaly detection, this threshold can be deliberately lowered in order to increase the sensitivity of the system to certain features to minimise missed anomaly events.

From a theoretical point of view, logistic regression assumes that the two classes, normal vs anomalous, are linearly separable in feature space. While this assumption is restrictive, it is intentional in this design stage. If anomalous and normal vibration windows are separable by a linear boundary, then the extracted features capture the discriminative structure of the data. Conversely, failure to achieve meaningful separation indicates that either the features are insufficient or the sensing strategy needs some refinement.

Since inference consists solely of a fixed number of multiplication operations and a scalar non-linear function, the computational costs of logistic regression remain constant and predictable. This makes it very well suited for real-time execution on resource-limited embedded devices.

Embedded Deployment

The extracted weights (*w)*, bias (*b)* term and normalisation parameters will be exported from the Python environment and saved as a header file. This file will then be included in the embedded development environment, enabling the microcontroller to perform logistic regression inference directly using these values. Correct classifier operation on the embedded system requires strict consistency between offline and embedded pipelines. Specifically, the same feature definitions, window length, and preprocessing steps must be applied locally in order to ensure numerical equivalence with the trained model.

5.2 Implementation & Methodology

This section serves as the complete implementation workflow used to develop the initial regression based anomaly detector system for the desk based, earthquake style vibration dataset.

5.2.1 Regression Model Training & Implementation

Table 5.1. *Regression model objectives.*

(i)	Construct a labelled dataset from raw tri-axial accelerometer readings.
(ii)	Convert the continuous signal into fixed-length windows suitable for supervised learning.
(iii)	Compute a set of physically meaningful time-domain features.
(iv)	Train and evaluate a lightweight logistic regression classifier.
(v)	Deploy the model on a ESP32-S3 to serve as a feasibility baseline.

Dataset Description

The dataset used for this regression model consists of tri-axial acceleration samples collected from an ADXL345 accelerometer at a target rate of 200 Hz using the USB CDC pipeline described in Chapter 4. The dataset is about 3 minutes long and consists of 26,000 rows of data. Each row of the recorded CSV contains a microsecond timestamp (Time_us) and acceleration samples along the three axes (x,y,z) expressed in m/s^2. The signals were loaded into Python and split into their raw channels using the naming convention *_sig to indicate unprocessed signals.

To provide an initial sanity check on sensor behaviour, code was written to print out the maximum and minimum observed values in each axis (both in m/s^2 and converted into g) before any further processing. This step ensures the dataset is physically plausible and can also reveal clipping or scaling issues prior to model development.

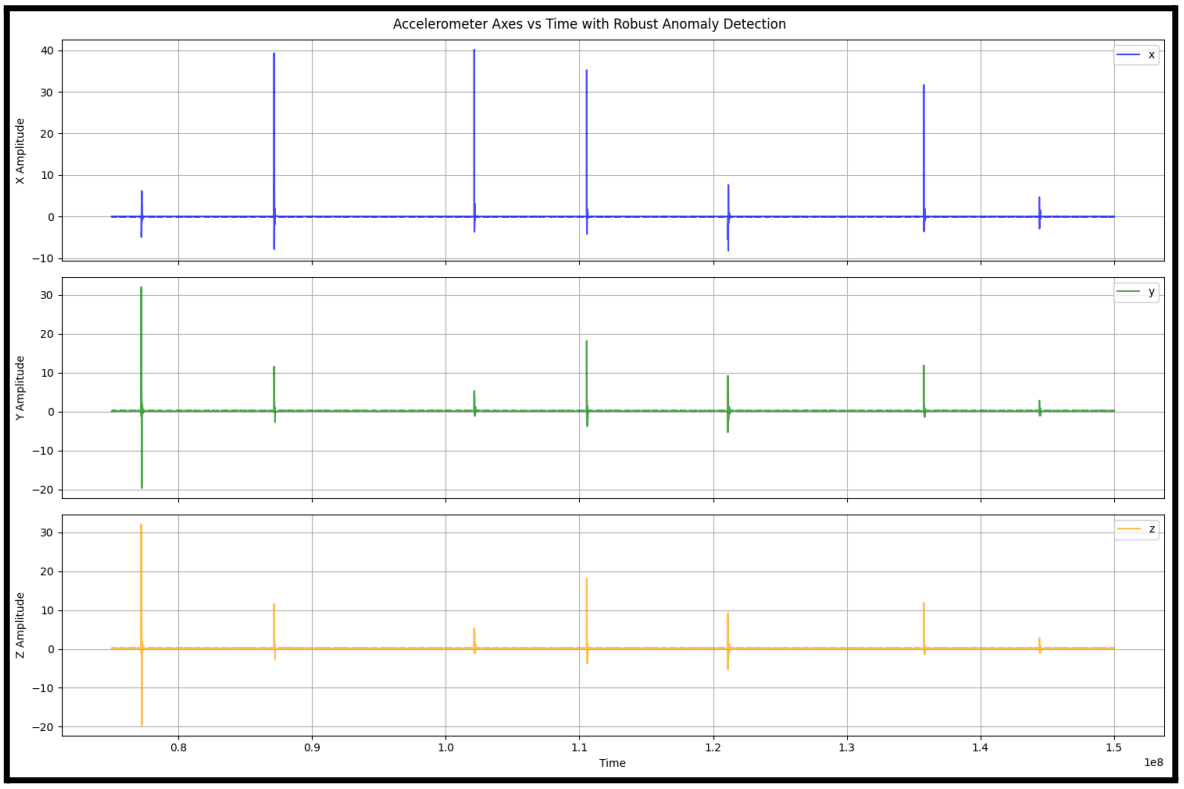


Figure 5.2. Trimmed Accelerometer Axes x (blue), y (green), z (yellow) in m/s^2 vs Time (us).

Weak Labeling via Z-Score Thresholding

Since the desk-based dataset does not contain explicit anomaly labels, weak labels for testing purposes were generated using a statistically spike detection approach. For each axis *x(t), y(t), z(t),* the mean and standard deviation were computed over the full recording. Upper and lower thresholds were then defined as the following.

$$x_{upper} = \mu_x + k\sigma_x, \quad x_{lower} = \mu_x - k\sigma_x$$

And equivalently for *y* and *z*, where k = 3 was selected as the sensitivity factor. From this, a sample was flagged as anomalous if any axis exceeded the threshold bounds:

$$\text{label(t)} = \begin{cases} 1, & (x(t) \notin [x_{lower}, x_{upper}]) \vee (y(t) \notin [y_{lower}, y_{upper}] \vee (z(t) \notin [z_{lower}, z_{upper}) \\ 0, & \text{otherwise} \end{cases}$$

This produces a binary label vector stored in the dataset as df[“label”]. To verify that the generated labels correspond to visually obvious spikes, a script was written to plot each axis versus time with the anomalous points overlaid as colourful scatter points. The reason this labelling method holds valid in this experiment is because anomalous taps are obvious in the controlled experiment and they can be visually verified with plots.

Sampling Rate Estimation from Microsecond Timestamps

The data logger actively targets to sample at 200 Hz but the true effective sampling rate had to be verified and estimated from the recorded microsecond timestamps. The function sampling_time_estimate() computes the Δt between consecutive timestamps, filters out invalid values (negative or abnormally large steps) and uses the median time step to estimate the sample rate.

$$f_s \approx \frac{10^6}{\tilde{\Delta} t}$$

Listing 5.1. *Sampling Rate Estimation*

```
def sampling_time_estimate(time_us_array):
    time_us_array = np.asarray(time_us_array)

    dt = np.diff(time_us_array)
    dt = dt[(dt > 0) & (dt < 20000)]  # reject rollover (>20 ms)
    dt_med = np.median(dt)  # microseconds
    return float(1_000_000.0 / dt_med)
```

Apart from its purpose as a validation step, the sampling rate estimation is used later for computing the time step for derivative based features such as the jerk, ensuring that the jerk magnitude is numerically consistent with the actual sampling interval.

Sliding Window Segmentation and Window Labeling

In order to convert the continuous time series data into supervised learning samples, the signal was segmented into fixed length windows using WINDOW_SIZE = 20 and WINDOW_STEP = 5. For each window starting at index s and ending at $e = s + WINDOW_SIZE$, features were computed from the segmented window. This was implemented using a majority vote strategy, based on the mean label value within the window. The code listing for this function is shown below.

$$y_{win} = \text{round}\left(\frac{1}{N}\sum_{i=s}^{e-1} y[i]\right)$$

This produces a window label of 1 if anomalous samples dominate the window, and 0 otherwise. The resulting dataset consists of a feature matrix $X \in \mathbb{R}^{M\times F}$ and a corresponding label vector $y \in \{0,1\}^M$, where M is the number of windows and F is the number of extracted features.

Listing 5.2. *Window creation and labelling.*

```
#Window Creation
def window_build(x_array, y_array, z_array, labels_array,
sample_rate_hz):

    n = len(labels_array)
    X_list, y_list = [], []

    for start in range(0, n - WINDOW_SIZE + 1, WINDOW_STEP):
        end = start + WINDOW_SIZE

        features =
create_features(x_array[start:end],y_array[start:end],z_array[start:end]
,sample_rate_hz)

        # Majority vote label in window
        win_label = int(np.round(np.mean(labels_array[start:end])))

        X_list.append(features)
        y_list.append(win_label)

    return np.vstack(X_list), np.array(y_list)
```

Feature Extraction Pipeline

For each window, a compact set of time-domain features was computed in create_features(). First, the acceleration magnitude was computed:

$$a_{mag}(t) = \sqrt{x(t)^2 + y(t)^2 + z(t)^2}$$

And mean-centres to remove DC offset:

$$a_c(t) = a_{mag} - \overline{a_{mag}}$$

Jerk, which is the derivative of acceleration w.r.t time was then computed using the numerical gradient on each axis:

$$j_x(t) = \frac{dx(t)}{dt}, \quad j_y(t) = \frac{dy(t)}{dt}, \quad j_z(t) = \frac{dz(t)}{dt}$$

From these calculations, the script computed six different features per window as shown below in **Table 5.2,** the corresponding code listing can be found in **Listing 5.3**

Table 5.2. *Table of extracted features.*

1.	RMS of centred magnitude	$\sqrt{\frac{1}{N}\sum a_c^2}$
2.	Peak of centred magnitude	$\max(\lvert a_c \rvert)$
3.	Peak-to-peak (p2p)	$\max(a_c) - \min(a_c)$
4.	Jerk RMS	$\sqrt{\frac{1}{N}\sum j_{mag}^2}$
5.	Jerk peak	$\max(j_{mag})$
6.	Crest factor	$\frac{\text{peak}}{(\text{RMS} + 10^{-6})}$

These features were selected because they are computationally lightweight and physically interpretable. Impulsive events cause spikes in peak-to-peak amplitude and disproportionately large jerk magnitude when compared to steady background vibration.

Listing 5.3. *Per window feature extraction.*

```
#Inputs are the windows
#Outputs are rms,peak,p2p,jerk_rms,jerk_peak,crest
def create_features(x_win, y_win, z_win, sample_rate_hz):

    dt = 1.0 / float(sample_rate_hz)#time step

    mag = np.sqrt(x_win*x_win + y_win*y_win + z_win*z_win)
    mag_c = mag - np.mean(mag)

    jx = np.gradient(x_win, dt)#x_win is in m/s^2 so gradient -> m/s^3
    jy = np.gradient(y_win, dt)
    jz = np.gradient(z_win, dt)
    jmag = np.sqrt(jx*jx + jy*jy + jz*jz)
```

```
    rms = np.sqrt(np.mean(mag_c*mag_c))
    peak = np.max(np.abs(mag_c))
    p2p = np.max(mag_c) - np.min(mag_c)

    jerk_rms = np.sqrt(np.mean(jmag*jmag))
    jerk_peak = np.max(jmag)

    crest = peak / (rms + 1e-6) #+1e-6 to not divide by 0

    return np.array([rms, peak, p2p, jerk_rms, jerk_peak, crest])
```

Although all six features were computed per window, a reduced 2D feature set was actually used for the baseline regression experiment. This was done for two reasons, (i) to simplify separability analysis and (ii) to enable direct visualisation of decision boundaries using a 2D decision plot. The implementation selects p2p magnitude (index 2) and jerk_rms (index 3). This forms the following input for the regression model:

$$X_{trim} = [\text{p2p}, \text{jerk_rms}]$$

Before training, features inside X_{trim} were normalised using StandardScaler():

$$x' = \frac{x - \mu}{\sigma}$$

This step is critical for logistic regression models since unnormalised feature scaling can significantly bias the learned coefficients towards higher magnitude features. Standardisation also supports later embedded reproducibility because the used μ and σ values can be stored and applied on-device prior to inference.

Training/Test Split and Model Training

The dataset was split into training and test sets using a stratified split. Stratification ensures that both training and test subsets contain representative proportions of anomalous windows, which is a critical consideration given the expected class imbalance in event-based anomaly datasets.

A logistic regression classifier was trained using scikit-learn with the class balancing enabled. Class weighting increases the penalty for misclassifying minority-class windows (ones with anomalies), in turn improving the models sensitivity in imbalance datasets. After fitting, the model learned a weight vector and bias corresponding to the linear separator in the standardised 2D feature space.

Listing 5.4. *Stratified train/test split.*

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_windows,
    test_size=0.3,
    stratify=y_windows,
    random_state=42
)

clf = LogisticRegression(class_weight="balanced")
clf.fit(X_train, y_train)
```

Performance Metrics

To visually assess class separability, a decision boundary plot was generated using the fitted models predictions over a meshgrid. Test samples were plotted in the same space, and the boundary corresponding to the class transition was drawn using a contour threshold near 0.5 probability. This step provides a qualitative check that the two selected features contain enough discriminative structure and that a linear separator is a reasonable feasibility baseline prior to CNN development.

Listing 5.5 *Decision boundary plot.* [46]

```
def plot_decision_boundary(predict, X, y):
    x1 = np.linspace(X[:,0].min(), X[:, 0].max(), 200)
    x2 = np.linspace(X[:,1].min(), X[:, 1].max(), 200)
    X1, X2 = np.meshgrid(x1, x2)
    pts = np.c_[X1.ravel(), X2.ravel()]
    #f = predict(pts, w, b).reshape(X1.shape)
    f = predict(pts).reshape(X1.shape)
    plt.figure(figsize=(7,7))
    plt.scatter(X[:,0], X[:,1], c=y, cmap='Accent', s=50)
    plt.contour(X1, X2, f, 1, levels=[0.5, 0.51])
    plt.xlabel('$x_1$', fontsize=16)
    plt.ylabel('$x_2$', fontsize=16)
    plt.title('Logistic regression decision boundary')

plot_decision_boundary(clf.predict, X_test, y_test)
```

The model was then evaluated using a confusion matrix and a classification report. These outputs quantify how well the regression model detects anomalous windows (true positives) and how often it fails to detect anomalies (false negatives). Since missed anomalies are typically the most critical failure mode in PdM, the false negative rate and recall are treated as primary indicators of feasibility.

Listing 5.6 *Confusion matrix and classification report.*

```
y_pred = clf.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print("")
print(classification_report(y_test, y_pred))
```

Parameter Export for Embedded Deployment

In order to enable deterministic real-time classification directly on the ESP32, the parameters used in the logistic regression model had to be extracted. A practical advantage of using logistic regression is that the inference stage reduces down to a small set of multiplication operations and a sigmoid mapping function. Therefore, once trained, the parameters required for inference are the standardisation parameters used (μ & σ) and the logistic regression parameters (w & b).

Listing 5.7 *Confusion matrix and classification report.*

```
mu = scaler.mean_.astype(np.float32)          # shape (2,)
sigma = scaler.scale_.astype(np.float32)      # shape (2,)
w = clf.coef_.reshape(-1).astype(np.float32) # shape (2,)
b = float(clf.intercept_[0])
print("mu:", mu)
print("sigma:", sigma)
print("w:", w)
print("b:", b)

header = f"""#pragma once
static const float SCALER_MEAN[2]  = {{ {mu[0]:.8f}f, {mu[1]:.8f}f }};
static const float SCALER_SCALE[2] = {{ {sigma[0]:.8f}f, {sigma[1]:.8f}f
}};

static const float LR_W[2] = {{ {w[0]:.8f}f, {w[1]:.8f}f }};
static const float LR_B    = {b:.8f}f;
"""

with open("model_params.h", "w") as f:
    f.write(header)

print("\nWrote model_params.h")
```

5.2.2 Embedded Deployment and On-device Validation

To validate that the trained regression model could operate under real-time embedded constraints, the final logistic classifier was deployed to an ESP32-S3 and executed directly on live ADXL345 accelerometer measurements. The embedded implementation of this model mirrors the offline Python pipeline exactly: (i) deterministic 200 Hz sampling, (ii) rolling window formation of 20 samples, (iii) computation of the same two features (p2p & jerk RMS), (iv) standardisation using the exported StandardScaler parameters, and (v) logistic regression inference using exported weights and bias. A NeoPixel LED was used as an immediate alert indicator, where normal operation displays a blue LED and detected anomalies trigger a red LED for a fixed duration. The same wrap safe microsecond scheduler as the datalogging system was used to enforce a constant sampling period of 5ms.

Rolling Window Buffer

Instead of collecting a full dataset, the ESP32 maintains a rolling buffer of the most recent 20 samples per axis, matching the window size used in the training stage. During startup, the buffer is filled, after which each new sample shifts the window forward by one.

Listing 5.8 *Rolling buffers to store the most recent 20 samples.*

```
//Rolling buffers
float xbuf[WINDOW_SIZE], ybuf[WINDOW_SIZE], zbuf[WINDOW_SIZE];
int bufCount  = 0;
//Shift buffer
 //While its still filling, writes new sample into next free slot &
increments bufCount
if (bufCount < WINDOW_SIZE) {
  xbuf[bufCount] = event.acceleration.x;
  ybuf[bufCount] = event.acceleration.y;
  zbuf[bufCount] = event.acceleration.z;
  bufCount++;
}
else { //once full, shift left and append new sample. Buffer always
holds 20 most recent samples
  for (int i = 0; i < WINDOW_SIZE - 1; i++) {
    xbuf[i] = xbuf[i+1];
    ybuf[i] = ybuf[i+1];
    zbuf[i] = zbuf[i+1]; }
  xbuf[WINDOW_SIZE - 1] = event.acceleration.x;
  ybuf[WINDOW_SIZE - 1] = event.acceleration.y;
  zbuf[WINDOW_SIZE - 1] = event.acceleration.z; }
```

To reduce computational load while maintaining responsiveness, features and inference are computed every `WINDOW_STEP = 5` samples. At 200 Hz this corresponds to an evaluation every 25 ms, while still using a 100 ms context window.

Listing 5.9 *Inference conditions.*

```
//Once window is filled do this
//When you read window_step run feature extraction on samples and run
inference
if (bufCount >= WINDOW_SIZE) {
  stepCount++;
  if (stepCount >= WINDOW_STEP) {
    stepCount = 0;
    //Compute Features
    float p2p, jerk_rms;
    compute_features(p2p, jerk_rms);
    //Calculate inference probability
    float p = infer_probability(p2p, jerk_rms);
    //Set prediction based on threshold
```

Feature Extraction

The embedded feature function below reproduces the same signals which were used in offline training in Python.

Listing 5.10 *Embedded Feature extraction.*

```
static void compute_features(float &p2p_out, float &jerk_rms_out){
  const float dt = 1.0f / (float)SAMPLE_RATE_HZ; //Ensure they are both
floats when dividing

  float mag[WINDOW_SIZE];
  float mag_sum = 0.0f;

  //Computing mean magnitude of a window
  for (int i = 0; i < WINDOW_SIZE; i++) {
    float x = xbuf[i];
    float y = ybuf[i];
    float z = zbuf[i];
    float m = fast_sqrt(x*x + y*y + z*z);
    mag[i] = m;
    mag_sum += m;
  }
  float mag_mean = mag_sum / (float)WINDOW_SIZE;

  float min_c = 1e9f; //real min_c will be much smaller than 1e9
```

```
  float max_c = -1e9f; //Used to avoid first itteration problems
  for (int i = 0; i < WINDOW_SIZE; i++) {
    float mc = mag[i] - mag_mean; //mc is magnitude centered, removes DC
offset (slow bias such as turning gently and gravity)
    if (mc < min_c){min_c = mc;}
    if (mc > max_c){max_c = mc;}
  }
  p2p_out = max_c - min_c; //min-max gives p2p

  //If the signal is quiet, max_c and min_c are close there small p2p
  //If the signal is sharp such as a tap, the swing is larger therefore
large p2p


  //Time derivative of acceleration a_dot (m/s^3)
  //To approximate, use discrete derivatives ie,
  //x[i]-x[i-1]/delta_t
  float jmag_sq_sum = 0.0f; //initialise
  for (int i = 1; i < WINDOW_SIZE; i++) { //i=1 since i=0 cant compute
x[0-1]
    float jx = (xbuf[i] - xbuf[i-1]) / dt;
    float jy = (ybuf[i] - ybuf[i-1]) / dt;
    float jz = (zbuf[i] - zbuf[i-1]) / dt;
    float jm = fast_sqrt(jx*jx + jy*jy + jz*jz); //magnitude of the
jerk, collapses all axis into one measure of suddenness
    jmag_sq_sum += jm * jm; //calculation for RMS needed (root mean
squared)
  }

  float jmag_sq_mean = jmag_sq_sum / (float)(WINDOW_SIZE - 1);
  jerk_rms_out = fast_sqrt(jmag_sq_mean);
}
```

Inference

To ensure the features computed above have the same numerical equivalence with the Python pipeline, the same standardisation is applied using the extracted SCALER_MEAN[] and SCALER_SCALE[] exported from Python at the end of training, followed by logistic regression inference using LR_W[] and LR_B. The sigmoid function then converts the linear score into a probability between 0-1. A final binary prediction is produced using a fixed decision threshold of 0.5.

Listing 5.11 *Normalisation parameters and learned weights/bias terms.*

```
static const float SCALER_MEAN[2]  = { 0.54609662f, 28.29724503f };
static const float SCALER_SCALE[2] = { 4.03975773f, 193.75241089f };

static const float LR_W[2] = { 1.04627526f, 0.48221943f };
static const float LR_B    = -11.88104071f;
```

Listing 5.12 *Inference probability and sigmoid function.*

```
static inline float sigmoid(float x) {return 1.0f / (1.0f + expf(-x));}

static float infer_probability(float p2p, float jerk_rms) {
  float x0 = (p2p - SCALER_MEAN[0]) / (SCALER_SCALE[0] + 1e-12f);
  float x1 = (jerk_rms - SCALER_MEAN[1]) / (SCALER_SCALE[1] + 1e-12f);

  float z = LR_W[0]*x0 + LR_W[1]*x1 + LR_B; //z = leanred_w[0]*x0... +
learned_bias
  return sigmoid(z); //Sigmoid function to produce probability value
}
```

Listing 5.13 *Inference decision threshold.*

```
//Decision threshold
static const float THRESH = 0.50f;
    //Calculate inference probability
    float p = infer_probability(p2p, jerk_rms);

    //Set prediction based on threshold
    int pred;
    if(p>=THRESH){
      pred = 1;
    }
    else{
      pred = 0;
    }
```

Local Alerting

For observability during testing, the system prints the computed features, probability, and decision to the serial console. The onboard NeoPixel LED provides an immediate real-time alert, where the LED remains blue during normal operation and turns red for one second when an anomaly is detected.

Listing 5.14 *Serial printing and local altering with the NeoPixel LED.*

```
Serial.print("p2p=");
Serial.print(p2p, 4);
Serial.print(" jerk_rms=");
Serial.print(jerk_rms, 4);
Serial.print(" prob=");
Serial.print(p, 3);
Serial.print(" pred=");
Serial.println(pred);

if (pred == 1) {
 alertActive = true;
 alertUntilMs = millis() + ALERT_MS;
 ledRed();
```

5.3 Experimental Results & Discussion

This section presents an evaluation of the proposed regression based anomaly detection system and its deployment on embedded hardware. The objective is to assess both the classification capability of the regression model under controlled conditions and its practical feasibility when executed in real time on an ESP32-S3.

5.3.1 Regression Model Offline Performance

The results presented below provide the necessary context and proof of feasibility to motivate a transition to CNN-based models in subsequent sections. Section **5.3.1 Regression Model Offline Performance** examines the results obtained from the logistic regression classifier trained and evaluated in Python while section **5.3.2 Embedded Regression Inference Performance** examines the results from real time inference.

Decision Boundary Plot

Figure 5.3 illustrates the raw accelerometer signal with its anomalous points highlighted using the statistical thresholding approach. It is immediately evident that the weak labelling strategy captures only a small portion of the true impulsive events present in the signal. Several obvious spikes in the acceleration data are not assigned anomalous labels (Green circles in **Figure 5.4**) indicating that the automatic labeling process is incomplete and poor in operation.

This behaviour is expected since it reflects a fundamental limitation of simple threshold-based labelling in high-frequency vibration data. The statistical thresholds were intentionally selected to avoid false positives, resulting in a dataset which is under-labelled for short-duration impulsive events.

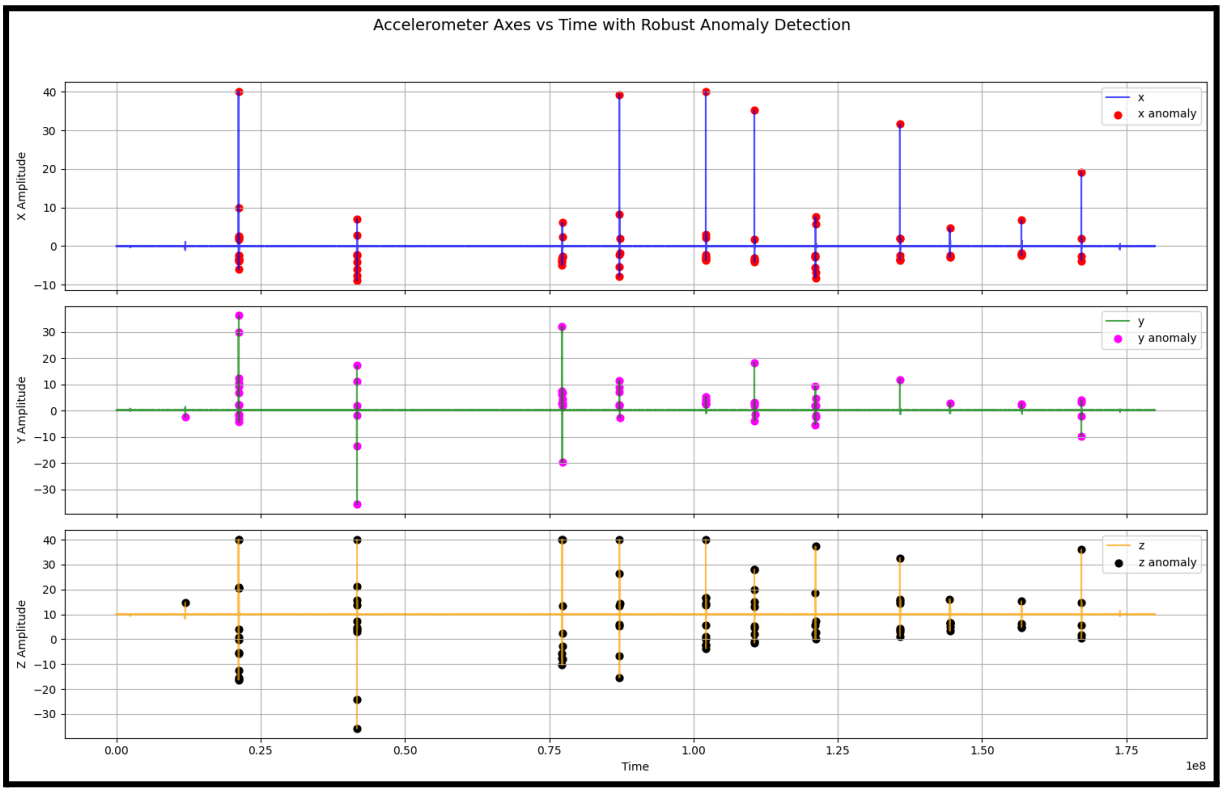


Figure 5.3. Triaxial acceleration time-series with statistical threshold anomaly detections.

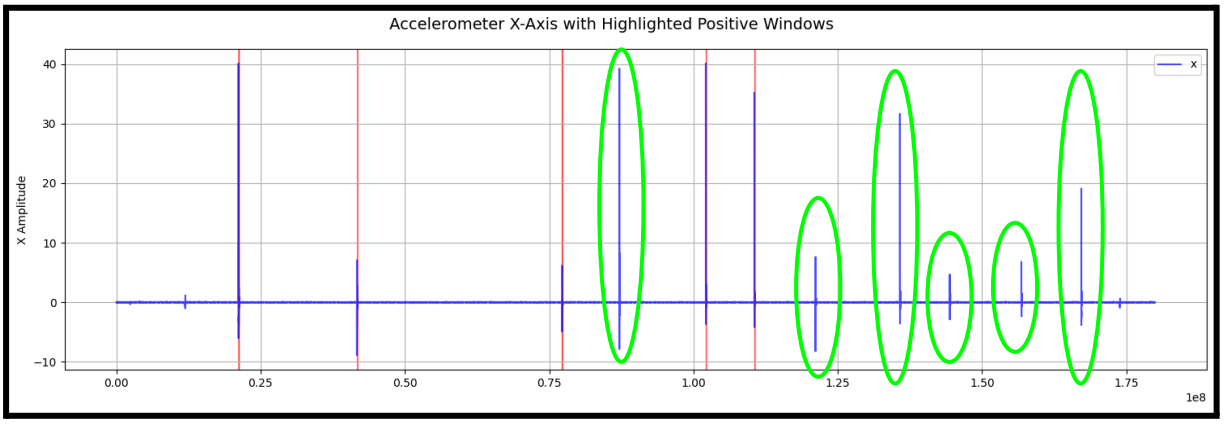


Figure 5.4. Under labelling behaviour of the threshold based anomaly detection approach.

Despite this limitation, **Figure 5.5** below illustrates that the logistic regression model is still capable of learning a meaningful decision boundary in the reduced feature space formed by p2p acceleration magnitude and jerk RMS. Although only a limited, incomplete number of windows were explicitly labelled as anomalous, the extracted features from these impulsive events occupy a distinct region of the feature space. This is an indication that the model is not memorising the labelled windows, but instead is learning a physically meaningful separation between low energy background vibrations and high energy impulsive events. Furthermore, several windows corresponding to the visually apparent disturbances, fall on the anomalous side of the learned decision boundary, even though they were not included in the labelled anomaly set. This is evident from the plot since the points are green, which

indicates a window with a label of 0 (normal operation), yet they're on the right side of the decision boundary which suggests they're anomalies. The regression model is effectively generalising beyond the incomplete labels and responding to the features extracted rather than solely relying on the presence of explicit labels.

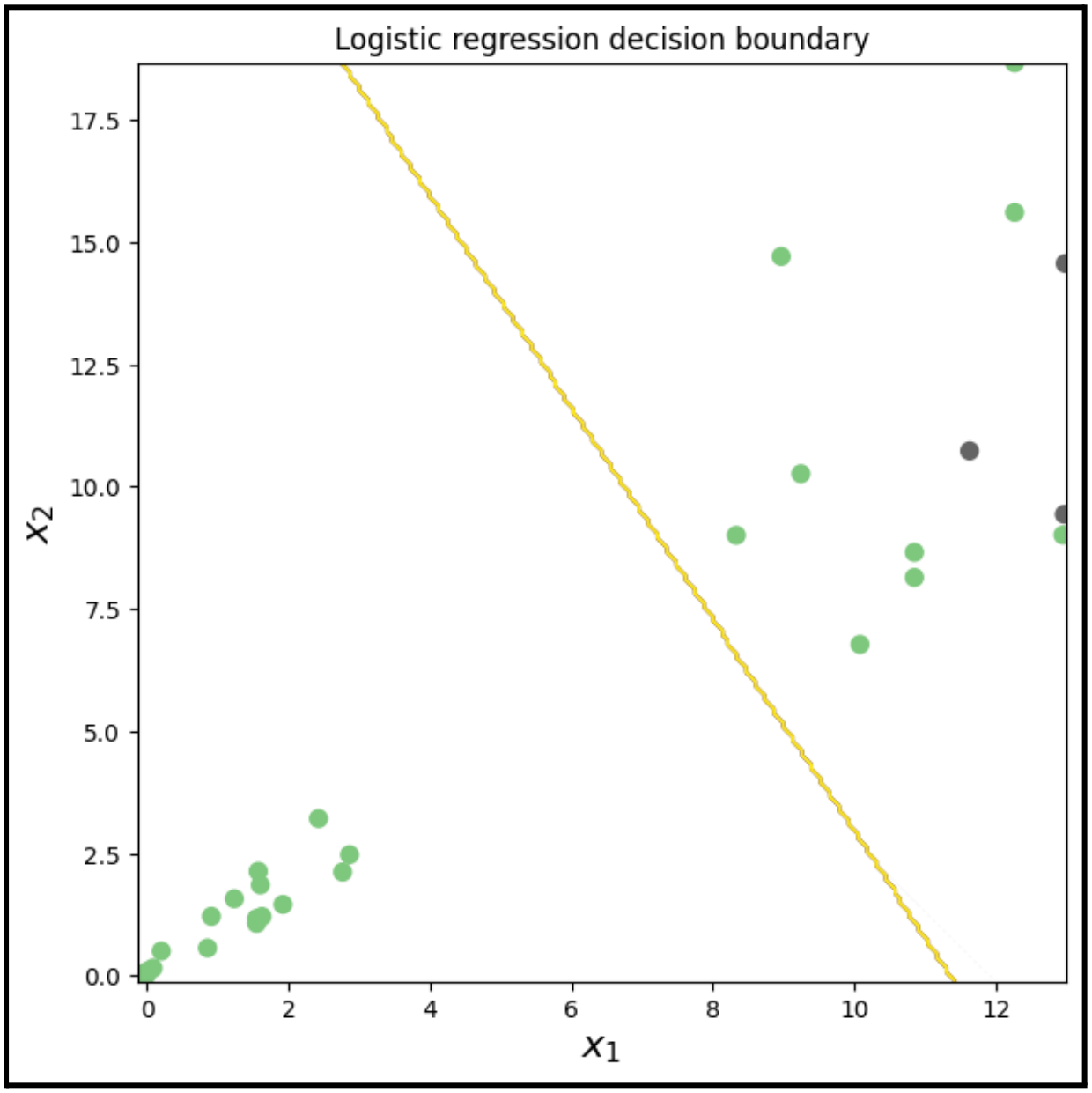


Figure 5.5. Decision Boundary Plot.

Observations

These results highlight two key observations. First, it confirms that the selected feature set captures enough physical properties associated with impulsive anomalies to enable linear separability, even under weak supervision. Second, it demonstrates that the regression model is robust to labelling noise and incompleteness at this early stage of development. Rather than undermining the validity of the results, the imperfect labelling and performance reinforces the role of the regression model experiment as a feasibility check. The fact that stable separation emerges in the learned feature space is significant, as it confirms that impulsive vibration events are linearly separable using a compact set of features. In this context, the objective of the regression experiment was not classification optimization, but verification that linear separability exists in the data.

To numerically investigate the results a confusion matrix was generated using the same held-out test set shown in the decision boundary plot. The confusion matrix can be seen below in **Table 5.3**, where it further validates all that was concluded above.

Table 5.3*. Confusion matrix for regression model*

	PREDICTED 0	PREDICTED 1
TRUE 0 (Normal Operation)	1548	9
TRUE 1 (Anomalous Operation)	0	3

Nevertheless, the overlap observed near the decision boundary exposes the inherent limitations of a linear classifier, trained on hand crafted features. As anomaly amplitude decreases or event windows shorten, separability degrades and classification accuracy decreases. These limitations are what motivate the investigation of convolutional neural networks in subsequent sections, where richer temporal structure can be learned directly from raw vibration data signals.

5.3.2 Embedded Regression Inference Performance

This section of the results evaluates the behaviour of the trained regression model when deployed on the ESP32-S3 and executed in real time using live data. The objective here is to assess whether the embedded system is able to (i) produce stable features from live data, and (ii) respond reliably to impulsive anomalies.

Table 5.4 shows a snippet of the serial output captured during embedded operation, presenting the computed p2p and jerk RMS, inferred anomaly probability (prob) and the final binary prediction (pred) for each evaluated window. During normal operation, the reported feature values remain stable, with low p2p magnitudes and moderate jerk RMS values. These low values result in probabilities of 0 and consistent prediction of the normal class.

When impulsive disturbances are introduced by tapping the accelerometer, a pronounced increase in jerk RMS and p2p is observed. These sharp changes in the features produce an increase in inferred probability, exceeding 0.8 and reliably crossing the fixed decision threshold of 0.5. As shown in the logs, the highlighted red rows are events classified as anomalous.

These logs confirm that the embedded feature extraction and inference pipeline developed mirrors the offline Python implementation. More importantly, it demonstrates that the regression model responds to rapid, high energy events rather than slow variations in acceleration.

Table 5.4. *Serial monitor output showing embedded feature values and predictions.*

p2p	jerk_rms	prob	pred
0.1553	15.0586	0.000	0
0.1553	16.6911	0.000	0
0.1553	18.4429	0.000	0
0.8808	17.3571	0.000	0
36.1391	1975.7776	0.899	**1**
47.0494	6101.0322	1.000	**1**
48.6926	6129.2593	1.000	**1**
48.6926	6129.6279	1.000	**1**
48.6926	5343.7888	1.000	**1**
4.5044	405.2199	0.000	0
4.7197	478.3479	0.000	0
4.7197	476.3493	0.000	0

Real-time Alert Mechanism

To provide intuitive feedback during testing, anomaly detections were mapped to the on-board NeoPixel LED. During normal operation, the LED remains blue and when an anomaly is detected, the LED switches to red for a second before returning to blue.

Video evidence captured during testing confirms that the LED transitions are coupled to impulsive events and do not trigger during slow rotation or steady handling. This demonstrates that the embedded system is entirely capable of providing immediate and interpretable feedback without any external monitoring tools. While the LED serves primarily as a debugging and demonstration tool in this prototype, it also illustrates how anomaly detection could be mapped to alarms, fault flags, display screens, MQTT communication networks or even higher-level control logic.

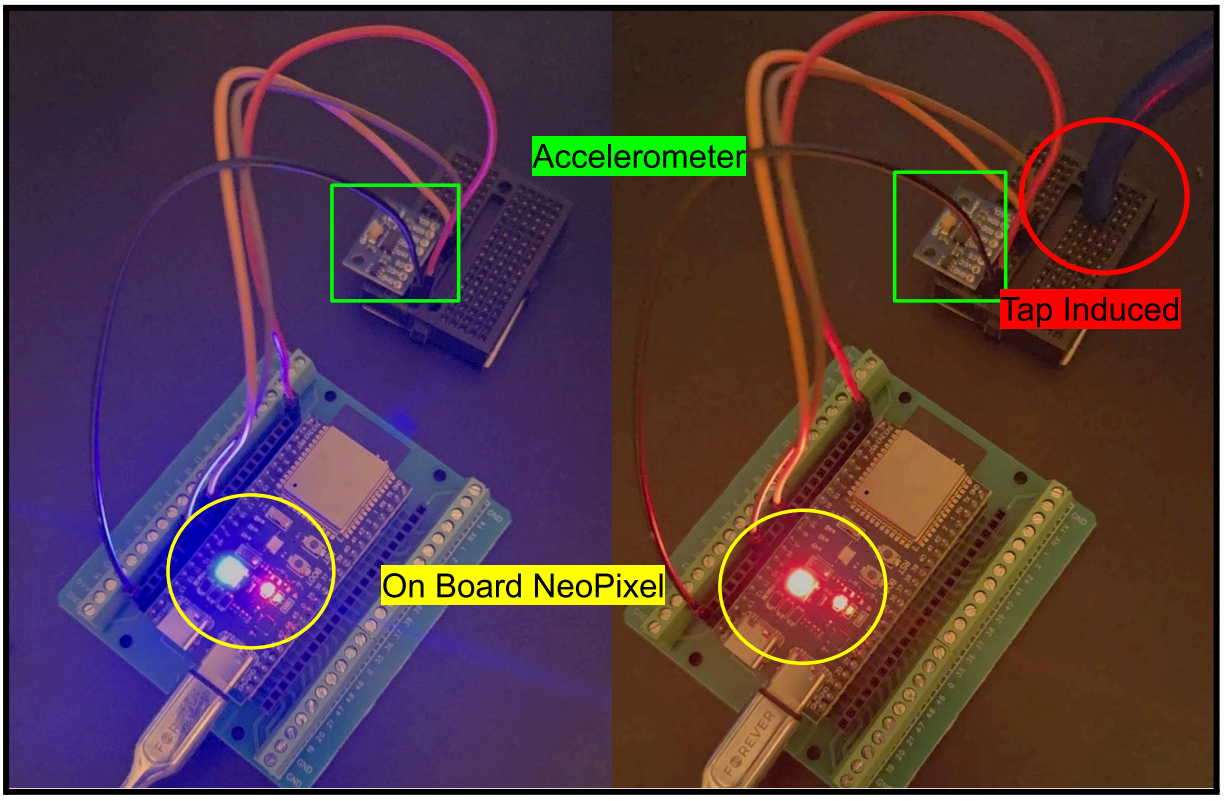


Figure 5.6 On-device visual alert mechanism triggered by induced tap.

A critical observation made during embedded testing is the systems robustness to change in sensor orientation. When the accelerometer was slowly rotated by hand (**Figure 5.7**), no anomaly detections were triggered and the LED indicator remained in the normal (blue) state. Although rotations do alter the static acceleration components of each axis due to gravity, these rotations do not introduce large jerk values or high frequency transients.

This behaviour aligns with the chosen feature set. Since both p2p magnitude and jerk RMS are computed over short windows of time and are dominated by rapid change, slow orientation changes are suppressed. As a result, the system avoids indicating false positives caused by benign motion or handling. In contrast, as shown above, impulsive taps generate sharp accelerations that are immediately displayed on the LED.

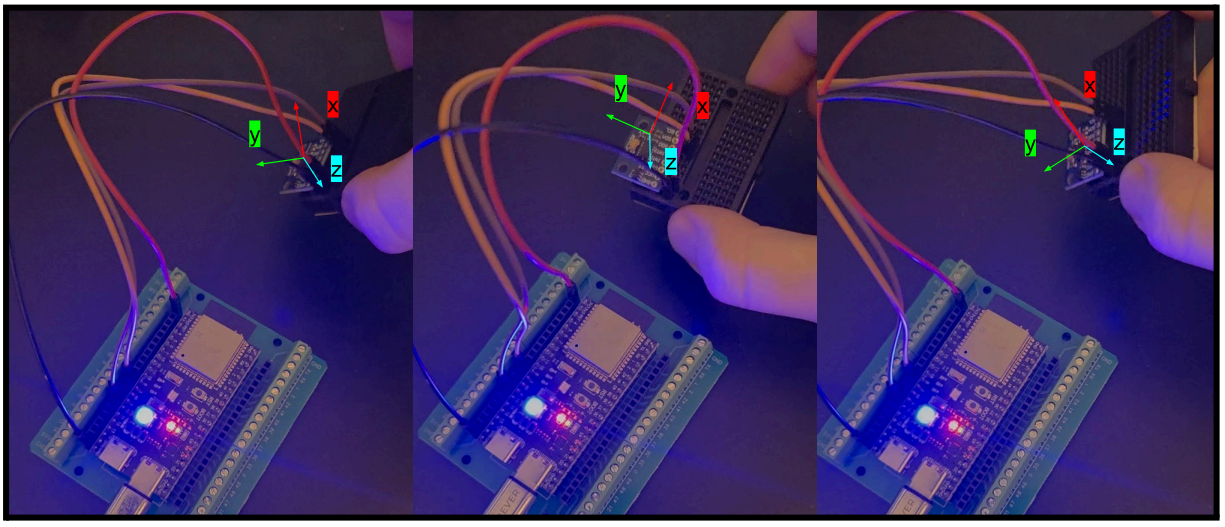


Figure 5.7 Devices invariance to slow rotations and orientation.

System Stability

Throughout the rest of the testing stage, the system maintained stable operation and managed to uphold the target sampling rate of 200 Hz, with no observed inference jitter or missed evaluations. The wrap-safe scheduler and lateness handling ensured consistent timing even with serial outputs and LED update handling. Additionally, the deterministic nature of the regression based inference ensured predictable execution time on the microcontroller. These positive results confirm that the deployed model satisfies real-time constraints and can operate continuously without degradation.

Practical Limitations

Although the embedded deployment successfully demonstrated real-time anomaly detection, the expected limitations were observed. The regression model remains sensitive to the specific vibration characteristics learned through the training data, and the decision boundary is fixed once deployed. As a result, any variations in the operating conditions, such as a phone vibrating on the desk nearby or being deployed on an industrial motor, may increase false positives and reduce detection confidence. Furthermore, the model operates on a very short temporal context of only 100 ms and relies exclusively on manually engineered features. This results in a model that struggles to capture subtle or distributed fault signatures.

Conclusion of Embedded Deployment Results

The embedded regression model achieved stable feature extraction, deterministic inference, and robust behaviour, resulting in a successful real-time anomaly detection system. Impulsive disturbances were reliably detected and clearly differentiated from slow motion, validating both the feature design choice and the embedded deployment pipeline.

However, the observed limitations reinforce that the regression model only serves as a feasibility and validation step. While suitable for detecting high-energy transients in a controlled environment, the model lacks representational capacity required for more complex vibration patterns or operation regimes. These observations directly motivate the transition to CNN models, which are explored first in a controlled environment, then on a functional motor.

Version 1B - Experimental 1D-CNN Model

The second desk based implementation investigates whether anomaly detection performance can be improved by replacing manually engineered features with a deep learning approach. This stage continues to address RQ1 - Embedded Anomaly Detection Feasibility, but shifts the focus from classical machine learning towards evaluating whether a convolutional neural network (CNN) can be trained, quantised and deployed on an embedded device.

Where Version 1A validated the feasibility of detecting anomalous events using linear decision boundaries, Version 1B explores whether hierarchical feature learning can improve robustness. The introduction of a CNN model represents a controlled increase in model capacity to evaluate the trade-off between detection accuracy and embedded deployment constraints.

5.4 Specific Theory & System Design

5.4.1 Convolutional Neural Networks

CNNs are capable of learning hierarchical feature representations directly from raw vibration signals, enabling the detection of complex temporal patterns that may not be linearly separable as shown in the regression model. This section presents the theoretical motivation and design considerations regarding the use of CNNs for vibration based anomaly detection.

Motivation for CNN Based Anomaly Detection

While the regression model successfully demonstrated that impulsive events are linearly separable under controlled conditions, its performance is constrained by the use of manually engineered features and a linear decision boundary. In practical applications, vibration signals often exhibit non-linear, non-stationary behaviour influenced by a range of variables like operating conditions, vibration coupling across mechanical devices and environmental noise. Under these kinds of conditions, the assumptions required for linear separability are unlikely to hold.

CNNs provide a data-driven approach by learning discriminative features directly from the accelerometer signal data. Rather than relying on predefined measures, such as jerk RMS or p2p magnitude, convolutional filters in the network layers adapt during training to emphasise signal structures that are most relevant for anomaly detection. This enables the model to capture more subtle vibration signatures that may not be apparent through low order statistical features. The proposed introduction of CNNs therefore represents an increase in model capacity, justified by the observed limitations of the regression based method.

Time-series Representation Input

CNNs operate on fixed size input tensors, which means the continuous vibration signal data needs to be segmented into windowed samples. A tensor refers to a structured, multi-dimensional array used to represent data in a form suitable for neural network computation. A scalar is a zero-dimensional tensor, a vector is a one-dimensional tensor, and a matrix is a two-dimensional tensor. High order tensors extend this concept beyond the third dimension, allowing multiple channels to be represented simultaneously.

Similar to the regression model, the vibration time series is divided into short, overlapping windows. However, instead of computing hand-crafted features from these windows, the raw acceleration samples for each axis are passed directly into the neural network.

For the tri-axial accelerometer data, each input window can be represented as a matrix:

$$\mathbf{x} \in \mathbb{R}^{T \times C}$$

Where T denotes the number of samples in each window and C represents the number of channels ($C=3$ for tri-axial data). Then, in practise when multiple windows are being processed during training, these matrices are stacked along an additional dimension to form a three dimensional tensor:

$$\mathbf{X} \in \mathbb{R}^{N \times T \times C}$$

Where N is the number of windowed samples in the dataset.

This representation of the accelerometer data preserves temporal ordering and the relationships between axes. This allows the convolutional filters in the network to operate along the time dimension while simultaneously incorporating the information across the three axes. Windowing therefore provides localised snapshots of the systems behaviour while retaining the full raw signal structure required for hierarchical feature learning.

One Dimensional Convolution

In 1D-CNN structures, convolutional filters slide along the temporal dimension of the input signal, where each time, the filter analyses a small segment of the waveform. Conceptually, each convolutional kernel can be imagined as a small template or pattern that is repeatedly compared against the signal as it moves from the beginning to the end of the window.

Mathematically, a convolutional layer computes a set of feature maps according to:

$$y_k(t) = \sum_{i=0}^{K-1} w_k(i)x(t+i) + b_k$$

Where w_k represents the kth learnable kernel of length K, b_k is a bias term, and $x(t)$ denotes the input signal at time index of t.

Essentially, this operation performs a weighted comparison between a short segment of the vibration signal and the kernel. If the shape of the signal within that segment resembles the learned pattern stored in the kernel, then the output activation $y_k(t)$ becomes large. If the section of the signal does not resemble that pattern, the activation output remains small. The kernel is then shifted forward by one time step and the process repeats, producing a feature map that highlights where in time that particular kernel pattern occurs.

In the context of this research, these learned kernels have the ability to adapt to detect characteristic signal structures such as sharp impulsive spikes, short bursts in oscillation or gradual changes in amplitude. For example, one kernel might evolve to resemble the shape of a transient acceleration spike, resulting in a strong activation output whenever an impulsive disturbance occurs. Another learned kernel may resemble a small oscillatory waveform, where its high activation output suggests the presence of periodic vibration components.

Early convolutional layers act as automatic feature extractors, detecting simple local patterns directly from the raw data. As additional layers are stacked, later layers combine lower level activations into more complex representations. For instance, a deeper layer may learn that a specific combination of oscillatory behaviour and amplitude modulation corresponds to anomalous machine behaviour. In this way, the CNN constructs a hierarchical representation of features, progressing from a simple waveform fragment into a more complex and abstract description of the system's dynamics.

Comparatively, unlike the regression model where features are manually defined, convolutional kernels are learned directly from the training data. While this enables the network to discover discriminative vibration patterns that may not be obvious, the aforementioned limitations regarding data quality and quantity are a critical consideration.

Non-Linear Activation and Pooling

Following each convolutional operation, a non-linear activation function is applied. Without this non-linearity, the entire network would reduce down to a series of linear transformations, resulting in a model similar to the regression based approach.

The most commonly used activation function in this field of work is the Rectified Linear Unit (ReLU), defined as:

$$\mathrm{ReLU}(x) = \max(0, x)$$

In this context, the ReLU activation step acts as a gate. If a convolutional filter detects a pattern strongly, the response is preserved. If the response is weak or negative, it is suppressed to zero [47]. This allows the network to selectively choose meaningful vibration patterns while discarding insignificant responses. In anomaly detection, this means that strong features can propagate through the network, while background noise or irrelevant oscillation patterns are attenuated.

Pooling layers are typically inserted between convolutional layers to reduce the temporal resolution of the feature maps. In one dimensional analysis, max pooling selects the largest activation found within a small temporal neighbourhood. This operation retains the strongest detected response while discarding less dominant activations, effectively performing local downsampling of the signal representation.

Fault related events such as impacts or transient spikes may not occur at precisely the same time index within each window. Small timing shifts can arise in the data due to speed variations, sensor placement, or noise. With the addition of pooling, the model becomes less sensitive to these minor misalignments in the temporal dimension. Essentially, instead of memorising exact positions of faults, the network learns to recognise the presence of characteristic patterns within a local region.

In effect, convolution detects what pattern is present, ReLU enables non-linear feature interactions, and pooling reduces temporal resolution while preserving dominant responses. Together these operations allow the CNN to build hierarchical representations that are expressive and robust.

Output Layer and Classification

After a sequence of convolutional and pooling layers, the extracted feature maps are transformed into a compact representation of the input window. This is achieved by collapsing the final feature maps into a one dimensional vector and passing it through one or more fully connected layers. These layers combine all the learned convolutional features into a single scalar score, which represents the network's internal assessment of how strongly the input window exhibits anomalous or normal characteristics. The same binary anomaly detection is used here as in the regression model (Version 1A), since the output layer consists of a single neuron with a sigmoid activation function and the classification decision is obtained by applying a threshold to the estimated probability found.

Even though the final decision mechanism is mathematically identical to the one used in the logistic regression model, the distinction lies in how the single scalar score (z) is constructed. In logistic regression, this value is computed as a linear combination of manually engineered features. In contrast, in the CNN model, the scalar score is the result of multiple layers of learned convolutions, non-linear activations, and pooling operations applied directly to the raw tri-axial vibration data.

Computational Considerations

The increased representational capacity of the CNN model is accompanied by a corresponding increase in computational and memory requirements. In logistic regression, the inference stage consists of a small, fixed number of multiplication operations followed by a single non-linear activation. In contrast, during inference for the CNN model, a convolutional layer performs multiple matrix multiplication operations for every kernel position across the input window. For a kernel of length K, applied to a window of length T, the computational cost scales approximately with $K \times T$ per filter, increasing further with the number of filters and layers.

In addition to mathematical operation complexity, CNN models require storage to hold the intermediate feature maps between layers. These feature maps can occupy significant space, particularly when multiple filters and layers are used. Consequently, model depth, kernel size, stride, and channel count must be carefully selected when targeting real time deployment on embedded platforms.

Despite these constraints, CNNs still remain suitable for edge deployment when designed with limitation considerations. By restricting the number of convolutional layers, limiting the filter counts, and selecting compact kernel sizes, computational cost can be bounded while preserving sufficient representational capacity. Furthermore, fixed-point and quantised maths both reduce the memory footprint and inference latency, making real time execution feasible.

5.4.2 Quantisation & Deployment

Deep neural networks are typically trained using 32-bit floating point (FP32) arithmetic. In this representation, each parameter and activation in the network is stored using 32 bits, allowing for high numerical precision. While suitable and preferred in desktop setups, 32 bit representations are inefficient for resource constrained embedded systems such as the ESP32-S3. The primary constraints which motivate quantisation are:

- Memory footprint - Model weights stored as FP32 consume 4 bytes per parameter.
- RAM usage - Intermediate feature maps require floating point buffers.
- Computing latency - Floating point multiplication is much slower and more intensive than integer mathematics.

Intermediate feature maps are the multi-dimensional tensors produced by each convolutional layer during inference. They are required to propagate the input forward through each layer of the learned CNN, and they occupy RAM inside the tensor arena during real time execution. Although training is performed offline in Python, these intermediate activations are still generated during deployment since inference consists of a full forward pass through the network. Quantisation addresses these limitations by representing the neural network parameters and activations using integer formats, most commonly 8-bit signed integers (int8).

For example, a feature map of size 124×16 requires:

$$\text{FP32: } 124 \times 16 \times 4 \text{ bytes} = 7936 \text{ bytes.}$$

$$\text{Int8: } 124 \times 16 \times 1 \text{ bytes} = 1984 \text{ bytes.}$$

Uniform Affine Quantisation Model

TensorFlow Lite uses uniform affine quantisation, which maps continuous floating-point values to discrete integer values using a linear transformation [48].

Let

$$x \in \mathbb{R}$$

be a floating point value. Its quantised representation:

$$q \in \mathbb{Z}$$

Is defined by:

$$q = \text{round}(\frac{x}{S} + Z)$$

Where

- S = scale (positive real number),
- Z = zero-point (integer offset),
- round() = nearest integer operator

The inverse operation (dequantization) reconstructs a floating point approximation:

$$\hat{x} = S \cdot (q - Z)$$

Scale and Zero-Point

The scale defines the step size between quantisation levels and therefore controls both numerical resolution and dynamic range. Dynamic range refers to the total span of real-value amplitudes that can be represented by the quantised integer format without clipping. In an int8 system, only 256 discrete values are available, meaning that increasing the representable amplitude range increases the spacing between the quantisation levels. Consequently, a larger dynamic range reduces numerical resolution, while a smaller dynamic range improves precision but risks saturation and clipping for large amplitude events.

If int8 is used:

$$q \in [-128, 127]$$

Then the representable floating point range is:

$$x_{min} \approx S(-128 - Z)$$

$$x_{max} \approx S(128 + Z)$$

The zero-point (Z) ensures that floating point zero is exactly representable in the quantised domain:

$$x = 0 \Rightarrow q = Z$$

Without a zero point, quantisation would be symmetric around zero and could introduce systematic bias in the network.

Numerical Equivalence Between Keras and TFLite

When comparing FP32 Keras output to the quantised TFLite output, small discrepancies are expected due to rounding during optimisation, finite integer precision, rescaling error and activation saturation effects. However, if the mean absolute difference between the two is small (<<0.01), then quantisation is considered successful for binary classification tasks. For sigmoid outputs near 0 or 1, small numerical shifts typically do not affect classification accuracy, unless operating near the decision threshold.

Practical Considerations

When deploying the quantised model onto the ESP32, input tensors must be manually quantised using the input scale and zero-point provided by the TensorFlow Lite interpreter. These values are determined during the post training quantisation process by feeding a representative calibration dataset into the TF Lite converter. An incorrect or unrepresentative calibration dataset may lead to activation clipping, reduced sensitivity and increased quantisation error.

On the embedded platform, output tensors must be dequantized for interpretation, typically using a lightweight wrapper that applies the inverse affine transformation. Finally, all preprocessing steps performed on the ESP32, such as windowing, normalisation, scaling and ordering must match the training pipeline exactly. Any mismatch between offline preprocessing and embedded inference will introduce prediction error independent of quantisation effects.

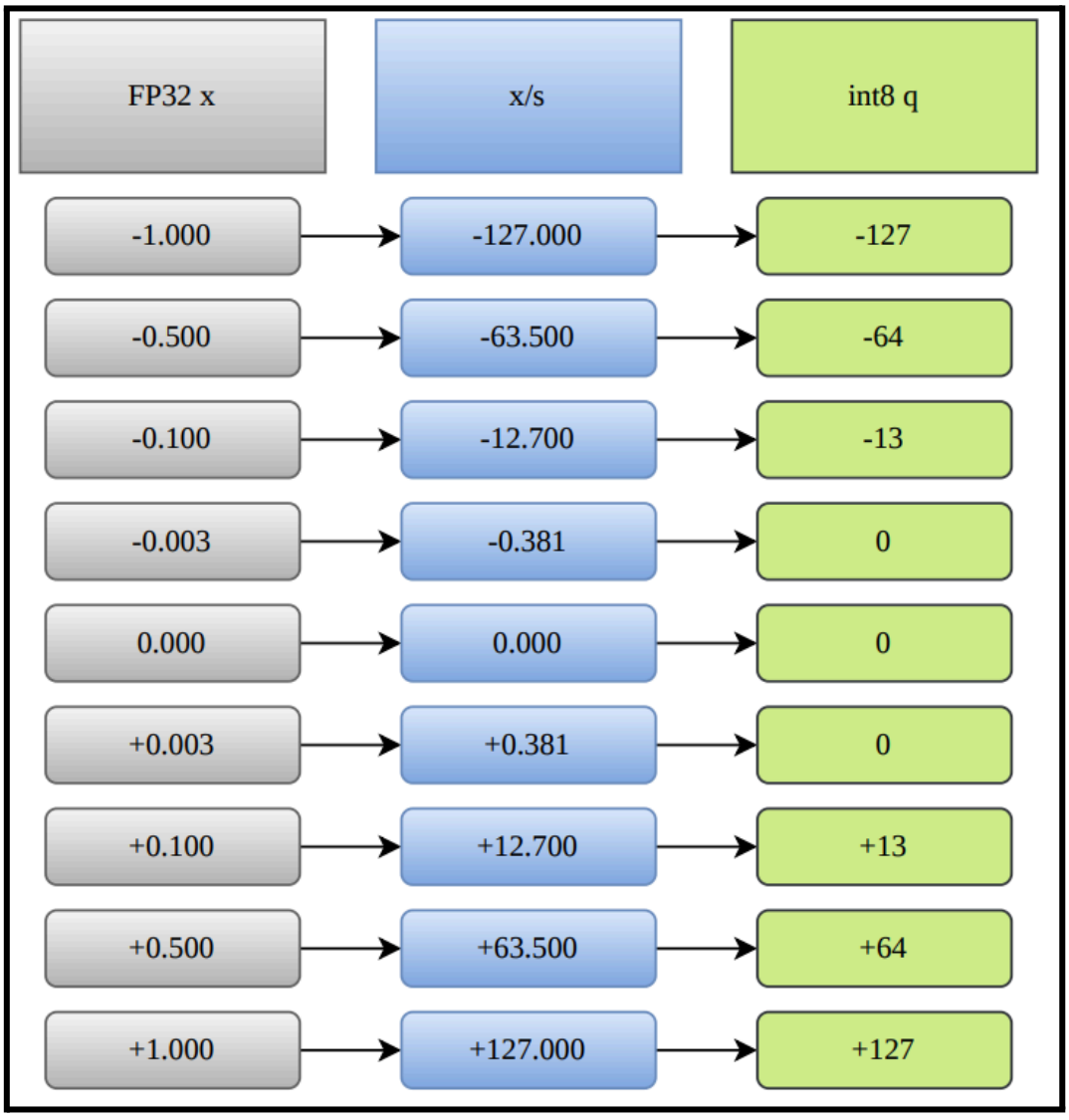


Figure 5.8. Mapping of FP32 values in the range [-1,1] to discrete int8 quantised values.

5.5 Implementation & Methodology

This section serves as the complete implementation workflow used to develop the experimental 1D-CNN based anomaly detector system for the desk based, earthquake style vibration dataset. The controlled environment allowed model behaviour to be evaluated without the additional complexity of a rotating motor.

5.5.1 Experimental 1D-CNN Model Training

The same method as seen in Version 1A was utilised, tri-axial acceleration data was collected over USB serial using the Python-based logger. The objective at this stage was not to maximise classification accuracy, but to implement a reproducible and computationally bounded CNN model that could later be deployed on embedded devices.

Tap Event Labelling

To generate labels without manual annotation, jerk magnitude was used as a physics-based anomaly indicator. Jerk magnitude, jerk median and jerk median absolute deviation (MAD) were computed as:

$$|j_{MAG}(t)| = \sqrt{j_x^2 + j_y^2 + j_z^2}$$

$$j_{MED} = \text{median}(j_{MAG})$$

$$j_{MAD} = \text{median}(|j_{MAG} - j_{MED}|)$$

where,

$$j_x(t) = \frac{dx(t)}{dt}, \quad j_y(t) = \frac{dy(t)}{dt}, \quad j_z(t) = \frac{dz(t)}{dt}$$

Rather than using a simple threshold on jerk magnitude, a robust outlier detection method was implemented using the median and MAD. This was chosen instead of mean and standard deviation since the vibration data collected contains heavy-tailed transient events, making MAD more robust to outliers.

The modified Z-score was computed as:

$$z_k = \frac{|j_{MAG}(t)| - j_{MED}}{1.4826 \cdot j_{MAD}}$$

The scaling factor of 1.4826 approximates the standard deviation for normally distributed data [49].

A threshold of $z_k > 20$ was selected after some tuning. This high threshold was deliberately chosen to capture only extreme transient events while avoiding mislabelling moderate vibration fluctuations. Lower thresholds were evaluated but resulted in excessive false positive data and label noise. The value of 20 therefore reflects a tradeoff between capturing sufficient anomaly samples and keeping the label column clean.

Listing 5.15 *Tap candidate detection using modified z-score*

```
dt = 1.0/SAMPLE_RATE_HZ

#Jerk calculation
X = df[["x","y","z"]].to_numpy()

jerk = np.diff(X, axis=0, prepend=X[:1]) / dt  # m/s^3
jerk_mag = np.sqrt(np.sum(jerk**2, axis=1))

j_med = np.median(jerk_mag) #normal jerk magnitude
j_mad = np.median(np.abs(jerk_mag - j_med)) #typical fluctuations due to
noise

z_score = (jerk_mag - j_med) / (1.4826*j_mad)  #scaling by 1.4826
estimates the standard deviation

j_thresh = 20.0 #An event is ~8 standard deviations above normal
df["tap_candidate"] = (z_score > j_thresh)

print("Tap candidate samples:", df["tap_candidate"].sum())
```

Window Segmentation

The time series data was then segmented into fixed length overlapping windows:

- Window length (N): 128 Samples,
- Stride: 16 samples.

At a sampling rate of 200Hz, each 128 sample window corresponds to 0.64 seconds worth of signal context. This window size was chosen to balance two competing factors. First, the window must be long enough to capture the full temporal structure of impulsive events. Secondly, the window must remain short enough to preserve temporal locality and minimise memory usage.

A stride of 16 samples (80% overlap) was selected to increase training sample density without excessively increasing computational cost. High overlap improves anomaly localisation and reduces the chance of events being missed in between windows.

To reduce boundary mislabelling, only the central 40% of each window was evaluated when assigning the anomaly labels. This design choice was considered to try to avoid events occurring near window edges from contaminating adjacent windows.

Signal Preprocessing

Each window underwent pre-window mean removal.

$$X_w = X_w - \mu_w$$

Where μ_w is the mean across the time dimension for each axis.

Mean removal was implemented to eliminate DC bias and static gravity components. This ensures that the network focuses on dynamic vibration content rather than static orientation effects. No additional filter was applied to avoid attenuating high-frequency transient components by mistake.

Listing 5.16 *Per-window Mean removal*

```
def preprocess_windows_mean_remove(Xw):
    mu = Xw.mean(axis=1, keepdims=True)  # (num_windows,1,3)
    Xw = Xw - mu
    return (Xw)
```

Dataset Preparation

A 80/20 train/test split was selected to ensure adequate data availability for model training. Stratified sampling was used to preserve the strong class imbalance present in the dataset.

Due to this imbalance, class weights were applied during the training stage of the model. False negatives (missed anomalies) are more critical than false positives in PdM systems. By weighting the anomaly class more heavily, the loss function penalises missed anomalies more strongly, encouraging the model to prioritise this sensitivity.

CNN Architecture

The architecture was intentionally kept compact to allow for easy debugging and embedded deployment.

Table 5.5. *1D-CNN architecture.*

Layer	Type	Parameters
1 (Block 1)	Conv1D	Filters: 8, Kernel Size: 5
2 (Block 1)	Activation.	ReLu
3 (Block 1)	MaxPooling	Pool size: 2
4 (Block 2)	Conv1D	Filters: 16, Kernel Size:5
5 (Block 2)	Activation	ReLu
6 (Block 2)	MaxPooling	Pool size: 2
7	GlobalAveragePooling1D	-
8	Dense Layer	1 Neuron
9	Activation	Sigmoid
10	Optimiser	Adam
11	Loss	Binary Cross-Entropy

A kernel size of 5 was selected to capture short duration temporal patterns in the accelerometer data, without excessively increasing the parameter count. The use of only two convolutional layers was also intentional to limit computational complexity and the memory footprint.

GlobalAveragePooling1D() (GAP) was chosen instead of Flatten() to significantly reduce the parameter count and mitigate overfitting. This layer averages each feature map over time, preserving feature presence while reducing dimensionality.

Binary Cross-Entropy was used as the loss function due to the binary classification objective of 0, no anomaly or 1, anomaly. Adam was selected as the optimiser to enable stable and efficient optimisation.

Quantisation and Deployment Pipeline

To enable deployment on the ESP32 using TensorFlow Lite for Microcontrollers, full integer quantisation was applied. Quantisation transforms floating-point values into 8-bit integer representations. A representative dataset of 500 windows was provided during calibration to ensure accurate dynamic range mapping.

Listing 5.16 *Quantisation using TFLite Converter.*

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]

def representative_data():
    #function to provide representative samples of data
    #500 used to provide enough samples to capture normal variability
    for i in range(min(500, len(X_train_n))):
        yield [X_train_n[i:i+1].astype(np.float32)] #(1,N,3) shape

converter.representative_dataset = representative_data #Calibration
#Forces all operators to be int8, no hybrid models using floats
converter.target_spec.supported_ops =
[tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8

tflite_model = converter.convert()

with open("tap_cnn_int8.tflite", "wb") as f:
    f.write(tflite_model)

print("Saved tap_cnn_int8.tflite, size =", len(tflite_model), "bytes")
```

To verify that post-training integer quantisation was applied correctly and that the quantised model preserved the behaviour of the original FP32 network, a direct output consistency check was performed between the FP32 Keras model and the fully quantised int8 TFLite model.

A random subset of 20 windows was extracted from the normalised test set, and evaluated using both models. First the FP32 Keras model produced probability outputs for each sample. The int8 TFLite model was then loaded in using the TensorFlow Lite Interpreter and its quantisation parameters were extracted from the input and output tensor metadata.

Using these parameters, affine quantisation was applied to the FP32 input windows prior to inference. The output int8 was then converted back to FP32 using the inverse mapping function (dequantise). The TFLite output probabilities were then compared against the Keras model predictions for the same windows. For transparency, the first five values from both model outputs were printed side-by-side, and the mean absolute error between them was calculated. This explicit quantise/dequantise procedure ensured that the numerical transformation between FP32 and int8 was valid.

Listing 5.17 *Quantisation validation check.*

```
#Pick test windows
X_sample = X_test_n[:20].astype(np.float32)

#FP32 Keras output
p_float = model.predict(X_sample).ravel()

#TFLite int8 output
interpreter = tf.lite.Interpreter(model_path="tap_cnn_int8.tflite")
interpreter.allocate_tensors()
in_det = interpreter.get_input_details()[0]
out_det = interpreter.get_output_details()[0]

in_scale, in_zp = in_det["quantization"] #IN S & Z
out_scale, out_zp = out_det["quantization"] #Out S & Z

def quantize(x): # affine quantisation
    q = np.round(x / in_scale + in_zp)
    return np.clip(q, -128, 127).astype(np.int8)

def dequantize(q): #inverse affine quantisation
    return (q.astype(np.float32) - out_zp) * out_scale

p_tflite = []
for i in range(len(X_sample)):
    xq = quantize(X_sample[i:i+1])
    interpreter.set_tensor(in_det["index"], xq)
    interpreter.invoke()
    yq = interpreter.get_tensor(out_det["index"])
    y = dequantize(yq)[0,0]
    p_tflite.append(y)

p_tflite = np.array(p_tflite) #TFlite output

print("float:", p_float[:5])
print("tflite:", p_tflite[:5])
print("mean abs diff:", np.mean(np.abs(p_float - p_tflite)))
```

5.5.2 Embedded Experimental 1D-CNN Model

To validate real-time edge inference, the trained 1D-CNN model was deployed directly onto the ESP32-S3 using TensorFlow Lite for Microcontrollers (TFLM). The embedded firmware integrates the quantised CNN model, deterministic sampling logic, sliding-window buffering, and a real-time visual indication via the preinstalled NeoPixel.

Quantised CNN Representation

The file “model_data.cc” contains the fully quantised TensorFlow Lite model stored as a static byte array. The array represents the serialised .tflite model converted to full 8-bit integer format. During the export, TensorFlow Lite performs post-training quantisation using learned scale and zero-point parameters. Embedding the model as a compiled byte array has several advantages when it comes to embedded deployment.

- Static memory allocation: The model is stored in flash memory and does not require dynamic loading,
- Deterministic execution: Inference does not rely on external storage or runtime model parsing,
- Portability: The same compiled firmware image always contains the same trained model.

The MicroTFLite wrappers (ModelInit, ModelSetInput, ModelRunInference, ModelGetOutput) interpret this byte array, allocate tensors within the predefined tensorArena, and execute inference using fixed-point mathematics.

The tensor arena size was set to 12 kB based on the model metadata. 12 kB is enough to provide sufficient memory for the models feature maps while remaining within the RAM constraints of the ESP32-S3. If the tensor arena is undersized, model initialisation fails, reinforcing the importance of architecture size control during the design of the CNN model. For debugging purposes, a short line of code was written as an allocation verification check to immediately detect insufficient tensor arena memory.

Listing 5.18 *Tensor Arena allocation check.*

```
if (!ModelInit(tap_cnn_int8_tflite, tensorArena, kTensorArenaSize)) {
  Serial.println("Model Initialization FAILED.");
  while (true) delay(1000);}
```

Preprocessing - Per Window Mean Removal

This experimental CNN model does not rely on global dataset normalisation parameters. Instead, preprocessing is performed locally within each inference pass. For every 128-sample window, the mean of each axis is computed:

$$\mu_x = \frac{1}{N} \sum_{i=0}^{N-1} x_i$$

and subtracted from the signal:

$$x_i' = x_i - \mu_x$$

This per window mean removal eliminates DC bias and static gravity components, ensuring that the network focuses on only the dynamic vibration behaviour. The decision to use window-level normalisation rather than global normalisation reflects the experimental environment of the desk based exercise. Since the accelerometer is not fixed to a motor its orientation and static offsets may vary in between recordings.

Deterministic Sampling

In the embedded code, sampling is governed by a fixed-rate scheduler operating at a fixed 200 Hz. The use of a “next deadline” accumulator based on micros() ensures deterministic sampling intervals and correct handling of the timer wraparound. This is critical since the CNN model was trained on uniformly sampled time-series data, so introducing any sampling jitter would alter the temporal structure learned during training.

A ring buffer size N = 128 samples per axis maintains the sliding window. Inference is executed every 16 new data samples (stride = 16), resulting in overlapping windows and inference updates every 0.08 seconds.

Real Time Validation via NeoPixel

To provide immediate physical feedback, the anomaly decision is mapped to the onboard NeoPixel, where a green light was used to represent normal behaviour and a red light to represent anomalous behaviour. This hardware level visualisation allows for rapid validation of the embedded inference pipeline without requiring serial monitoring. The LED response confirms that the model is executing correctly, proper thresholding behaviour and real-time responsiveness.

5.6 Experimental Results & Discussion

This section presents the experimental results of the proposed 1D-CNN anomaly detection system and its deployment on embedded hardware. The objective is to assess the classification capability and its practical feasibility when executed in a controlled environment on an ESP32-S3.

5.6.1 Experimental 1D-CNN Model Performance

Dataset Characteristics

The following tri-axis dataset was fed into the labelling pipeline, and 110 windows out of 10,458 were identified as taps. This indicates a positive rate of about 1.05%. The dataset therefore exhibits the expected strong class imbalance, where anomalous events are rare relative to normal operation. To address this imbalance, class weighting was applied during training:

$$\{0 : 1.0, \quad 1 : 94.07\}$$

This weighting significantly penalised false negatives relative to false positives.

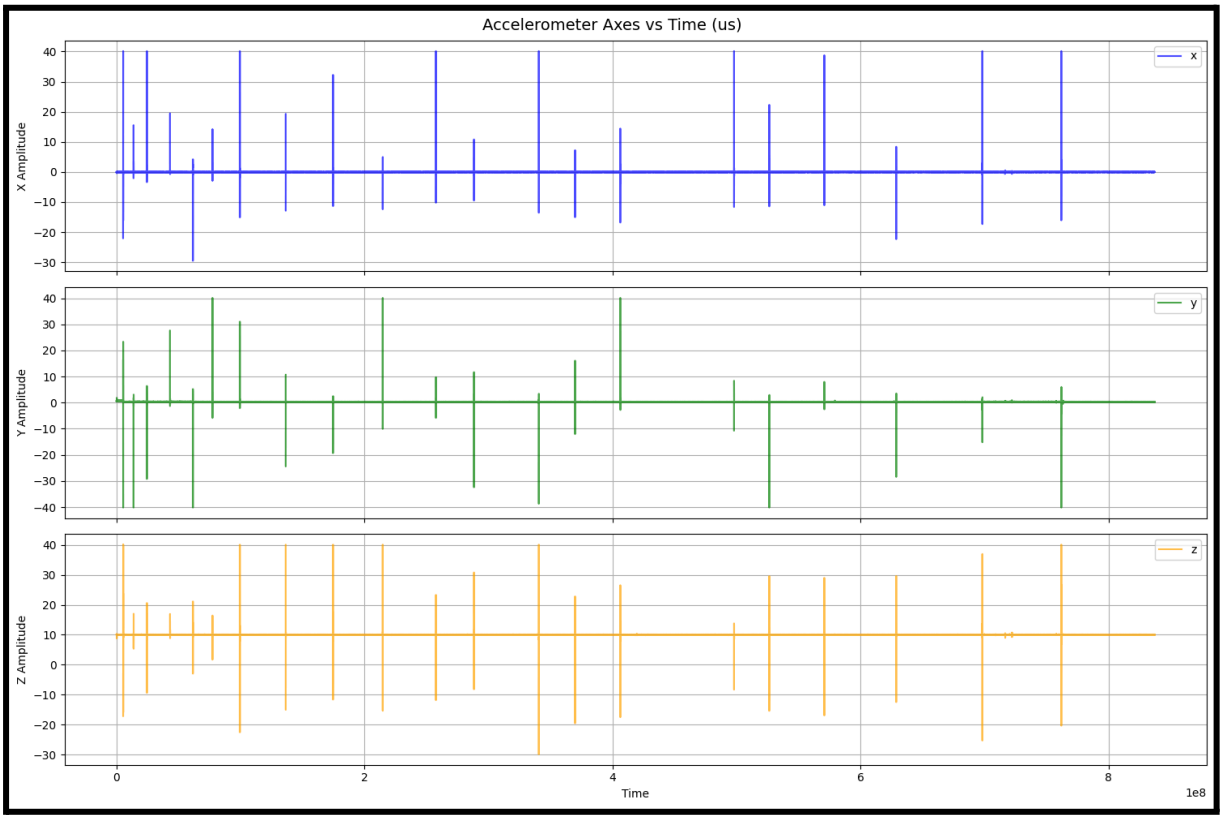


Figure 5.9 Raw triaxial acceleration time-series used for windowed anomaly labelling.

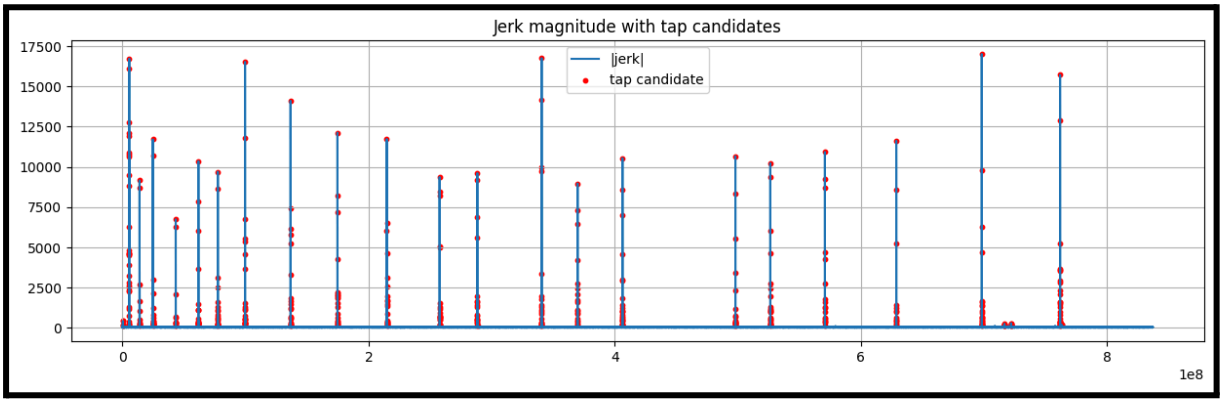


Figure 5.10 Jerk magnitude computed from triaxial acceleration with threshold based anomaly candidates highlighted.

Confusion Matrix

Using a validation derived threshold of:

$$\tau = 0.1573$$

The resulting confusion matrix was:

$$\begin{bmatrix} 2054 & 16 \\ 0 & 22 \end{bmatrix}$$

The classification report was:

***Table 5.6**. Classification report for Version 1B*

	Precision	Recall	F1-score
0	1.0000	0.9923	0.9961
1	0.5789	1.0000	0.7333

The threshold was found using Youden's J to specifically try to minimize false negative rates. Using this threshold, the following results were obtained:

- True Positives (TP): 2054,
- False Negatives (FN): 0,
- False Positives (FP): 16,
- True Negatives (TN): 22.

The model achieved perfect recall on the anomaly class, indicating that no tap events were missed in the test set. This is a critical result, since in predictive maintenance, false negatives mean missed anomalies which are completely unacceptable. The precision value of 0.579 indicates that some normal windows were misclassified as anomalies. However, given the extreme class imbalance in this dataset, even a small number of false positives can substantially affect the precision metric. The weighted average F1-score of 0.993 reflects strong overall performance.

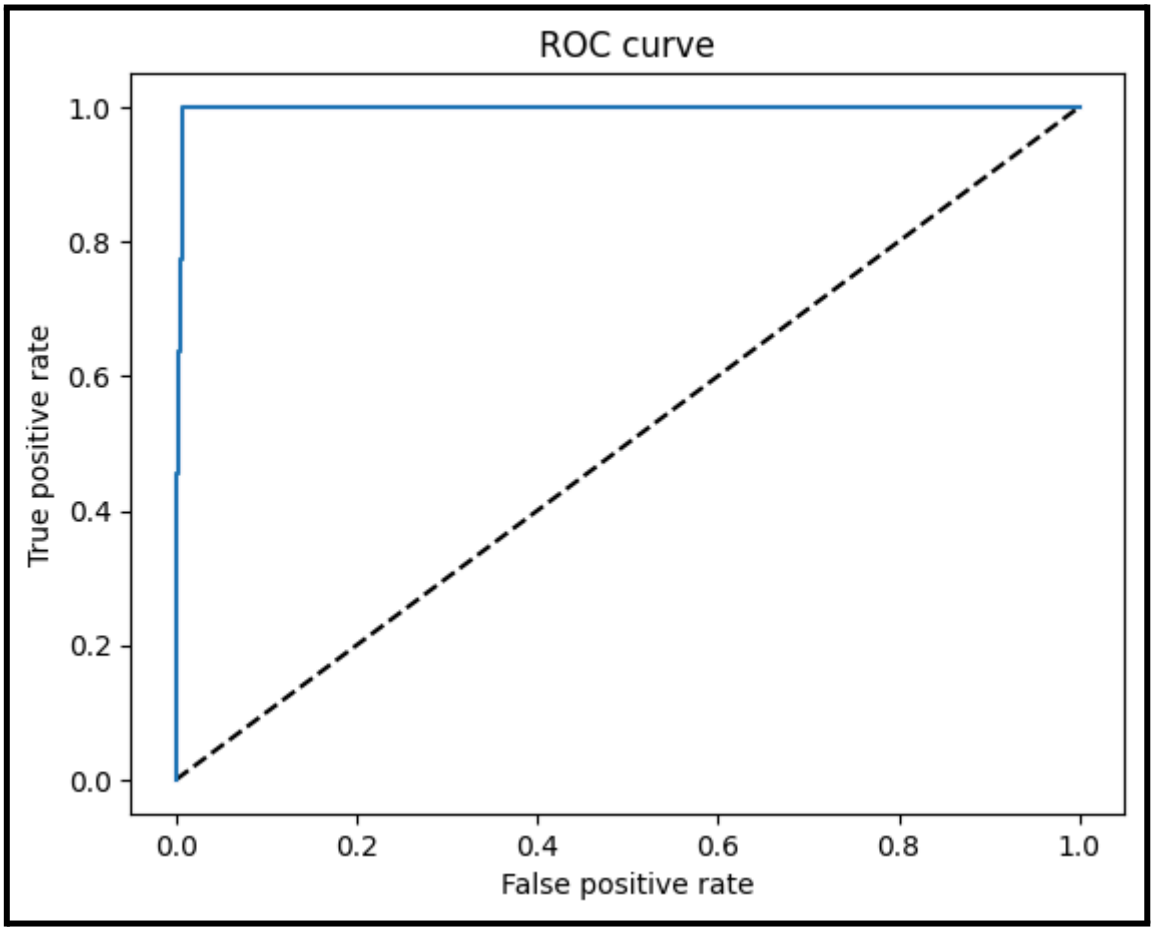


Figure 5.11 ROC Curve for Version 1B

Quantisation Consistency

To verify embedded deployment, the float model output was compared against the int8 TensorFlow Lite model output. The output values for the float model ranged between ~0.113-0.118, while the quantised outputs were ~0.121. The mean absolute difference between the two models was 0.00578. The tiny error between the two model outputs confirms that post-training quantisation does introduce some model degradation, but the error is so small that it is negligible.

5.6.2 Embedded Experimental 1D-CNN Model

This section of the results evaluates the Experimental 1D-CNN model when deployed on the ESP32-S3 and executed in real time using live accelerometer data. The primary objectives here are to (i) verify correct execution of the quantised CNN model on embedded hardware and (ii) assess whether the system responds reliably to impulsive taps under real time conditions.

Real Time Serial Output

Table 5.7 shows a snippet of the serial output captured during embedded operation. The reported probability corresponds to the sigmoid output of the CNN model, and the anomaly flag reflects the threshold-based classification.

During offline validation, the optimal classification threshold was selected using ROC based analysis. However, when deployed in real-time operation, the baseline probability observed during normal operating conditions was consistently elevated to ~0.35. This shift is attributable to the minor differences between controlled dataset conditions and live deployment. Some of these conditions include slight variations in sensor angle, ambient micro-vibrations or quantisation effects. Therefore, rather than relying strictly on the validation derived threshold, the classification boundary was adjusted to 0.5.

This adjustment was made with an informed estimate through real time observations, since normal behaviour was around 0.35-0.43 and tap events were as high as 0.81. The resulting threshold provided a clear margin between normal fluctuations and impulsive disturbances. Importantly, this does not indicate model instability, but rather shows the sensitivity of sigmoid outputs to distribution shifts in small datasets. The clear separation between normal and anomalous behaviour with the new threshold demonstrates the model's discriminative capability under deployment conditions.

The serial log reveals these two distinct operating regimes. The transition from ~0.42 to ~0.82 in samples 10->11, represents a substantial increase in the activation function. This confirms that the convolutional filters successfully capture impulse-like temporal patterns, even when executed in quantised int8 format.

Table 5.7. *Embedded CNN inference probabilities and decision threshold behaviour.*

Sample	Probability	Anomaly
1	0.347656	0
2	0.35938	0
3	0.429687	0
4	0.347656	0
5	0.386719	0
6	0.414062	0
7	0.320313	0
8	0.320313	0
9	0.429687	0
10	0.429687	0
11	0.816406	1
12	0.816406	1

Real Time LED Validation

The anomaly decision shown in **Table 5.7** was directly mapped to the onboard NeoPixel. The green light represents the system when there are no anomalies detected and the red light indicates the opposite. Visual testing confirmed that the LED remained green during stationary or slow movements, the LED transitioned to red immediately upon tap events and that the response latency was visually instantaneous. Given a sampling rate of 200 Hz and a stride of 16 samples, inference was executed every 0.08 seconds. This latency makes the system's response appear instantaneously during operation. The rapid LED transition therefore confirms that the embedded inference pipeline is capable of detecting and reacting to impulsive anomalies in near real time.

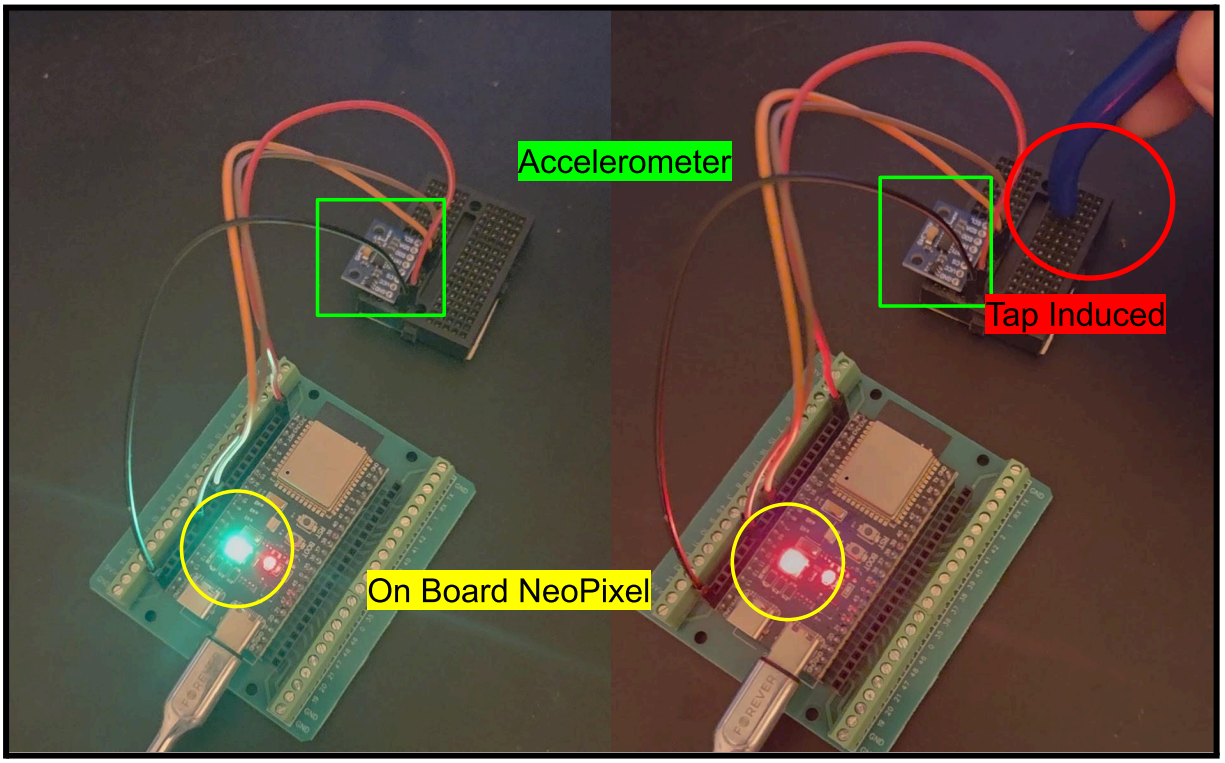


Figure 5.12. Real time NeoPixel LED response during embedded anomaly detection (green = normal, red = anomaly).

Version 2A - Physical Anomalies

The next stage of the investigation focuses on evaluating anomaly detection performance under realistic operating conditions. While Version 1 demonstrated the feasibility of performing anomaly detection using an embedded device, the experiments were conducted with controlled vibration signals that contained obvious impulsive events.

Version 2A therefore addresses the next research question RQ2 - Real Motor Operation Anomaly Detection, which investigates whether a trained deep learning model can detect anomalous behaviour from real motor vibration signals.

This stage represents a significant increase in realism. Unlike the controlled experiment, real motor vibration signals are influenced by structural resonance, mechanical vibrations and environmental disturbances. These effects introduce variability into the signal which can obscure fault signatures.

Version 2A introduces physically generated anomalies on a running motor, enabling the embedded model to be tested under dynamic conditions which are more representative of actual PdM applications. The objective is to determine whether anomaly detection models trained on controlled datasets can be deployed and detect the same anomaly type in real time.

5.7 Specific Theory & System Design

5.7.1 Physical Anomaly Generation

In the tap experiments Version 1A and Version 1B, weak labels could be generated using jerk magnitude and statistical thresholds since taps on the accelerometer would produce large, isolated spikes. However, in a rotating system, vibration signals exhibit persistent oscillatory behaviour even under normal operation. As a result, threshold based weak labelling becomes unreliable. Background vibration may contain high frequency components that resemble transient events, while genuine fault signatures may be subtle or distributed over time.

Although unsupervised learning methods could in principle be used to detect deviations from normal operation, such approaches are beyond the scope of this project. The system being developed is targeted towards small-scale industrial applications, where large volumes of historical fault data are not available. Consequently, a controlled method of gathering labelled anomalous data is required.

To address this, a physical anomaly generation strategy is introduced. The objective is to create repeatable, physically meaningful disturbances in the motor system while maintaining full control over their timing and occurrence. By inducing controlled transient disturbances in the casing which supports the accelerometer, labelled anomaly windows can be generated. During the periods of induced disturbance, the system state can be assigned to label anomalous behaviour, while periods without disturbance are labelled as normal.

The disturbances introduced in this manner are designed to mimic transient fault-like behaviour, such as bearing slip or erratic contact events. In mechanical systems, bearing related faults often produce short duration, irregular bursts of high frequency vibrations as rolling elements can momentarily lose smooth contact before re-engaging. By artificially exciting the accelerometer casing to create comparable transient vibration patterns, in theory, a supervised dataset containing anomalous and normal operation can be constructed.

It is important to acknowledge that such induced anomalies represent only a small fraction of possible motor faults. The theoretical expectation is that a model trained on a single motor operating at a fixed rotational speed and only subjected to a specific disturbance mechanism will exhibit limited generalisation capability. Specifically, the learned representation may become strongly associated with the vibration characteristics corresponding to that operating speed. Consequently, making the model over-specialised (overfit) to that specific motor speed and disturbance configuration.

That being said, the purpose of this version is not to produce a universally robust industrial detection system. Rather, its purpose is to establish a controlled experiment framework in which supervised learning can be evaluated under realistic rotating conditions. By introducing repeatable, physical disturbances, the model can be assessed on its ability to distinguish structured motor vibrations from induced anomalies.

The staged approach presented, from physics based desk experiments, to controlled anomaly generation on rotating machinery, ensures that an increase in system complexity is only introduced after each challenge has been addressed. The design presented in this section aims to address the underlying data acquisition and labelling challenges. Physical anomaly generation therefore serves as a bridge between controlled proof of concept testing and real world deployment scenarios.

5.8 Implementation & Methodology

5.8.1 Motor 1D-CNN Model - Physical Anomalies

Following the desk based CNN validation performed in Version 1B, the next stage was to train and deploy a 1D-CNN model directly on vibration data captured from the operating motor. In this environment, the vibration signal is continuous, periodic, and dominated by rotational dynamics.

Physical Anomaly Labelling

To obtain supervised labels under rotating conditions, the data logging rig was modified to include a secondary motor that vibrates the accelerometer casing. This method creates repeatable transient disturbances intended to simulate "bearing-like" slip events, by injecting erratic vibration bursts on top of the normal motor vibration [50].

Labelling was implemented from the software side as a state-based process, rather than a signal threshold process:

- When the anomaly actuator is off, the dataset logs Anomaly = 0,
- When the anomaly actuator is on, the dataset logs Anomlay = 1.

This approach produces labels aligned exactly to the time in which the physical injection mechanism is running. The rig configuration is shown in **Figure 5.12**, the servo on/off states are shown in **Figure 5.13** and, and the fully assembled rig shown in **Figure 5.14.**

***Listing 5.19**. Anomaly latching system on embedded device.*

```
  if (pressedDebounced(PIN_ANOM_BTN)) {
    anomaly_latched = !anomaly_latched;
    applyServoAndLed(anomaly_latched);
  } else {
(...)
   Serial.print(t_us);
    Serial.print(',');
    Serial.print(log_val);
    Serial.print(',');
    Serial.print(event.acceleration.x, 6);
    Serial.print(',');
    Serial.print(event.acceleration.y, 6);
    Serial.print(',');
    Serial.print(event.acceleration.z, 6);
    Serial.print(',');
    Serial.println(anomaly_latched ? 1 : 0);
```

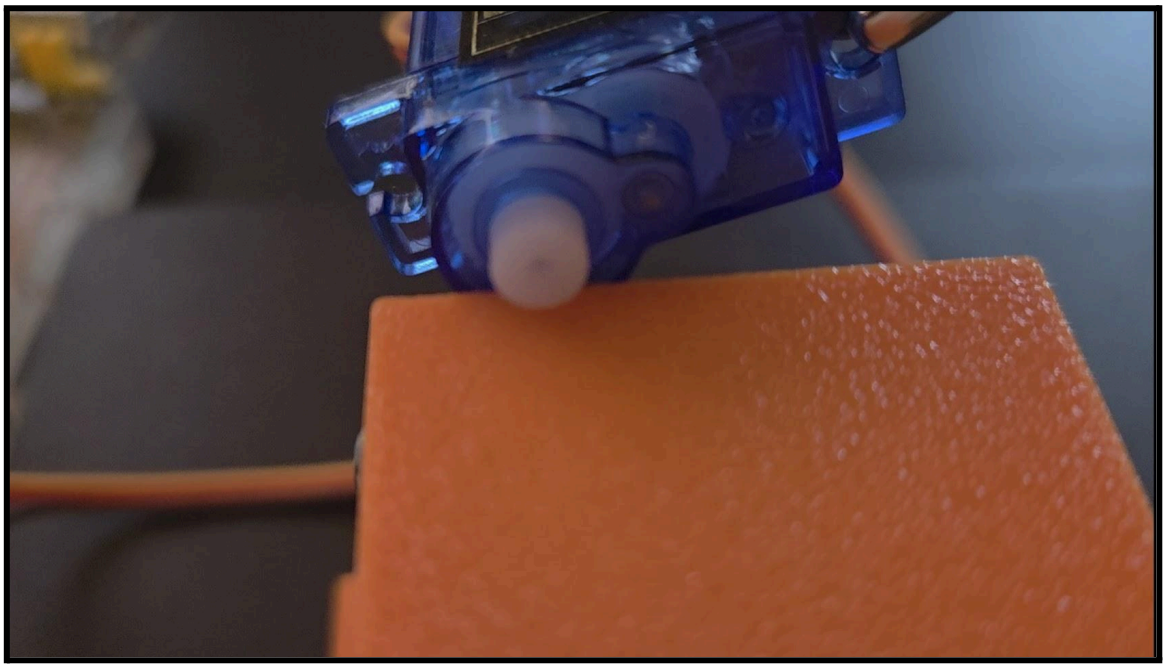


Figure 5.13. Servos position on the accelerometer casing.

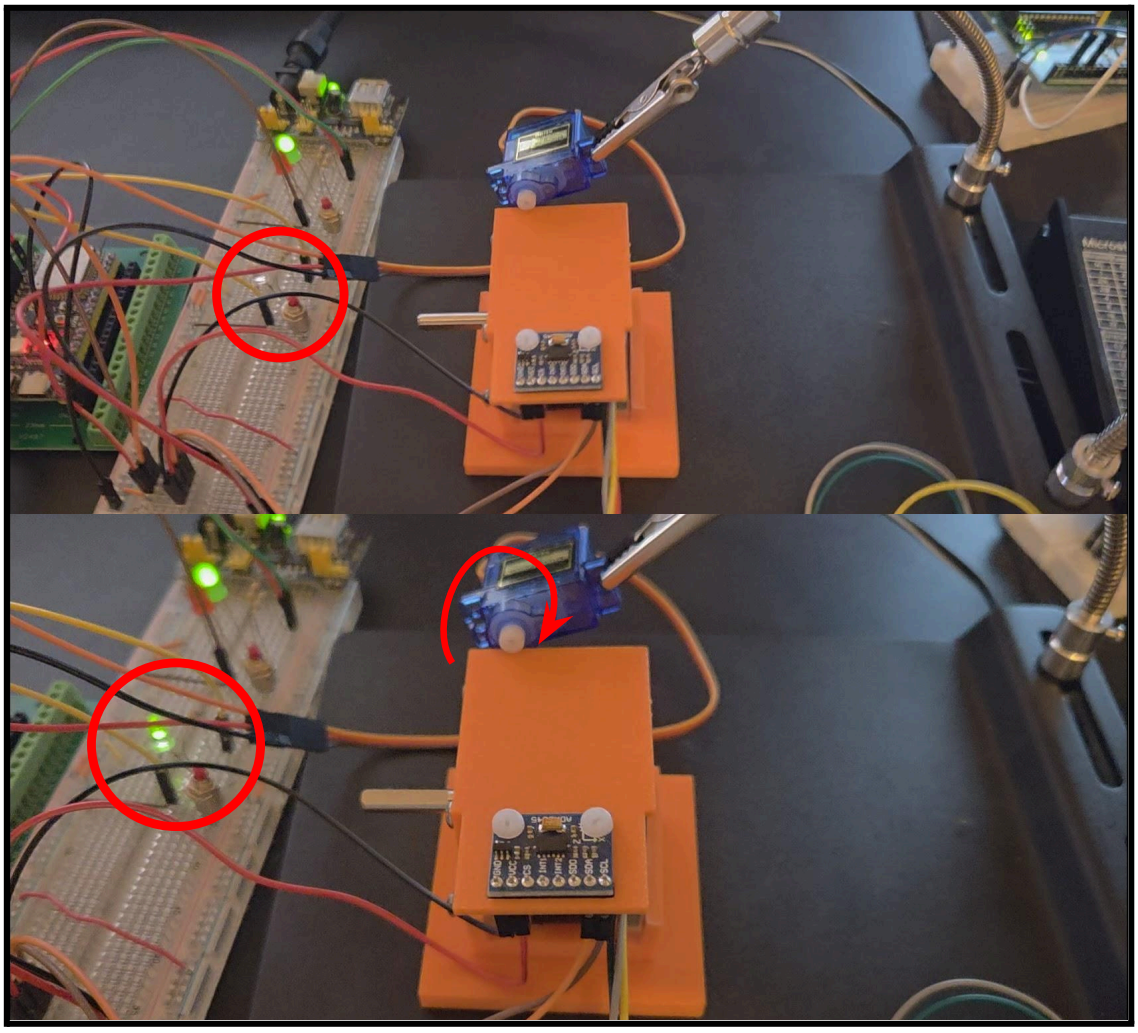


Figure 5.14. Servo on and off states.

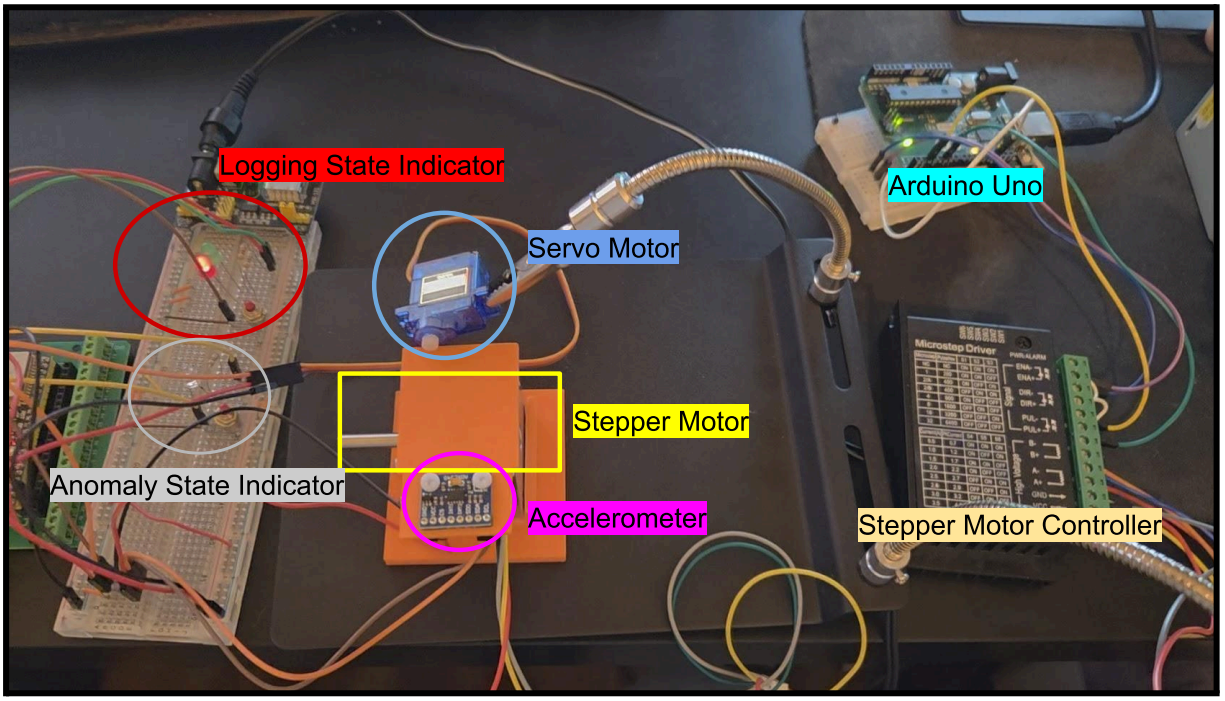


Figure 5.15. Fully assembled rig with annotations.

Data Processing and Window Generation

The window length and stride were both set to 256 samples, corresponding to each window representing 1.28 seconds of signal. This meant that there was no overlap between the sliding windows. This configuration was intentionally selected here to simplify the time-based splitting and to avoid leakage between train/validation/test subsets since the dataset is temporally correlated.

An input window label was assigned using a fractional coverage rule. If a window contained at least 10% anomalous samples, the entire window was labelled as anomalous. This logic was directly implemented in `make_windows()` via the setting `frac_thresh=0.10`.

Listing 5.20. *Window generation and labelling using fractional coverage logic.*

```
def make_windows(X, y, n, stride, frac_thresh=0.10):
    Xw, yw = [], []
    for start in range(0, len(X) - n + 1, stride):
        end = start + n
        Xw.append(X[start:end])

        frac = np.mean(y[start:end] == 1)
        if frac >= frac_thresh:
          yw.append(1)
        else:
          yw.append(0)
    return np.array(Xw), np.array(yw)
```

Time-Based Train/Validation/Test Split

To preserve the temporal structure of the signal, the dataset was split in chronological order rather than random shuffling. This decision prevents leakage between subsets, which is particularly crucial for vibration data where adjacent windows are highly correlated. 70% of the data was picked for training, 15% for validation and the last 15% was left completely untouched for testing.

Listing 5.21. *Time based Train/Validation/Test split in chronological order.*

```
#Time-based split
n_total = len(Xw) #Number of windows
n_train = int(0.70 * n_total)
n_val = int(0.15 * n_total)

X_train, y_train = Xw[:n_train], yw[:n_train]
X_val, y_val = Xw[n_train:n_train+n_val], yw[n_train:n_train+n_val]
X_test, y_test = Xw[n_train+n_val:], yw[n_train+n_val:]

print("Train/Val/Test:", len(X_train), len(X_val), len(X_test))
print("Pos rates:", y_train.mean(), y_val.mean(), y_test.mean())
```

Since the split preserves chronological ordering, classical stratified sampling was not applied. Stratification requires randomised allocation to ensure that there is class balance across all three subsets, which in turn would destroy the temporal continuity in the signal. Instead, class balance was verified post-split by explicitly computing and comparing the positive label rates in each subset.

An additional advantage of the controlled experimental setup here is that the anomaly events were manually injected with the press of a button. This allowed anomaly occurrences to be deliberately spaced throughout the recording period. As a result, it was ensured during data acquisition that each chronological segment of the dataset contained some positive anomaly samples. This does not constitute data leakage, as anomaly patterns were not engineered to match training data. Without sufficient positive samples in the testing segment, reliable estimation of recall, precision and ROC analysis would not be possible.

Global Normalisation

Unlike the desk CNN model, which used per-window mean removal, the motor CNN model used global z-score normalisation computed using the training set data. Global normalisation differs fundamentally from per-window mean removal. Rather than centering each window independently, a single global mean and standard deviation are computed from the training data subset and applied to all samples. This preserves all slow variations in vibration magnitude while ensuring numerical stability. During deployment, the same global parameters are hardcoded into the embedded firmware, ensuring that the statistical distribution of inference data matches the training set.

***Listing 5.22**. Global normalisation using training data subset.*

```
mu = X_train.reshape(-1, 3).mean(axis=0)
sd = X_train.reshape(-1, 3).std(axis=0) + 1e-8

def norm(X):
    return (X - mu) / sd

X_train_n = norm(X_train)
X_val_n = norm(X_val)
X_test_n = norm(X_test)

print("mu:", mu, "sd:", sd)
```

CNN Architecture and Training Configuration

For this Version, the CNN architecture was slightly expanded when compared to the desk-based model (Version 1B). This was done in order to increase the representational capacity of the model for the more complex task. While the overall structural design was similar, the number of filters and depth of the network were increased.

Specifically, the motor CNN consisted of three convolutional blocks instead of two (see **Table 5.5**). Each convolution was followed by a ReLU activation and MaxPooling1D. The network concluded the same way as the desk experiment with GlobalAveragePooling1D, Dense(1) and Sigmoid activation.

The increase in filter count (8,16 -> 16, 32, 64) was introduced to allow the model to learn a richer hierarchy of vibration features. The motor-based dataset exhibits much greater vibration content, more complex vibration patterns and wider variability in operating behaviour.

He-uniform kernel initialisation was also applied to the deeper convolutional layers to stabilise variance propagation through the ReLU activations. This feature reduces the risk of vanishing or exploding activations during training.

Importantly, even though the model's capacity was increased, the architecture remains lightweight and deployable. This configuration maintains direct comparability with the previous experiment, while providing that needed additional power to represent the increased complexity of the motor based data.

ROC-Based Threshold Selection

After training, a decision threshold was selected using ROC analysis on the validation subset. This produced a high threshold for anomaly detection. This high threshold does not prioritise missed anomalies, instead it limits the amount of false positives found during testing, which is more useful when testing the systems sensitivity during early testing stages.

This is important for Version 2A, since the normal motor vibration energy is high and structured, so the probability distribution is typically shifted upward compared with Version 1B. As a result, threshold selection must be dataset specific and cannot be assumed to remain at 0.5.

Listing 5.23. *Global normalisation using training data subset.*

```
#threshold on VAL dataset
p_val = model.predict(X_val_n).ravel()
fpr, tpr, thr = roc_curve(y_val, p_val)
J = tpr - fpr
best_thr = thr[np.argmax(J)]
```

Conversion and Quantisation

The same post training quantisation and float to int8 validation procedure described previously was applied to the motor CNN model. Reusing the validated pipeline ensures numerical consistency between the FP32 and embedded int8 implementations and maintains continuity across experimental stages.

5.8.1 Embedded Deployment: 1D-CNN - Physical Anomalies

The quantised motor CNN model was deployed to the ESP32-S3 the same way. Real-time inference was implemented with deterministic sampling, ring-buffer windowing and global normalisation which matches training. The same NeoPixel alerting system was implemented.

Embedded Preprocessing

Unlike the desk experiment, there was another file extracted from the Python training environment. The “norm_params.h” file was used to keep track of the global z-score normalisation parameters of kMu[3] for axis means and kSd[3] for axis standard deviations.

Each sample is normalised as:

$$x' = \frac{x - kMu_x}{kSd_x}$$

and then streamlined into the model input tensor. This mirrors the offline training pipeline exactly, where global normalisation is used instead of per-window mean removal.

***Listing 5.24**. Global normalisation on live data.*

```
static bool run_cnn_inference(float &prob_out) {
  //motor model was trained with GLOBAL z-score normalization using
kMu/kSd, dont do per window mean removal

  int idx = 0;

  //Feed input tensor into model in time order: [t0 x,y,z]...[tN-1
x,y,z]
  for (int t = 0; t < N; t++) {
    //Normalization: match training: (x - mu) / sd
    const float x_norm = (get_x(t) - kMu[0]) / kSd[0];
    const float y_norm = (get_y(t) - kMu[1]) / kSd[1];
    const float z_norm = (get_z(t) - kMu[2]) / kSd[2];

    //ModelSetInput() quantize internally
    if (!ModelSetInput(x_norm, idx++)) return false;
    if (!ModelSetInput(y_norm, idx++)) return false;
    if (!ModelSetInput(z_norm, idx++)) return false;
  }

(...)
```

Inference Scheduling

Inference was executed once every 256 new samples were gathered. Given a sampling frequency of 200 Hz, this corresponds to an inference interval of:

$$\frac{\text{STRIDE}}{f_s} = \frac{256}{200} = 1.28s$$

Unlike Version 1B, which employed overlapping windows to achieve rapid detection updates, this motor CNN model was configured with a stride equal to the window length. This configuration removes overlapping inference and ensures that each prediction is computed on a completely new segment of vibration signal data. This was selected for two reasons, firstly, non overlapping windows reduce the number of forward passes required, simplifying real time execution. Secondly, each prediction corresponds to a distinct 1.28 second block of data, making it much easier to interpret the models responses during experimental testing.

The tradeoff that comes with this simplification is the increased detection latency when compared to Version 1B. Nevertheless, at this stage the objective was not to optimise detection speed but to validate that the embedded CNN architecture could successfully detect physically induced motor anomalies under real operating conditions.

The anomaly decision threshold was set to 0.957, which is based on the validation set ROC curve analysis. This very high threshold prioritises precision by reducing false positives during normal, continuous motor operations. Exhaustive threshold optimization was not the primary focus of this experimental phase, therefore the selected value was used during live testing to prevent false positive anomaly flags.

5.9 Experimental Results & Discussion

5.9.1 1D-CNN Model - Physical Anomalies

Visual Signal Characteristics

Figure 5.16 below presents the raw z-axis acceleration during motor operation without anomaly labels overlaid. The figure is left without labelling to show that the injected anomalies are clearly visible as high amplitude transient disturbances superimposed on the normal periodic signal. The magnitude of these anomalies is significantly greater than the baseline oscillatory behaviour of the rotating system. It is important to note that these anomalies are deliberately strong and controlled. In real industrial environments faults in motors may be more subtle and less visually apparent. This motivates the later introduction of augmented anomaly generation and more complex modelling strategies.

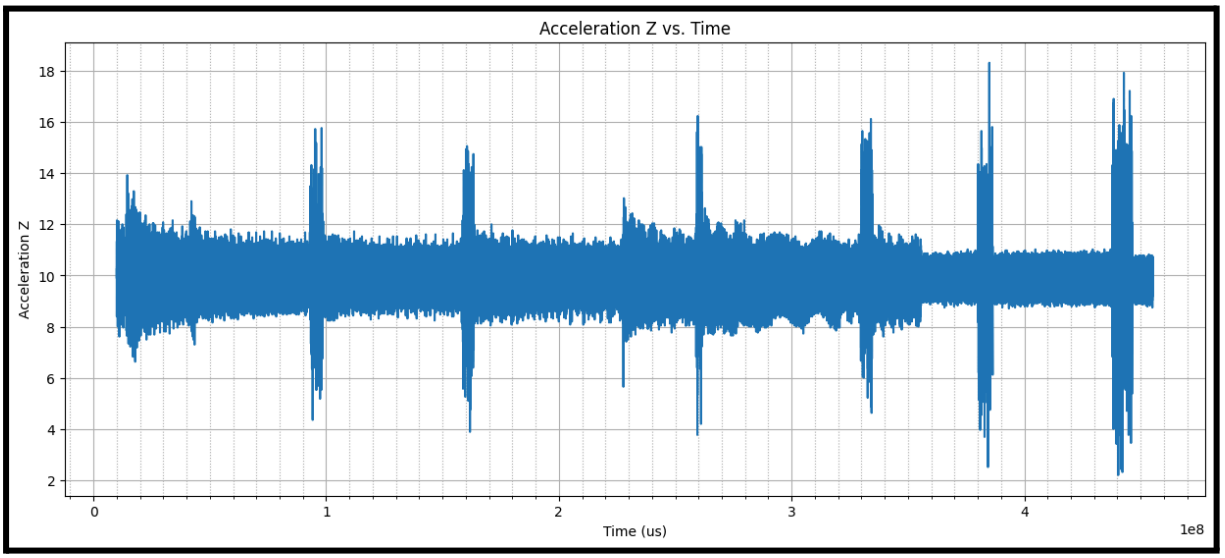


Figure 5.16 Raw z-axis acceleration during motor operation showing visually distinct high amplitude transient disturbances.

Figure 5.17 shows all three axes (x,y,z) with the normal operation highlighted in blue and anomalous operation in red. This plot confirms that the servo-induced vibrations generated repeatable, high-energy transient patterns, suitable for supervised learning.

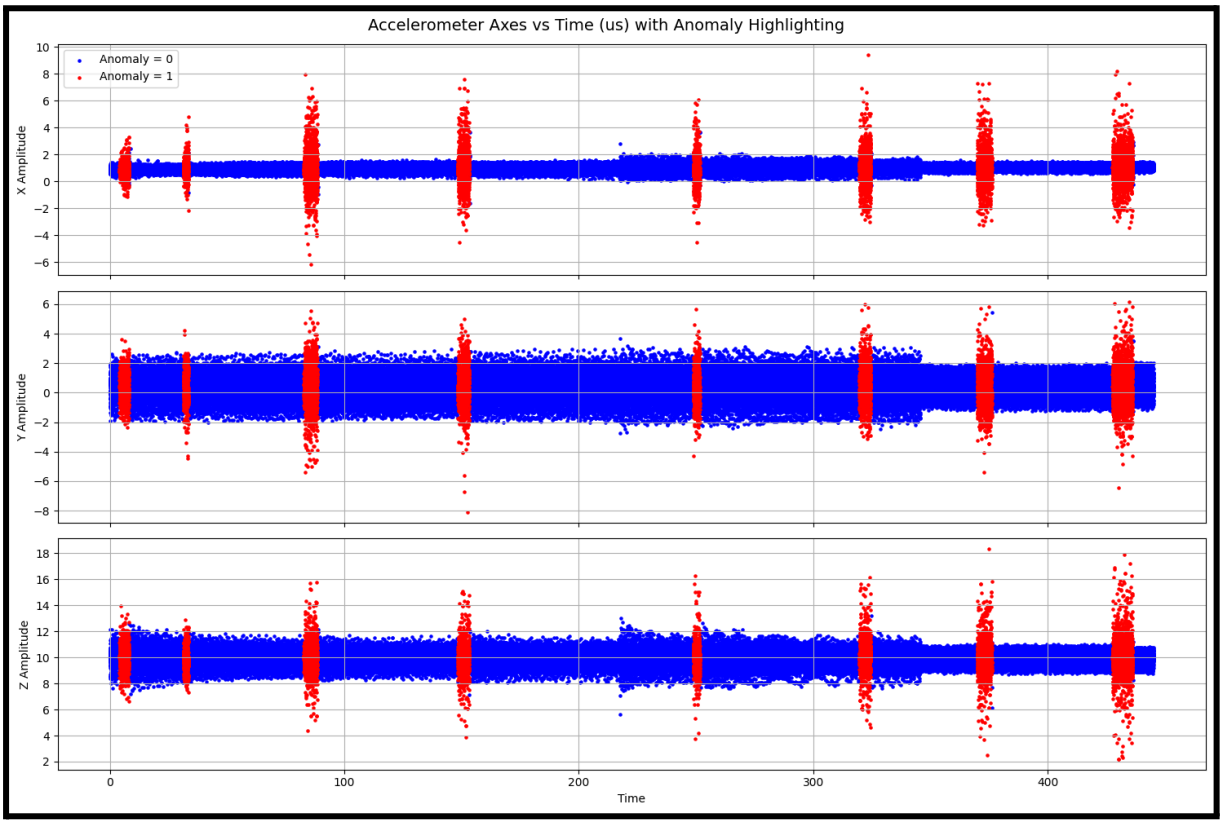


Figure 5.17 Triaxial acceleration time series highlighting clear separation between normal motor operation and induced anomalous events.

Windowing

The 88,483 raw samples were segmented into 345 non-overlapping windows of 256 samples each. Each window represents 1.28 seconds of vibration data. Of the 345 windows created, 37 of them were labelled as anomalous and the other 308 were labelled nominal. This corresponds to a positive window rate of 10.7%.

The anomaly rate remains intentionally low to reflect realistic class imbalance conditions, while still providing sufficient positive examples for supervised learning. This dataset therefore maintains the "rare anomaly" aspects of real world predictive maintenance data without rendering model training statistically unstable.

Global Normalisation Parameters

Global z-score normalisation using the training data produced the following axis-wise statistics:

$$\mu = [0.9658, \quad 0.3856, \quad 10.0057]$$

$$\sigma = [0.3900, \quad 0.8616, \quad 0.6175]$$

The z-axis mean of approximately 10 m/s^2 is consistent with acceleration due to gravity, which confirms correct sensor orientation and scaling.

The non-zero means found for the x and y axes indicate that there was a slight sensor tilt during data acquisition. Rather than being problematic, this is physically realistic. Small mounting misalignments produce a constant bias component, which are appropriately handled by the global normalisation step and do not affect the models performance. The figure below shows how these misalignments can occur in the setup. The screw on the right side of the accelerometer is slightly looser than the one on the left, therefore creating a slight tilt in the acceleration readings, which throws the mean off of 0.

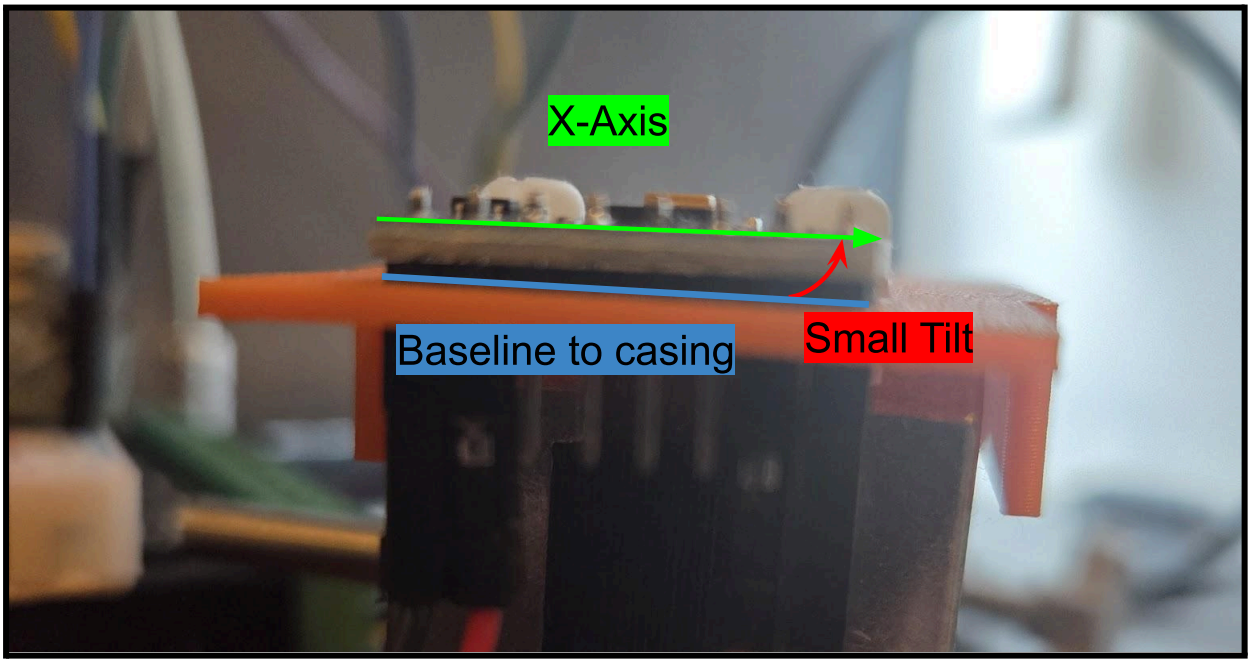


Figure 5.18 Example of slight accelerometer mounting tilt.

Classification Performance

The confusion matrix for the test subset is shown below:

$$\begin{bmatrix} 46 & 0 \\ 1 & 6 \end{bmatrix}$$

These values correspond to:

- Accuracy: 98.11%
- Precision (anomaly): 1.00
- Recall (Anomaly): 0.8571
- F1-score (Anomaly): 0.9231

The model correctly classified all 46 normal windows and 6 out of 7 of the anomalous windows correctly. This means that only one labelled anomalous window was missed and the model predicted that it was in normal operation.

The absence of false positives indicates that the selected ROC derived threshold prioritises precision, ensuring that normal motor operation is not incorrectly flagged as anomalous. The slight reduction in recall reflects the conservative nature of the derived threshold.

Given the very small number of anomalous windows in the test set (7), performance metrics have to be interpreted carefully. With such a limited positive sample size, a single misclassification affects the recall and F1-score greatly. Nevertheless, the results demonstrate that there is strong separability between the two classes under controlled physical anomaly conditions.

Again, it's important to emphasise that at this stage, maximising classification performance was not the primary objective and the missed anomaly is not detrimental. The purpose of this experiment was to validate that the CNN architecture can learn physically induced motor anomalies; be quantised; execute reliably using an int8 model; and that the inference pipeline is stable and outputs in real time.

The model was trained on a single motor speed and a single, deliberately obvious injected anomaly type. As such, it does not represent a generalised industrial fault detector. Broader evaluation and robustness testing are conducted later using augmented datasets, which are specifically designed to introduce greater variability in operating conditions and fault characteristics. Therefore, under these experimental constraints, the achieved performance is considered sufficient to validate end-to-end system feasibility.

Quantisation Verification

To validate the effectiveness and reliability of the int8 deployment model, predictions from the FP32 Keras model were compared against the quantised TFLite model on a subset of test windows.

The following results were obtained:

```
float: [0.10601502 0.07437405 0.06806184 0.07348387 0.06808041]
tflite: [0.109375   0.0703125  0.06640625 0.0703125  0.06640625]
mean abs diff: 0.0023118681
```

The mean absolute difference between the FP32 and int8 outs was 0.00231. This extremely small deviation confirms that the post training quantisation preserved the predictive behaviour of the neural network. The quantised model therefore provides enough numerical stability to render stable inference on the embedded device without significant degradation in output probability.

5.9.2 Embedded Deployment: 1D-CNN Physical Anomalies

To ensure that the embedded system classifier (ESP32) did not receive any implicit cues about anomaly timing, the servo activation mechanism was controlled independently of the ESP32. The servo trigger button was connected to the Arduino, which was responsible for both driving the motor and the anomaly injector. Serial commands were issued through a separate device for the same reason. This way, the ESP32 running the quantised CNN model only received the raw accelerometer data and had no direct knowledge of the motor or servo state. The two sets of figures below show (i) a closeup of the accelerometer and servo configuration and (ii) an overview of the complete rig for both states, no anomaly and anomaly detected.

During live testing, with the motor running and the servo inactive, the NeoPixel on the ESP32 remained green, indicating normal operation (**Figure 5.18**). When the servo was activated, inducing the learned anomaly pattern, the NeoPixel turned red (**Figure 5.19**).

This behaviour was consistent across repeated trials and confirmed that the embedded inference pipeline successfully detected the induced anomalies. Since the stride was set to 256 samples, inference only occurred every 1.28 seconds. This inference interval was visibly noticeable during testing. In contrast to Version 1B, which had a stride of 16 samples (0.08s update rate), this configuration required the servo to remain active for almost an entire inference window before the anomaly classification was updated.

This was expected given the non overlapping window configuration. The observed delay does not reflect computational latency, but rather the intentional scheduling of inference at discrete 1.28 second intervals.

No serial probability logs were recorded for this experiment since the real time LED response provided clear qualitative confirmation. The quantised CNN model executed successfully on the ESP32, the global normalisation method worked, the model detected the physically induced anomaly pattern during live motor operation. The complete pipeline of sensing-> processing -> decision functioned as designed.

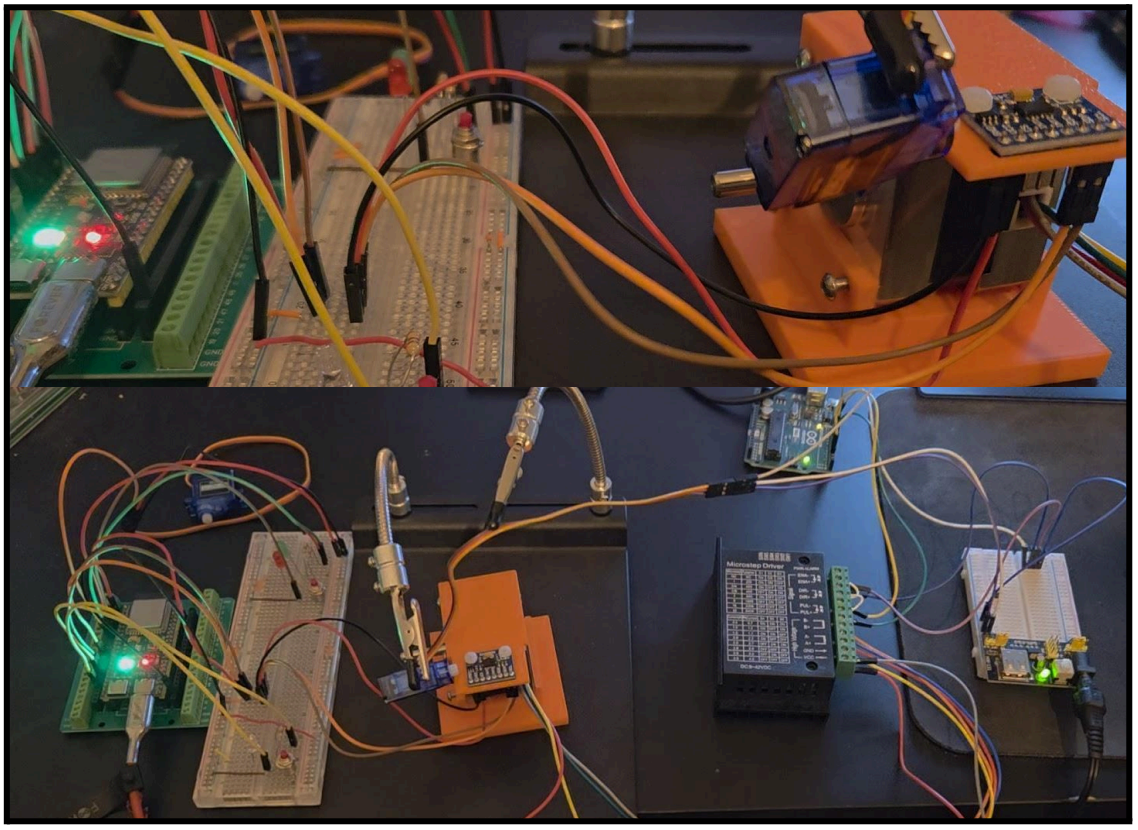


Figure 5.19. Embedded system during normal operation with servo inactive (LED green).

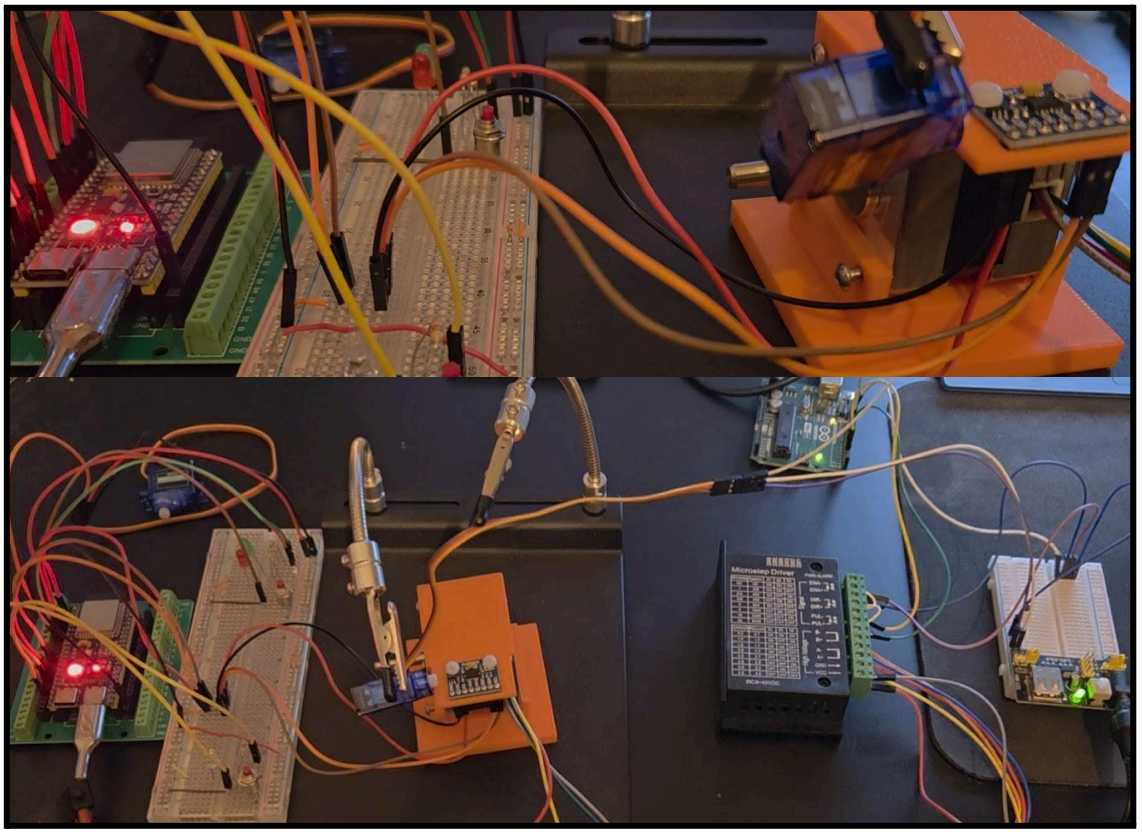


Figure 5.20. Embedded system during normal operation with servo active (LED red).

Version 2B - Lengthened Data

While Version 2A demonstrated that anomaly detection from real motor vibration signals is feasible on an embedded device, the dataset used was limited in size. In practical PdM applications, obtaining large volumes of labelled fault data is often difficult, particularly in small scale industrial environments.

Version 2B therefore continues to address RQ2 - Real Motor Operation Anomaly Detection, but with a specific focus shift. The objective now is to evaluate whether simple data lengthening techniques, like duplication or temporal concatenation can provide a meaningful increase in training data.

From a machine learning perspective, increasing the dataset size is generally associated with improved model generalisation. However, when additional data is generated by stitching existing signals together, there is a risk that the model may learn artificial temporal patterns or become biased. This can lead to severe overfitting or reduced robustness when exposed to unseen vibration behaviour.

To evaluate these effects, Version 2B investigates the impact of signal duplication and dataset lengthening. This stage serves as a controlled experiment to test whether simple data expansion strategies can help with data scarcity issues.

Importantly, this investigation also contributes to the progression towards RQ3 - Robust Embedded Predictive Maintenance System. Since the final system must operate reliably under constrained data availability, data preprocessing techniques that improve the performance of the model will be incorporated into the full system implementation.

5.10 Specific Theory & System Design

A rotating motor operating at a constant speed produces a quasi periodic vibration signal. This means that the signal exhibits behaviour which is almost, but not quite fully periodic. In these systems, the quasi-periodic signal is dominated by a fundamental rotational frequency f_r and its harmonics. In steady-state operation:

$$x(t) = \sum_{k=1}^{\infty} A_k \sin(2\pi k f_r t + \phi_k) + \epsilon(t)$$

where:

- f_r is the dominant rotational frequency,
- A_k are the harmonic amplitudes,
- ϕ_k are phase offsets,
- $\epsilon(t)$ represents noise and mechanical variability.

Since the system is approximately periodic, segments of vibration separated by integer multiples of the rotation period exhibit similar phase characteristics. This periodic structure enables controlled recombination of signal segments without introducing physically impossible discontinuities.

The core idea is to estimate the dominant rotational frequency, identify the time indices corresponding to consistent phase points, slice the signal into segments aligned with identical phase states and finally, reassemble these segments in a new sequence to create longer synthetic recordings.

If two signal segments begin at equivalent phase positions, then concatenating them would produce minimal discontinuity:

$$x_1(t_n) \approx x_2(t_n)$$

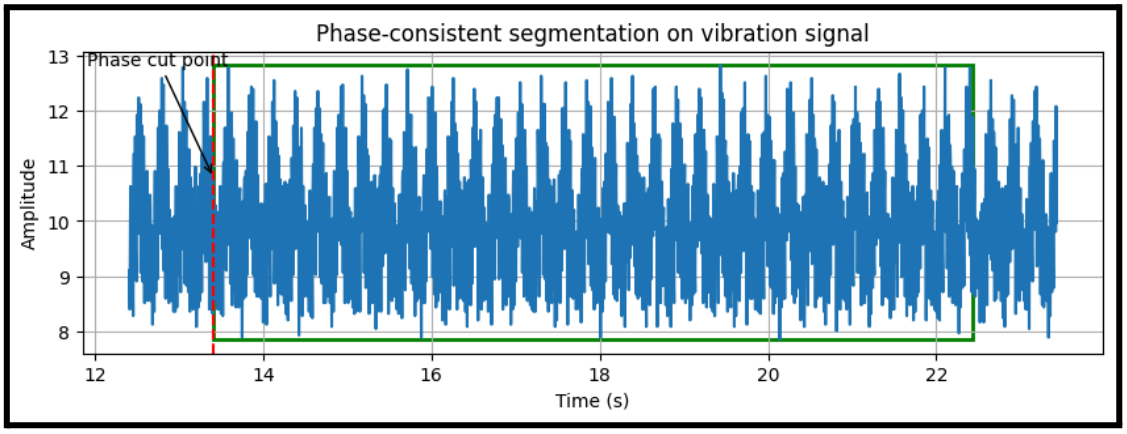


Figure 5.21. Quasi periodic vibration signal with phase consistent segmentation point indicated.

Even with phase alignment, small amplitude or noise variations may exist between segments. A short overlap crossfade window smoothes the transition:

$$x_{join}(t) = (1 - w(t))x_1(t) + w(t)x_2(t)$$

where, $w(t) \in [0, 1]$ is the smooth transition window. This crossfade method ensures continuity in amplitude, slope and tries to minimise artificial impulses at segment boundaries.

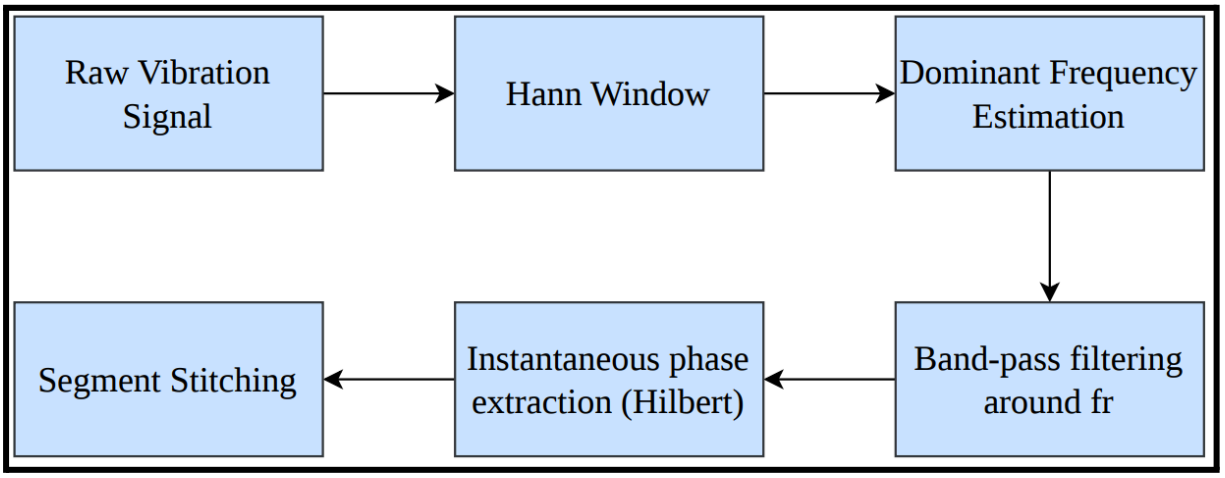


Figure 5.22. Phase-consistent signal lengthening workflow.

5.11 Implementation & Methodology

Dominant Frequency Estimation

To identify the phase aligned cut off point for stitching, the dominant rotational frequency (f_r) was first estimated using a Fast Fourier Transform (FFT). Only the first 30 seconds of the vibration data were used to estimate this to try to capture the steady-state behaviour of the motor.

Before computing the FFT, the signal was mean centered to remove any DC offset and multiplied by a Hanning window to reduce spectral leakage [51], [52]. Spectral leakage occurs when a finite length time segment does not contain an integer number of periods. This leakage results in discontinuities at the segment boundaries, where peaks are smeared and frequency estimations lose reliability. The Hanning window smoothly tapers the segment to near-zero at both ends, reducing boundary discontinuities and concentrating energy more tightly around the dominant frequency.

The dominant peak within a bounded frequency range (1-100 Hz) was selected as the fundamental rotational frequency. This frequency provides a reference for defining "same phase" cut off points used for stitching.

Listing 5.25 *Function used to estimate the fundamental frequency.*

```
def estimate_fr_hz_fft(x, fs, fmin=1.0, fmax=100.0):
    x = x - np.mean(x)
    n = len(x)
    X = np.fft.rfft(x * np.hanning(n))
    f = np.fft.rfftfreq(n, d=1.0/fs)
    band = (f >= fmin) & (f <= fmax)
    k = np.argmax(np.abs(X[band]))
    return float(f[band][k])

z = df["z"].to_numpy(np.float32)
fr = estimate_fr_hz_fft(z[:int(30*fs_hz)], fs_hz, fmin=1.0, fmax=100.0)
print("Estimated fr (Hz):", fr)
```

Bandpass Isolation

To compute reliable phase information, the vibration signal was bandpass filtered around the estimated fundamental frequency. The raw vibration signal contains noise and harmonics which can distort the instantaneous phase if used directly.

$$f_{lo} = f_r - \Delta f$$

$$f_{hi} = f_r + \Delta f$$

The bandwidth Δf was defined as a relative fraction (25%) of f_r with a minimum absolute bandwidth constraint to ensure filter stability. The minimum bandwidth prevents unstable filters when f_r is small or when the band becomes too narrow. In practice, overly narrow bands can lead to numerical issues with ringing and poor robustness to frequency drift.

A 4th order Butterworth filter was selected because it provides a maximally flat passband response. A maximally flat passband means that, within the passband region, the magnitude response of the filter has no ripple. In practical terms, this means that frequencies within the desired band are passed with uniform amplitude, with no oscillatory amplification or attenuation. The choice of a 4th order filter represents a balance between frequency selectivity and numerical stability [53].

Filtering was then applied using filtfilt [54], which performs forward and reverse filtering. This configuration yields zero phase filtering, which means that the phase distortion introduced by the forward pass is automatically cancelled by the reverse pass. This is critical since the phase is later used to determine stitching points, so a phase shift would bias the cut locations.

Listing 5.26 *Bandpass function.*

```
def bandpass(x, fs, f_lo, f_hi):
    nyq = 0.5 * fs
    f_lo = max(0.1, f_lo)
    f_hi = min(0.99*nyq, f_hi)
    b, a = butter(4, [f_lo/nyq, f_hi/nyq], btype="band")
    return filtfilt(b, a, x)
```

Instantaneous Phase Estimation

To compute phase aligned cut points, the instantaneous phase of the bandpassed signal was estimated using the Hilbert transform [55]. The Hilbert transform constructs an analytic representation of a real valued signal:

$$x_a(t) = x(t) + j\mathcal{H}\{x(t)\}$$

The instantaneous phase was then obtained as:

$$\phi(t) = \mathbf{unwrap}(\angle x_a(t))$$

Phase unwrapping converts the wrapped angle into a continuous phase trajectory by adding and subtracting 2π when discontinuities occur. This allows consistent detection of phase cycles over long signal datasets.

Cut points were chosen at the phase wrap events corresponding to increments of 2π. These indices were obtained by tracking integer phase cycle counts. To improve the stitching consistency, only cut points with a positive slope in the bandpassed waveform were retained. This reduced the probability of selecting random points where the signal crosses the same phase state in different orientations.

Listing 5.27 *Function used to identify the phase cut off points*

```
def find_phase_cut_points(x_phase_sig, fs, fr, band_rel=0.25,
min_bw_hz=1.0):
    #Bandpass around fr to isolate the main 1x component
    bw = max(min_bw_hz, band_rel * fr)
    f_lo = fr - bw
    f_hi = fr + bw
    x_bp = bandpass(x_phase_sig, fs, f_lo, f_hi)
    #Hilbert analytic signal to get instatanoues phase
    hil_singal = hilbert(x_bp)
    phase = np.unwrap(np.angle(hil_singal))
    #phase crossings at multiples of 2π (same phase)
    k = np.floor(phase / (2*np.pi))
    #possible indices where k increments (phase crosses next 2π)
    cross = np.where(np.diff(k) > 0)[0] + 1
    #Require positive slope of bandpassed signal at cut
    dx = np.gradient(x_bp)
    cross = cross[dx[cross] > 0]
    return cross, x_bp, phase
```

Segment Construction

Using the detected phase-aligned cut points, segments were constructed. For each cut index s a corresponding end cut near $s + L$ was selected, where L was equal to 6 seconds. Only segments exceeding a minimum duration were retained to ensure stability.

Crossfade Concatenation

Although phase aligned cuts significantly reduce discontinuities, small amplitude differences may still exist between segments due to noise, speed variation or load fluctuation. Direct concatenation of the two segments would produce a step discontinuity. To mitigate this, overlap crossfading was implemented using a linear ramp. The linear ramp reduces high frequency spikes by replacing the steps between segments with a gradual ramp blending.

Listing 5.28 *Function used to crossfade between added signal segments.*

```
def crossfade_concat(a, b, xfade):
#Concatenate 2 arrays a and b with an overlap-add crossfade of length
xfade.a: (L,3), b: (M,3)
    xfade = min(xfade, len(a), len(b))
    w = np.linspace(0, 1, xfade)[:, None]  # (xfade,1)
    a_tail = a[-xfade:] * (1 - w)
    b_head = b[:xfade] * w
    mid = a_tail + b_head
    return np.concatenate([a[:-xfade], mid, b[xfade:]], axis=0)
```

Signal Assembly

Segments were randomly sampled and concatenated until the target duration was reached. The final synthetic signal was extended to the desired duration. Importantly, the extended dataset retained the original sampling frequency of 200 Hz, its tri-axial structure, physically possible harmonic structure and continuous time indexing.

Listing 5.29 *Signal Assembly.*

```
    while (out is None) or (len(out) < target_len):
        s, e = segments[rng.integers(0, len(segments))]
        seg = X[s:e]
        if out is None:
            out = seg
        else:
            out = crossfade_concat(out, seg, xfade)
    return out[:target_len]
```

5.12 Experimental Results & Discussion

5.12.1 1D-CNN Model - Lengthened Data

The dataset was successfully extended from 147,277 samples to 1,200,000 samples, effectively increasing the data amount ten fold. The dominant rotational frequency was estimated at 51.63 Hz, consistent with the expected operating regime of the motor. A total of 107 phase aligned cut off points were identified within the original dataset, an example of the first cut off point can be seen in the plot below, at index 2682.

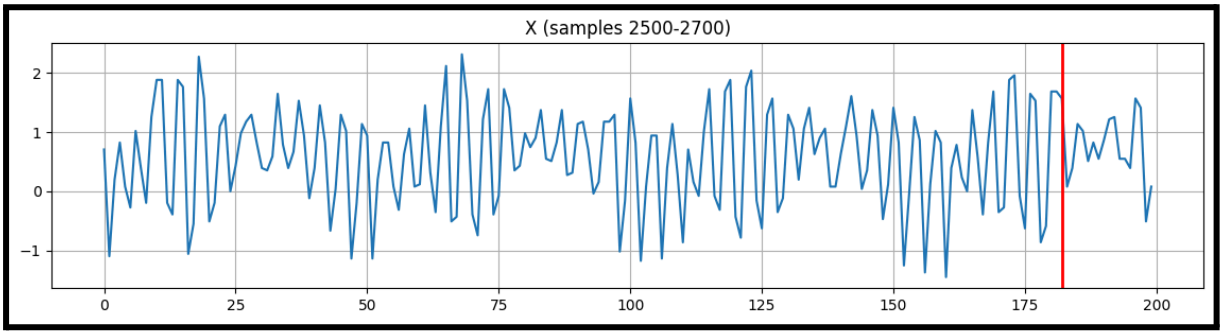


Figure 5.23. Phase aligned segmentation point at index 2682 identified using the estimated frequency.

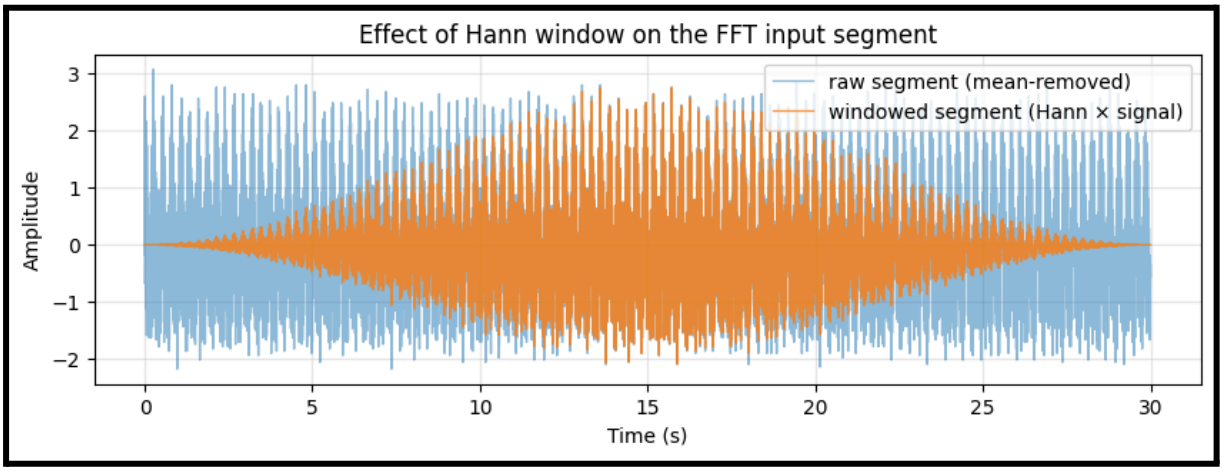


Figure 5.24. Effect of the Hann window on a FFT input segment.

Visual Inspection

Figure 5.25 compares the first second of the original x-axis signal against the extended x-axis signal. The phase alignment and amplitude appear to be consistent, indicating that the stitching process did not introduce any obvious discontinuities or artificial high-frequency spikes.

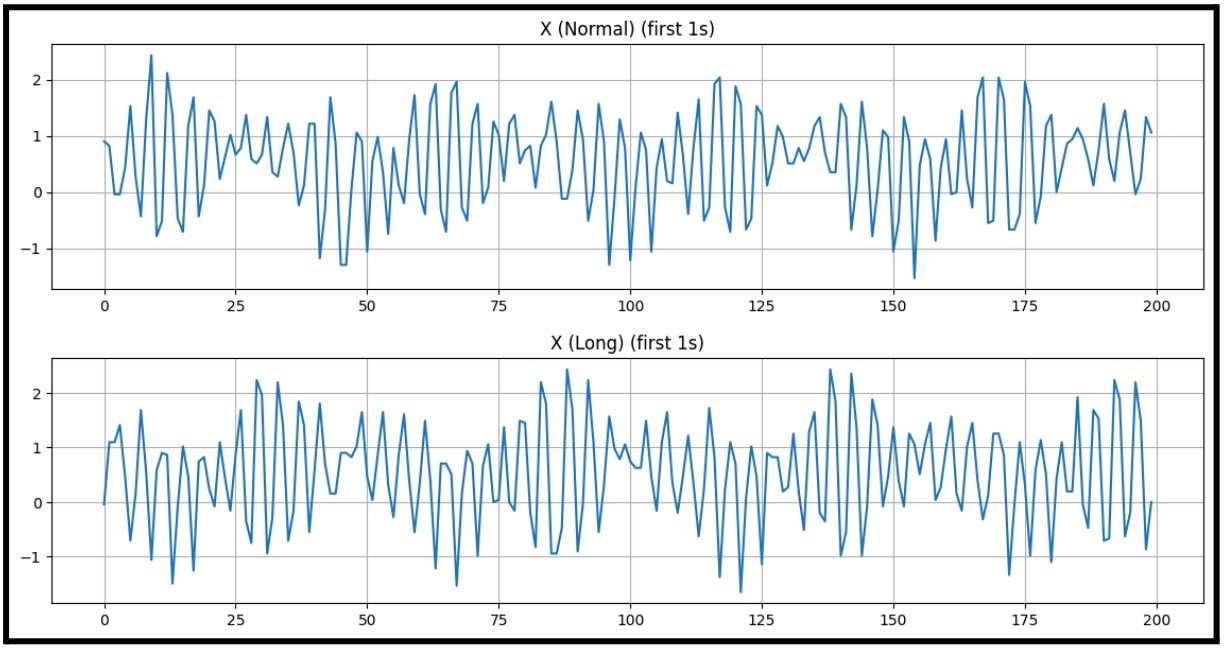


Figure 5.25. One second comparison of original (top) and phase-lengthed (bottom) x-axis vibration signals demonstrating continuity and absence of stitching artefacts.

Further comparison over the first 10 seconds shows that the extended signal retains the harmonic structure and amplitude characteristics of the original signal. The stitched dataset appears to be visually similar to the source signal, with slight rearrangement of phase aligned segments, as intended.

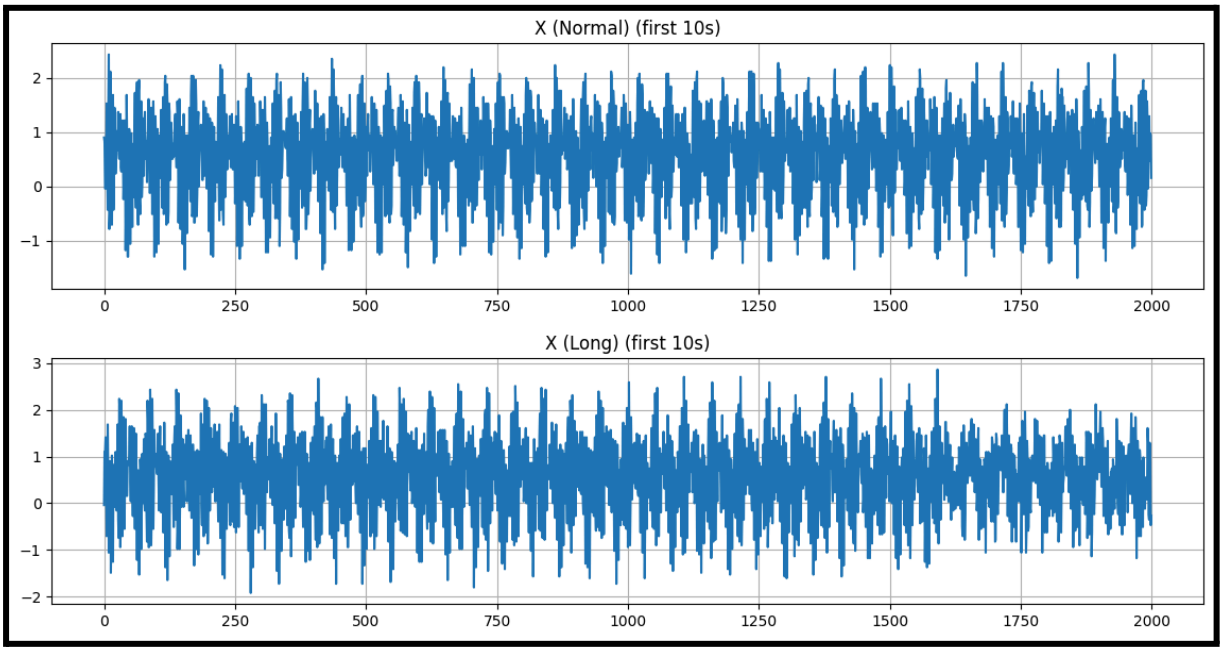


Figure 5.26. Ten second comparison of original (top) and lengthed (bottom) x-axis vibration signals showing preserved harmonic structure and amplitude.

Finally, the two full length signals 147k samples vs 1.2M samples were plotted. The extended dataset preserves the global vibration structure while exhibiting increased variability in segment ordering. In theory, this introduced variability should improve the robustness of the model since the CNN is exposed to minor amplitude variations and phase shifts that are not indicative of anomalies.

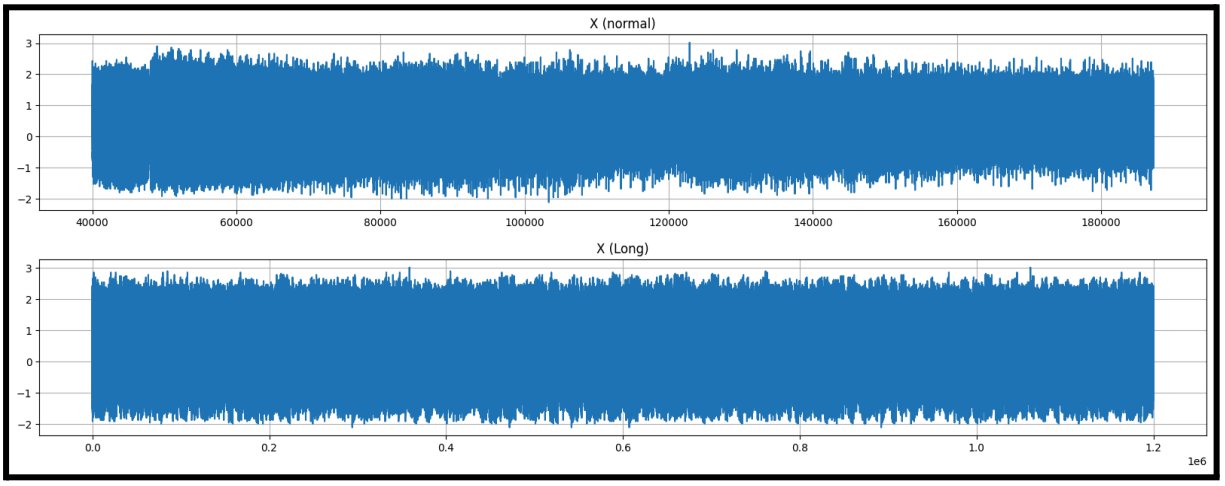


Figure 5.27. Full length comparison between original (top) and lengthed (bottom) vibration datasets showing preserved global structure.

Additionally, another plot was generated to show the effects of the crossfade function at the boundary between two extracted segments. Visually, this transition appears smooth and free from any discontinuities or abnormal amplitude spikes.

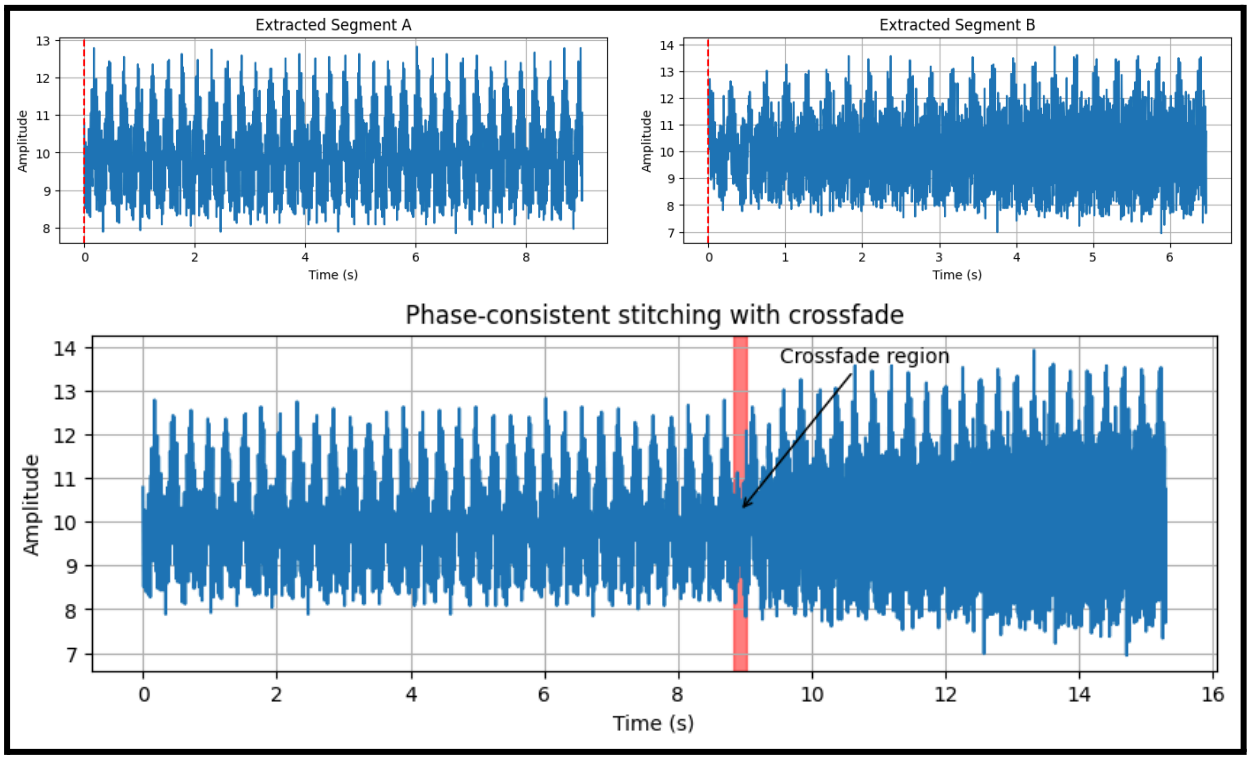


Figure 5.28. Crossfade region between two stitched segments.

Model Behaviour

Despite the visually promising characteristics of the stitched signal, results from embedded deployment revealed significant degradation in model performance. When previously validated CNN models were trained on this extended dataset, the sigmoid output probability was consistently valued around 0.9. The classifier predicted anomalies constantly, even when the motor was running normally. Model stability and performance collapsed completely. This behaviour indicated that the stitched dataset introduced characteristics that were not representative of the motors dynamics whatsoever.

Possible modes of failure here include the fact that the model could have become overconfident. The repeated exposure to the recombined segments may have caused the CNN to learn an overlay narrow representation of “normal operation”, resulting in high confidence predictions and very poor calibration. Furthermore, in real rotation machinery, long duration correlation evolves naturally, so synthetically recombining segments disrupts these temporal dependencies. In conclusion, the result is a dataset that is visually plausible but statistically inconsistent with true motor behaviour.

Table 5.8. *Serial monitor output.*

prob=0.945312	anomaly=1
prob=0.894531	anomaly=1
prob=0.804687	anomaly=1
prob=0.945312	anomaly=1
prob=0.894531	anomaly=1
prob=0.804687	anomaly=1
prob=0.945312	anomaly=1
prob=0.894531	anomaly=1
prob=0.804687	anomaly=1

Version 2C - Augmented Anomalies

The experimental results from Version 2B indicated that simple dataset lengthening did not consistently improve anomaly detection robustness. While increasing the volume of training data can be beneficial in machine learning applications, the repetition of nominal vibration signals introduces instability. This finding highlighted that meaningful improvements in model performance require not only more data, but also more informative variability in the training data.

Version 2C therefore continues to address RQ2 - Real Motor Operation Anomaly Detection, by investigating whether synthetic anomaly generation can be more effective for improving classification performance. Therefore, rather than manipulating and multiplying normal data, the focus was shifted towards generating controlled anomalies.

The objective of Version 2C is not to generate a complete library of industrial faults, but instead to prove that data augmentation works and to enrich the training distribution without requiring additional physical experiments that could damage the motor. This investigation also contributes to RQ3 - Robust Embedded Predictive Maintenance System, where the final implementation will be robust to multiple speeds and anomaly types.

5.13 Specific Theory & System Design

5.13.1 Data Augmentation - Bearing Like Anomalies

The augmentation methodology was designed to remain consistent with established bearing fault representations from research but to also work from physical reasoning. For example, it is well documented that a localised impact causes structural resonance within the machine casing. Consequently, the accelerometer does not observe a single-sample spike in the amplitude, but rather a short excitation followed by a damped oscillatory response.

A critical design constraint is that the augmentation must not leak into the evaluation of the dataset. Synthetic faults are injected into the training and validation sets only, while the test set must remain untouched. This ensures that reported performance reflects generalisation to unseen data rather than the model being able to recognise injected anomalies. For this reason, most of the testing will be done on the physical rig, as not having anomalies in the test set will mean that the classification matrix and ROC curve are redundant.

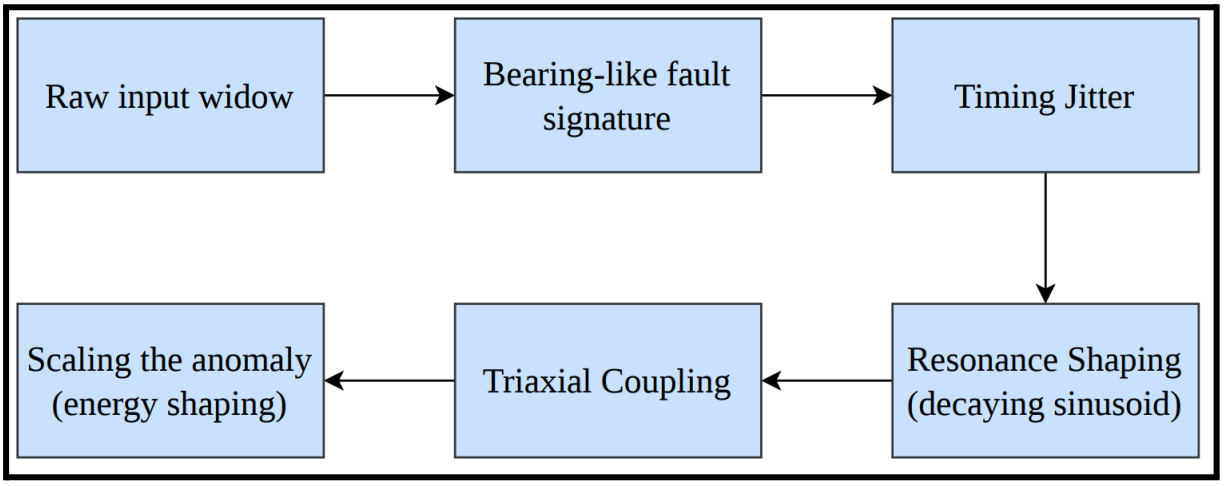


Figure 5.29. Bearing-like anomaly injection workflow.

Bearing Fault Signature

Some bearing defects can be modelled as repetitive impacts, for example when a rolling element passes over a defect, the contact generates a short impulsive excitation [50]. In the time domain, this can be represented as the following [56], [57]:

$$s_{imp}(t) = \sum_{n} a_n \delta(t - t_n)$$

where:

- t_n are the impact times,
- a_n are the impact magnitudes,
- $\delta(.)$ is the Dirac delta impulse.

In a simplified rotational setting as such, the impacts occur at a repeated frequency, which is proportional to the rotational frequency f_r. In the augmentation pipeline, the fault repetition rate is defined as $f_f = kf_r$, where k is picked out from a small range to represent variability in the impact repetition rates. The theory behind adding variability (k) is to avoid generating a single narrow fault signature which would be similar to the physical anomalies generated in Version 2A.

Timing Jitter

Real impacts and bearing faults are not perfectly periodic due to speed ripple, load variation, slip and measurement noise [57]. To prevent the augmented faults from being too perfect and unrealistically timed, a small timing jitter is introduced into the impact indices.

$$t' = t_n + \epsilon_n$$

$$\epsilon_n \in [-\alpha T_f, \alpha T_f]$$

where:

- α is a small fraction,
- T_f is $\frac{1}{f_f}$.

Additionally, bearing impulses often exhibit stronger impacts when the defect is in the loaded region of the bearing. This effect is approximated by modulating the amplitude at the 1x frequency [58]. This method produces impacts in the vibration data that are still repetitive but vary a slight bit more in strength. This was implemented in order to strengthen the models generalisation to different types of bearing like faults.

$$a_n = (1 + \eta_n)(c_0 + c_1 \sin(2\pi f_r t_n + \phi))$$

where:

- η_n is random amplitude scatter (Gaussian),
- c_0,c_1 control the modulation depth,
- ϕ is the random phase offset.

Resonance Shaping

In practice, an impact does not appear as a single-sample spike at the accelerometer. Instead it excites resonant modes in the motor structure, mount, and sensor casing. This is modelled into the data by filtering the impulse with a decaying sinusoid impulse response. This step is important because it generates a fault that resembles real vibration impacts, short excitation followed by damped oscillation, rather than just isolated amplitude spikes [57].

$$h(t) = e^{\frac{-t}{\tau}} \sin(2\pi f_0 t)$$

where:

- f_0 is a resonance frequency,
- τ is a decay time constant.

Tri-Axial Coupling

A single mechanical impact typically couples into multiple axes. Therefore, the generated fault waveform is injected into all three axes. This helps avoid unrealistic augmentation where only one axis contains fault energy, further improving the robustness of the model to sensor orientation changes.

Energy Control Using Target SNR

Finally, in order to prevent the injected faults from being either trivially obvious or unrealistically weak, the injection amplitude is scaled using a target signal-to-noise ratio (SNR), relative to the window RMS of the original signal. Scaling the injected fault signal to match the RMS target produces controlled variability in injection strength and helps the CNN learn a spectrum of "bearing-like" severity amplitudes rather than a single amplitude.

5.14 Implementation & Methodology

5.14.1 Data Augmentation - Bearing Like Anomalies

Windowing and Splitting

Raw triaxial accelerometer samples are first segmented into fixed-length windows. The exact same configuration used for the motor CNN is retained to ensure consistency with deployment.

- Window length: N = 256 samples,
- Stride: 256 samples (non overlapping),
- Sampling frequency: 200 Hz.

Each window is assigned a binary label based on the fraction of anomalous or normal samples in the window. A window is marked as anomalous if at least 10% of its samples are anomalous.

To preserve temporal structure and prevent information leakage across splits, the windowed data is split chronologically. Since the splitting is time-based, stratification is not applied.

Bearing-Like Injection Model

The injection implementation consists of five major steps:

1. Impulse generation at a repetition frequency $f_f = kf_r$, where $k \in [3, 10]$ is randomly sampled to introduce variability.
2. Small timing jitter (+/- 3% of the period) to avoid perfectly periodic, unrealistic impulses.
3. Amplitude modulation at 1x to mimic load zone variation.
4. Resonance shaping by convolving the impulses with a decaying sinusoid.
5. Tri axial mixing and target SNR scalding to control the strength of the injections relative to the baseline window RMS.

Two dedicated functions were implemented to do this augmentation, both of which are provided in **Appendix C**. This first function approximates the structural response of the casing/mount to a bearing impact. It changes the augmented data from being a single sample spike to a short decaying oscillation. The second function contains the rest of the injection code.

Augmentation and Injection Rates

Augmentation is performed on the dataset by selecting a subset of a few healthy windows and converting them into synthetic anomaly windows. Separate injection probabilities are used:

- Training injection rate: 0.20,
- Validation injection rate: 0.05,
- Test injection rate: 0.00.

The reduced validation injection rate is used to prevent the model from being overly optimistic. Some anomalies exist in the validation set to expose the model to a limited amount of variety.

Listing 5.30 Augmented Window Generation.

```
def augment_train_windows(X_train, y_train,
fs,p_inject=0.2,mix={"bearing":1},seed=123):
    rng = np.random.default_rng(seed)
    X0 = X_train[y_train == 0]
    n0 = len(X0)
    n_add = int(p_inject * n0)
    if n_add <= 0:
        return X_train, y_train
    idx = rng.choice(n0, size=n_add, replace=False)
    X_new = []
    y_new = np.ones(n_add)

    for wi in idx:
        w = X0[wi]
        fr = estimate_fr_hz(w[:,2], fs, fmin=1.0, fmax=100.0)
        w2 = inject_bearing_like(w, fs, fr_hz=fr)
        X_new.append(w2)

    X_new = np.stack(X_new, axis=0)
    X_aug = np.concatenate([X_train, X_new], axis=0)
    y_aug = np.concatenate([y_train, y_new], axis=0)
    perm = rng.permutation(len(X_aug)) #shuffle data in the training
window so its not all side by side
    return X_aug[perm], y_aug[perm]
```

Dataset Export

The augmented datasets are saved as compressed .npz files. The CSV version of larger datasets were cumbersome and took Colab a while to load into the environment. Converting the files into .npz allows for faster loading of the data.

The files were created to contain:

- X_{train}, y_{train},
- X_{val}, y_{val},
- X_{test}, y_{test} (untouched),
- Metadata: f_s, window length, stride, injection probabilities.

Visualisation

As a verification step, augmented windows were plotted with colour coded labels to confirm that the injected anomalies manifest in the signal as impulsive disturbances. This step is used to confirm that the injection process did not produce any unrealistic amplitudes or discontinuities.

5.14.2 Motor 1D-CNN - Augmented Data

Dataset Loading

The CNN architecture used for the augmented motor experiment was intentionally kept identical to the previously validated structure from Version 2A. This ensured that any performance differences could be attributed to the augmented data rather than changes in network capacity.

The primary modification made in this experiment was the transition from CSV dataset to the generated augmented .npz files.

This structure guarantees that there is strict separation between training, validation and test splits, and that there is no augmentation leakage into the test set.

Listing 5.31 Loading NPZ files.

```
def load_npz_splits(path):
    d = np.load(path)
    return (
        d["X_train"], d["y_train"],
        d["X_val"],   d["y_val"],
        d["X_test"],  d["y_test"],
        float(d["fs"][0]) if "fs" in d else None
    )
```

Train Only Shuffling

The training set was shuffled to break any short term temporal correlations, prevent batch level regime clustering and improve gradient stability. Validation and test sets remained in chronological order to preserve realistic evaluation conditions.

Listing 5.32 Train data shuffling.

```
#Shuffle TRAIN
rng = np.random.default_rng(123)
perm = rng.permutation(len(X_train_n))
X_train_n = X_train_n[perm]
y_train = y_train[perm]
```

Deployment

No other changes were made to the Python code or the ESP32 firmware. The files containing the quantised model itself and the normalisation parameters were updated accordingly.

5.15 Experimental Results & Discussion

5.15.1 Data Augmentation - Bearing Like Anomalies

Visual Inspection

Figure 5.30 shows the first 250 training, validation and testing windows from the augmented dataset in the z axis (channel 2). Datapoints corresponding to healthy windows are shown in blue, while synthetically injected anomalies are shown in orange. The injected anomalies are distributed throughout the entire training set rather than clustered in a single region. The injected faults exhibit clear impulsive behaviour with variable amplitude and decay characteristics, consistent with the theory described.

Importantly, the variability in the anomalous windows suggests that the augmentation process does not generate a single repeated template, but instead a distribution of impulsive signatures. From a qualitative standpoint, the augmentation appears to satisfy its design objectives. There is increased anomaly diversity without introducing discontinuities or unrealistic waveform behaviour.

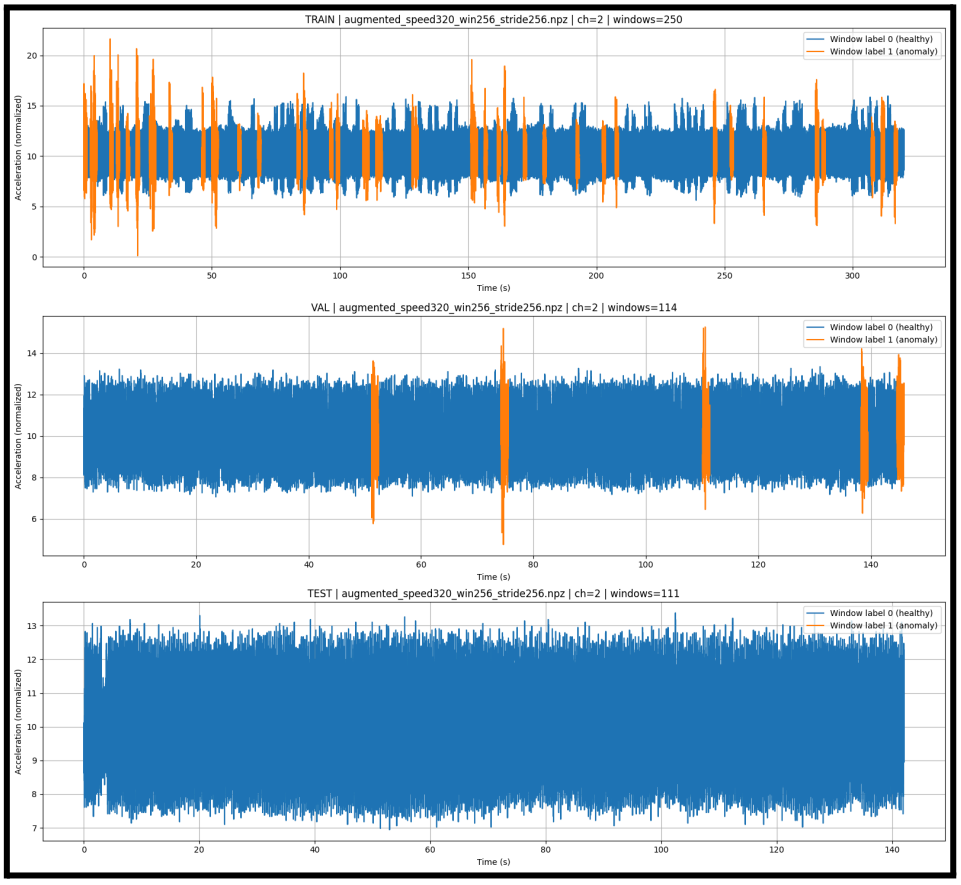


Figure 5.30. First 250 windows from the augmented dataset (z-axis), showing healthy samples (blue) and synthetically injected bearing-like anomalies (orange) across training, validation and testing sets.

5.15.2 Embedded Deployment: 1D-CNN - Augmented Anomalies

The 1D-CNN model, trained on the augmented datasets, was deployed to the ESP32-S3 and evaluated under real motor operation. Validation was performed using both the LED indication system and direct serial probability output.

Real Time Serial Output

After successful initialisation, the firmware printed out the diagnostic information to confirm correct preprocessing behaviour. The values presented in the table below confirm that global normalisation is correctly implemented, the mean is approximately centered around zero and that the standard deviation is non zero and stable.

Table 5.9. *Embedded Normalisation diagnostics.*

-----Setup complete-----			
DEBUG first 5 normalized samples:			
0:	-0.089091	-0.131153	-0.016194
1:	-0.163450	-0.048657	-0.016194
2:	-0.126271	-0.103655	0.008466
3:	-0.089091	-0.076156	0.008466
4:	-0.089091	-0.048657	-0.016194
norm mean:	0.110	-0.083	-0.006
norm std:	0.058	0.045	0.048

During steady motor operation with no induced anomaly, the output probabilities were consistently low, notably the lowest they have been up to this point in the research. Most probabilities were found to be under 0.2 and many were clustered around 0.07-0.09. Some quantised values dropped as low as 0.0000 and 0.0039.

Despite the occasional mid-range spikes of 0.3828, 0.5585 or 0.7656, the anomaly decision remained 0 due to the conservative nature of the selected threshold (0.8). Nevertheless, when the motor casing was shaken violently or when other anomalies were introduced, the probability increased sharply, up to values of 0.8945 and 0.9765. This proved there is a clear class separation between the baseline operating region of 0.2 and anomalies which exceed 0.89.

Quantisation Behaviour

The quantised probabilities in the table all appear in discrete increments. Each increment corresponds to:

$$\frac{1}{256} = 0.00390625$$

Examples from the table that showcase this include: 0.0039, 0.0078 and 0.0117.

This confirms that the int8 quantisation is functioning correctly. The output resolution matches the expected quantisation granularity and no numerical saturation or clipping is occurring. Critical to the evaluation of the model, quantisation does not affect its performance. Since the anomaly probabilities remained near 1 and probabilities during normal operation remained near 0, the full range of floating points was not needed. This indicates that full integer deployment did not degrade classification behaviour in practice.

Table 5.10. *Serial monitor output.*

prob=0.054687	anomaly=0
prob=0.011719	anomaly=0
prob=0.003906	anomaly=0
prob=0.234375	anomaly=0
prob=0.894531	anomaly=1
prob=0.332031	anomaly=0
prob=0.027344	anomaly=0
prob=0.054687	anomaly=0
prob=0.003906	anomaly=0
prob=0.281250	anomaly=0
prob=0.054687	anomaly=0
prob=0.019531	anomaly=0
prob=0.976562	anomaly=1
prob=0.160156	anomaly=0
prob=0.234375	anomaly=0
prob=0.000000	anomaly=0
prob=0.914062	anomaly=1
prob=0.007813	anomaly=0

Version 3A - Multi-Speed Augmented CNN

The final implementation stage uses all the key findings obtained throughout prior Versions. While Version 1 established the feasibility of performing anomaly detection on an embedded device, Version 2 focussed on evaluating model performance under increasingly more realistic conditions.

The results of Version 2A demonstrated that a CNN could successfully detect physically generated anomalies on a live motor in real time. Importantly, the deployed model architecture remained within the computational and memory constraints of the embedded platform.

Version 2B investigated the use of dataset lengthening techniques as a strategy to aid with the limited training data. However, experimental evaluation showed that duplicating and extending normal vibration signals did not consistently improve classification accuracy. While this approach did increase the training dataset size, it introduced instability, limiting its effectiveness in the final system design.

In contrast, Version 2C demonstrated that physically inspired augmentation of anomalous vibration patterns provided measurable improvements. By enriching the training dataset with controlled anomalies, the model was exposed to a broader range of fault-like signals, improving classification accuracy.

Version 3A therefore represents the combination of these findings and directly addresses the final research question RQ3 - Robust Embedded Predictive Maintenance System. The final system integrates the validated CNN architecture from Version 2A with the data augmentation method from Version 2C. Furthermore, to improve model robustness and generalisation, the model is trained using multiple datasets collected from the motor running at different speeds.

By combining validated modelling approaches, realistic vibration signals and verified embedded deployment into a single architecture, this stage moves the project from experimental feasibility towards practical PdM system realisation. The objective is to evaluate whether the proposed system can operate reliably as a standalone embedded monitoring solution, capable of detecting anomalous motor behaviour under varying operating conditions. Additionally, MQTT communication is integrated to assess whether the system can also support remote monitoring through network communication.

5.16 Specific Theory & System Design

Previous experimental stages focused on training the model for anomaly detection under a single operating regime. While these experiments were useful for validating the models behaviour and embedded deployment, real rotating machinery rarely operates at one constant speed. Models trained on a single operating condition risk overfitting to that specific regime and may fail when deployed in more realistic conditions.

To solve this problem and improve generalisation, the final iteration of this system introduces multi-speed data collection. Vibration patterns need to be recorded at several discrete motor speeds, producing datasets with different dominant rotational frequencies and harmonic structures. By combining these subsets into a single training dataset, the CNN is exposed to a much wider range of operating behaviours. This approach encourages the model to learn features that are characteristic of anomalous behaviour, rather than deviations from rotational speed.

In addition to varying the motor speed, the validated anomaly augmentation strategy proposed in Version 2C will be applied to all operating speeds. This is done to ensure that the CNN observes anomalous signatures across the different vibration baselines, rather than associating anomalies with one operating condition.

Fault Injection

To evaluate the robustness of the final system, additional realistic fault injection mechanisms had to be tested on the experimental rig. Rather than injecting anomalies through external actuators like the servo, the motor will be operated at a speed corresponding to its mechanical resonance frequency. Motors operating near resonance are known to significantly increase their vibration amplitude due to the interaction between the excitation frequency and the natural frequency of the mechanical structure. In industrial applications, this condition is avoided since resonance excitation can accelerate mechanical wear, induce excessive vibrations and increase the likelihood of component damage. By deliberately adjusting the motor speed, the system will experience unwanted, amplified vibration patterns that resemble anomalous behaviour.

Additionally, a more severe fault condition will also be introduced into the experimental rig. By disconnecting one coil of the stepper motor from the driver, the motor loses proper phase synchronisation. This in turn causes strong mechanical vibrations and unstable motion. Such behaviour resembles real electrical or wiring faults that could possibly occur in industrial environments. Electrically, this experiment can produce back-EMF and irregular current regulation in the driver. These irregularities can overheat or damage the driver and the motor itself. Consequently, this experiment will have to be conducted cautiously in order to minimize this risk. The testing setup will involve temporarily isolating the system from any power source before modifying the wires, and operating the motor for short periods of time. This limits any thermal and electrical stress on the driver and motor.

Together, resonance excitation and phase interruption provide two real fault scenarios, one mechanical and one electrical. Evaluating the anomaly detection system under these conditions will help determine whether the embedded CNN model can reliably detect abnormal activity in industrial environments.

IoT Implementation

Finally, the system's anomaly indication architecture will be extended beyond the NeoPixel LED. Earlier experiments relied on the LED to provide immediate indication of detected anomalies. While this setup was effective for debugging and demonstrating purposes, industrial monitoring systems usually require remote monitoring. This final version of the PdM system will integrate MQTT communication to transmit any anomaly events from the ESP32-S3 to a remote server. MQTT is widely used in industrial IoT applications due to its lightweight architecture and low bandwidth requirements [59].

Through this implementation, anomaly detections produced by the CNN model on the embedded device can be transmitted in real time to monitoring dashboards or alarm systems. This will transform the experimental prototype into an IoT integrated, predictive maintenance device capable of integration in larger industrial systems.

At present, the information transmitted by the device is intentionally limited. The system only sends a single anomaly probability value and a binary anomaly indicator (0 or 1), additionally it's only for one test motor rig. As a result, the transmitted data does not contain detailed machine signals, how many machines exist, any parameters or sensitive operational information. This design reduces potential cybersecurity risks associated with transmitting raw data or proprietary operational metrics. Should future iterations of the system incorporate additional information, further cybersecurity measures such as encryption or authentication would be required.

5.17 Implementation & Methodology

5.17.1 Motor 1D-CNN - Multiple Speeds

To improve the models generalisation across different operating conditions, the 1D-CNN model was retrained using accelerometer readings at multiple motor speeds.

The NEMA17 stepper motor was driven using a TB6600 driver configured for 16x microstepping:

$$N_{steps} = 200 \times 16 = 3200 \text{ pulses/rev}$$

Motor speed was controlled in the Arduino UNO firmware by adjusting the pulse delay between step signals sent to the TB6600 driver.

Speed Control

The pulse period is defined as:

$$T = 2 \cdot \text{delay}_{\mu s}$$

where:

- delay is the microsecond delay between HIGH and LOW transitions,
- The factor of 2 accounts for the full pulse cycle.

Step frequency can therefore be expressed as:

$$f_{step} = \frac{1}{2 \cdot \text{delay}_{\mu s}}$$

RPM is calculated as:

$$RPM = \frac{f_{step}}{3200} \times 60$$

Table 5.11. *Stepper motor operating speeds derived from step time parameters.*

Step Delay (µs)	Step Frequency (Hz)	RPM
180 µs	2777.8 Hz	52.1 RPM
240 µs	2083.3 Hz	39.1 RPM
320 µs	1562.5 Hz	29.3 RPM

The selected speeds between 29 and 52 RPM represent moderate rotational speeds for the experimental rig. These speeds provide distinct harmonic spacing, different vibration energy distribution and variation in dynamic loading conditions.

Importantly, these speed values do not vary drastically. This allows the model to learn speed-invariant fault characteristics rather than extreme operating differences. It also leaves the opportunity for testing how well the model will generalise to speeds as slow as 11.7 RPM (step delay of 800 µs) or speeds as fast as 156.25 RPM (step delay of 60 µs).

For each operating speed, the firmware step delay was set via a serial command. The vibration data was logged using the Python logger and the ESP32-S3. Normal operating conditions were recorded for about 15 minutes for each speed. Data from all three speeds was then exported and processed independently.

Dataset Augmentation

Each speed specific dataset was passed independently through the augmentation pipeline from Version 2C. Each dataset was split into windows and had the synthetic bearing-like anomalies injected. The resulting augmented datasets were then exported into the 1D-CNN training pipeline. In the training code, the three separate datasets were concatenated together with the respective splits. The training dataset was used to globally normalise the combined dataset and it was randomly shuffled to prevent the model from learning operating regimes. Validation and training were left in chronological order to mimic real industrial motor vibrations.

Model Deployment and Testing

The trained model was quantised and deployed using the same embedded deployment procedure as before. The quantised TensorFlow Lite model, along with the normalisation parameters were downloaded to the embedded system.

Testing of the model was carried out in two stages;

1. In-Distribution testing, where the motor was operated at the three speeds used during training to confirm baseline performance consistency.
2. Out-of-Distribution testing, where the motor was operated outside the training speed range to evaluate generalisation beyond the learned operating spectrum.

The two anomaly generation techniques described above were used during deployment to assess the models performance.

- The resonance excitation frequency was found using trial and error. By visually inspecting the rig to see the motor and casing vibrating abnormally along with the increase in noise, the speed was found to be: 93.75 RPM (100 µs).
- A pushbutton was implemented to temporarily disconnect the coil from the driver to the motor.

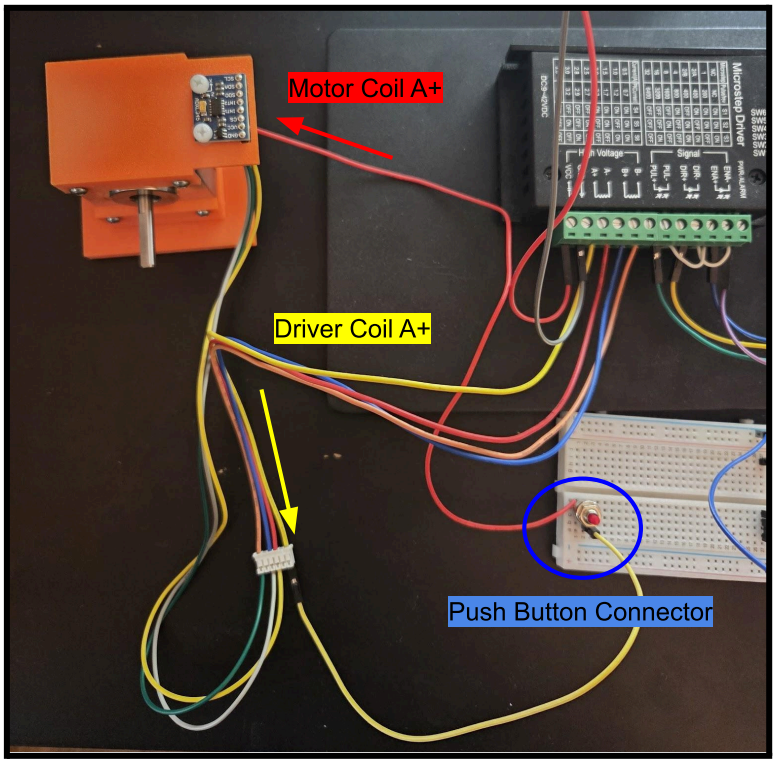


Figure 5.31 Push button operated coil disconnector.

5.17.2 IoT Implementation

Mosquitto Broker Config

A local Mosquitto broker was used to receive messages from the embedded device. The broker was configured through a simple file to listen for incoming MQTT connections on port 1883 and to accept connections from local network listeners.

Listing 5.33 *Initial Mosquitto Configuration file (mosquitto.conf)*

```
listener 1883 0.0.0.0

allow_anonymous true

log_type all
```

Using this configuration, “listener 1883 0.0.0.0” instructs Mosquitto to listen on the default MQTT port 1883. This allows the broker to accept connections from other devices on the same local network, including the ESP32-S3. The “allow_anonymous true” setting removes the need for username and password authentication, making initial testing easier. Finally “log_type all” enabled broker logging which is useful for debugging connections and publishing events.

Network and MQTT Setup

On the embedded device, Wi-Fi and MQTT support were implemented using the WiFi.h and PubSubClient.h libraries. The ESP32-S3 first connects to the local network, then it attempts to establish a connection with the Mosquitto broker running on the host device. The host device IPV4 address was identified using “ipconfig”.

Listing 5.34 *MQTT Server Configuration*

```
//MQTT broker
const char* MQTT_host = "192.168.1.3"; //pc ipv4 address
const uint16_t MQTT_port = 1883;
const char* MQTT_user = nullptr;
const char* MQTT_pass = nullptr;
//MQTT topics
const char* sent_status = "rig/motor1/status";
const char* sent_alarm = "rig/motor1/alarm";
```

The broker address is defined as the IPv4 address of the device running Mosquitto. Two topics were used in this case. The first, “rig/motor1/status”, transmits structured information containing the anomaly probability and the binary decision. The second “rig/motor1/alarm”, transmits a simple 1 or 0 alarm state. This topic separation was chosen for dashboard integration and a simple alarm style notification.

Non-blocking MQTT Reconnection

The MQTT client is configured to use the broker and port defined earlier. Because the vibration acquisition and inference loop must critically remain deterministic, it was important to avoid long network operations during runtime. For this reason, MQTT reconnection was implemented using a rate limited function. The function attempts reconnection only once every two seconds, limiting repeated calls inside the main loop.

Listing 5.35 *MQTT Server Reconnection logic.*

```
bool mqtt_connect_nonblocking() {
  //Avoid blocking inside fast loop
  //Only attempt every ~2s
  const unsigned long now = millis();
  if (now - last_mqtt_attempt_ms < 2000) return false;
  last_mqtt_attempt_ms = now;

  if (mqtt.connected()) return true;

  Serial.print("MQTT connecting...");
  //Client ID unique
  String clientId = "ESP32S3-motor1-" +
String((uint32_t)ESP.getEfuseMac(), HEX);

  bool ok;
  if (MQTT_user && MQTT_pass) {
    ok = mqtt.connect(clientId.c_str(), MQTT_user, MQTT_pass);
  } else {
    ok = mqtt.connect(clientId.c_str());
  }

  Serial.println(ok ? "OK" : "FAIL");
  return ok;
}
```

Latched Alarm

To put the system into a more practical sense, a latched alarm mechanism was implemented. In previous versions, the alarm state was directly coupled to the probability output of the model. While it was fine for initial testing, such behaviour can be difficult to monitor in practice, since a short anomaly can be missed before an operator has time to see it.

In this final implementation, once the anomaly probability exceeds the selected decision threshold, the alarm state is latched and remains active even if the following inference windows return back to normal. This latched state drives the MQTT alarm topic and the NeoPixel LED. As a result, anomalous events are preserved until they are explicitly acknowledged by an operator.

To clear the latched state, a physical reset button was added to the experimental rig. Reset is only allowed if the current model output indicates that the anomaly is no longer present. If the operator attempts to reset the alarm while the anomaly is active, the reset is blocked and the alarm remains latched. This setup is more representative of industrial alarm handling, where alarms must be physically acknowledged rather than just disappearing automatically.

Listing 5.36. *Latched alarm reset logic*

```
const bool anomaly_now = (prob >= ANOMALY_THRESHOLD);

if(anomaly_now){
  anomaly_latched= true;
}

if(pressedDebounced(RESET_PIN)){
  if(!anomaly_now){
    anomaly_latched= false;
    Serial.println("alarm latch cleared");
  }
  else{
    Serial.println("Reset blocked anomaly present");
  }
}
```

MQTT Publishing

Once the CNN model computes a probability and applies the threshold, the result is published to the MQTT broker. The publish function sends a JSON status message with the probability value, the instantaneous anomaly state, and the latched alarm state. A separate binary alarm topic is also transmitted.

Full operators dashboards can connect to the JSON status topic to see the probability values printed out over time, while lightweight alarm devices can subscribe only to the alarm topic to receive binary information regarding the alarm state.

Listing 5.37 *MQTT publish function.*

```
void mqtt_publish_status(float prob, bool anomaly_now, bool
alarm_latched) {
  if (!mqtt.connected()) return;

  char payload[160];
  snprintf(payload, sizeof(payload),

"{\"prob\":%.6f,\"anomaly_now\":%d,\"alarm_latched\":%d,\"ms\":%lu}",
            prob,
            anomaly_now ? 1 : 0,
            alarm_latched ? 1 : 0,
            (unsigned long)millis());

  mqtt.publish(sent_status, payload, true);
  mqtt.publish(sent_alarm, alarm_latched ? "1" : "0", true);
}
```

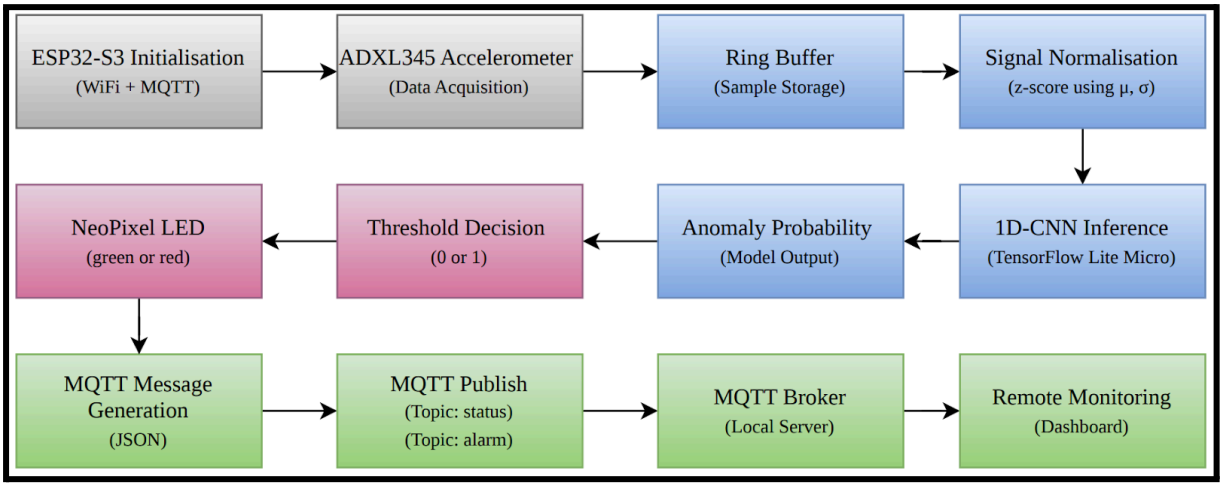


Figure 5.32. Full system integration inference pipeline workflow.

MQTT Password Protection

After confirming full functionality, the mosquitto.config file was adjusted to include user and password authentication. The file was configured to “allow_anonymous false” and the configuration was pointed to a password.txt file.

Listing 5.38 *Password Protected Mosquitto Configuration file (mosquitto.conf)*

```
listener 1883 0.0.0.0

allow_anonymous false
password_file password.txt

log_type all
```

The mosquitto authentication mechanism stores the users credentials in a password file that is generated using “mosquitto_passwd”. Importantly, the file generated using this function does not store passwords in plain text. Instead, the password is protected using a cryptographic hashing function and stored as a hashed value within the password.txt file [60]. As a result, even if the file was accessed by an unauthorized user, the original password cannot be directly read or easily reconstructed.

Listing 5.39 *Password file generation*

```
mosquitto_passwd -c passwords.txt motoruser
```

Chapter 6: Results & Discussion

This chapter presents the results obtained from the final implementation of the embedded PdM system. Unlike earlier stages of the project, where results were presented alongside the development, this chapter focuses exclusively on the performance evaluation of the completed system architecture (Version 3A).

This separation of results from previous experimental versions reflects the transition from feasibility driven investigations towards the final system validation. Earlier versions explored individual research questions, limitations and improvements in isolation. These stages served to identify viable modelling approaches, signal processing techniques and deployment constraints.

In contrast, the results presented below evaluate the fully completed PdM architecture. This version incorporates the validated CNN model trained using multi-speed, augmented datasets, as well as real time embedded inference with network monitoring capabilities. This chapter serves as a clear assessment of the systems robustness, detection reliability and practical development considerations.

The findings presented directly address RQ3 - Robust Embedded Predictive Maintenance System, and form the basis for the concluding evaluation of the project's contributions.

6.1 1D-CNN Model - Multiple Speeds

6.1.1 Generated Anomalies

The three figures below illustrate a few of the input windows used to train the multi-speed 1D-CNN model. For each speed condition (180µs, 240µs, 320µs step delay), both normal and augmented anomalous windows are displayed. Each window contains the normalised x, y and z accelerometer axes. Throughout this section, motor speed is referred to in terms of the microsecond step delay rather than RPM. This convention is adopted because of the Arduino UNO firmware implementation, where speed is explicitly controlled via the step delay parameter.

On the left hand side of the figures, the normal operating windows are presented. On the right hand side, the anomalous windows are shown. As expected from the augmentation design, the injected anomaly occupies approximately half of the windows duration (128 samples).

An interesting observation emerges when comparing the various speeds together. The anomalous windows that appear at 180 µs appear visually similar to the normal operating windows at 240 µs. Similarly, anomalous windows at 320 µs resemble normal operation at 240 µs. This visual overlap of how similar the anomalous and normal windows look demonstrates that simple visual inspection is insufficient for reliable anomaly detection across varying speeds.

Despite the visual similarity, the CNN model is capable of differentiating between anomalous and non-anomalous signals across operating regimes. This supports the earlier theoretical discussions that CNNs can extract temporal and spectral features. The model is therefore learning structural characteristics of anomalous behaviour rather than solely relying on amplitude shifts.

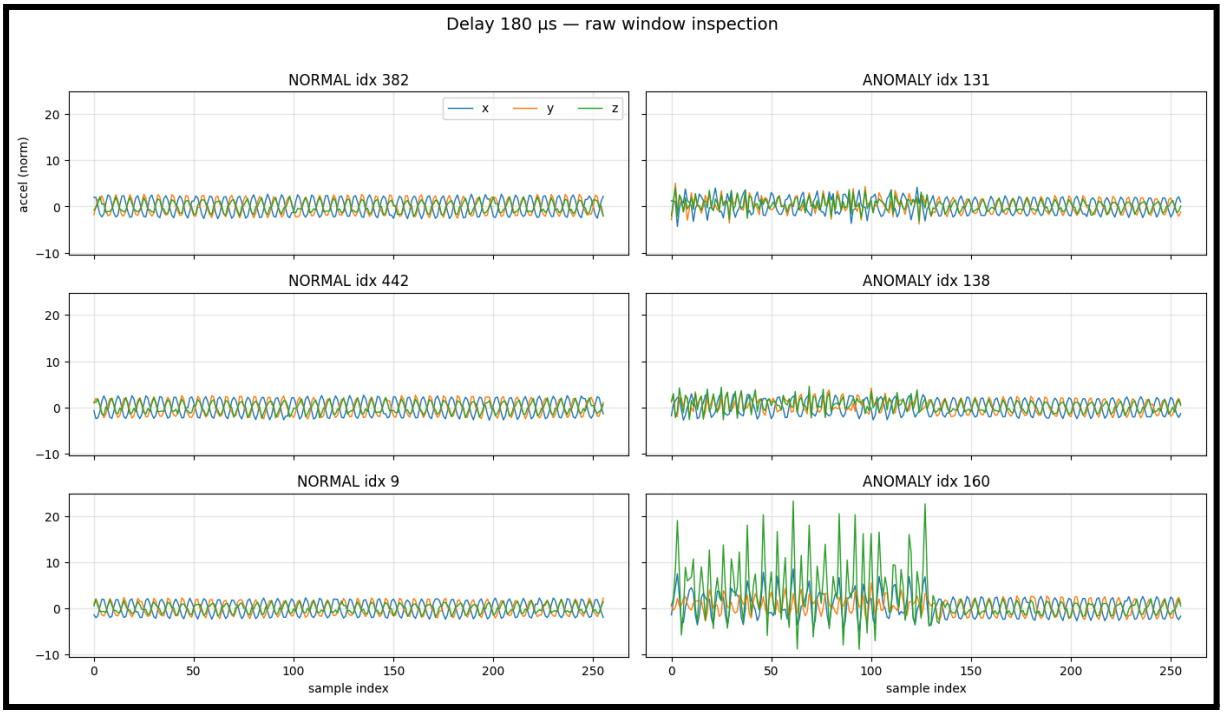


Figure 6.1. Example input windows for 180 µs step delay showing normal operation (left) and augmented anomalous operation (right) across the normalised x, y, z axes.

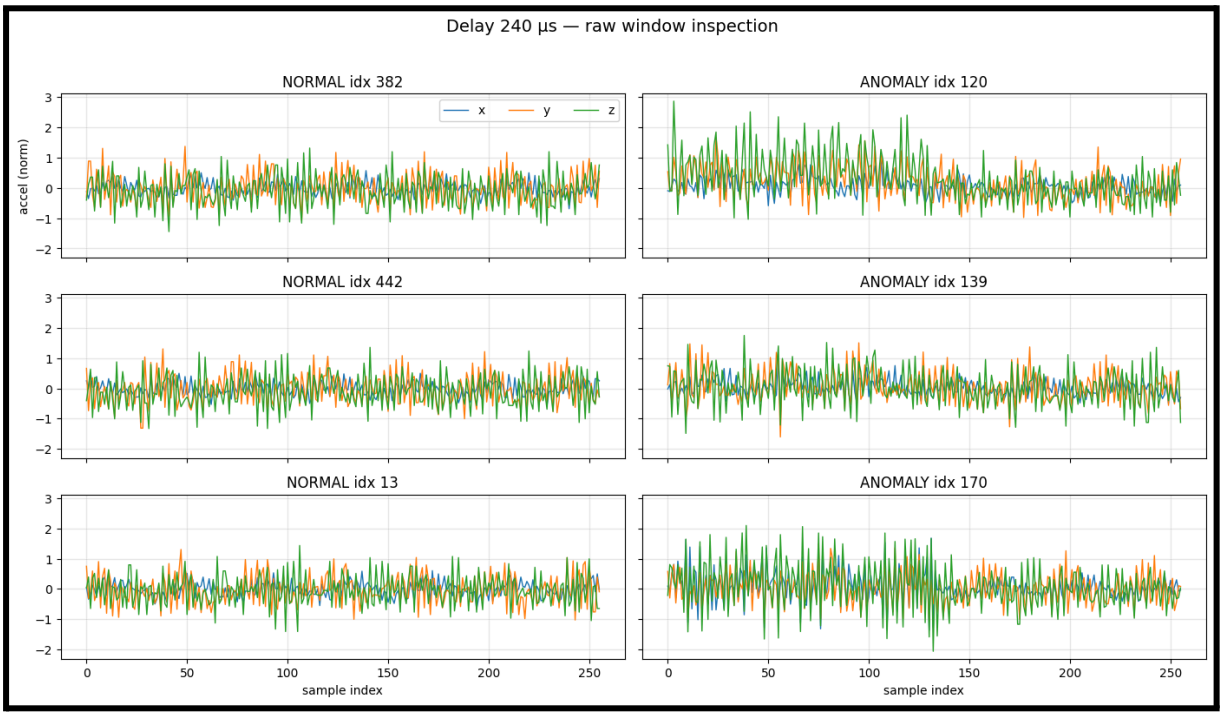


Figure 6.2. Example input windows for 240 µs step delay showing normal operation (left) and augmented anomalous operation (right) across the normalised x, y, z axes.

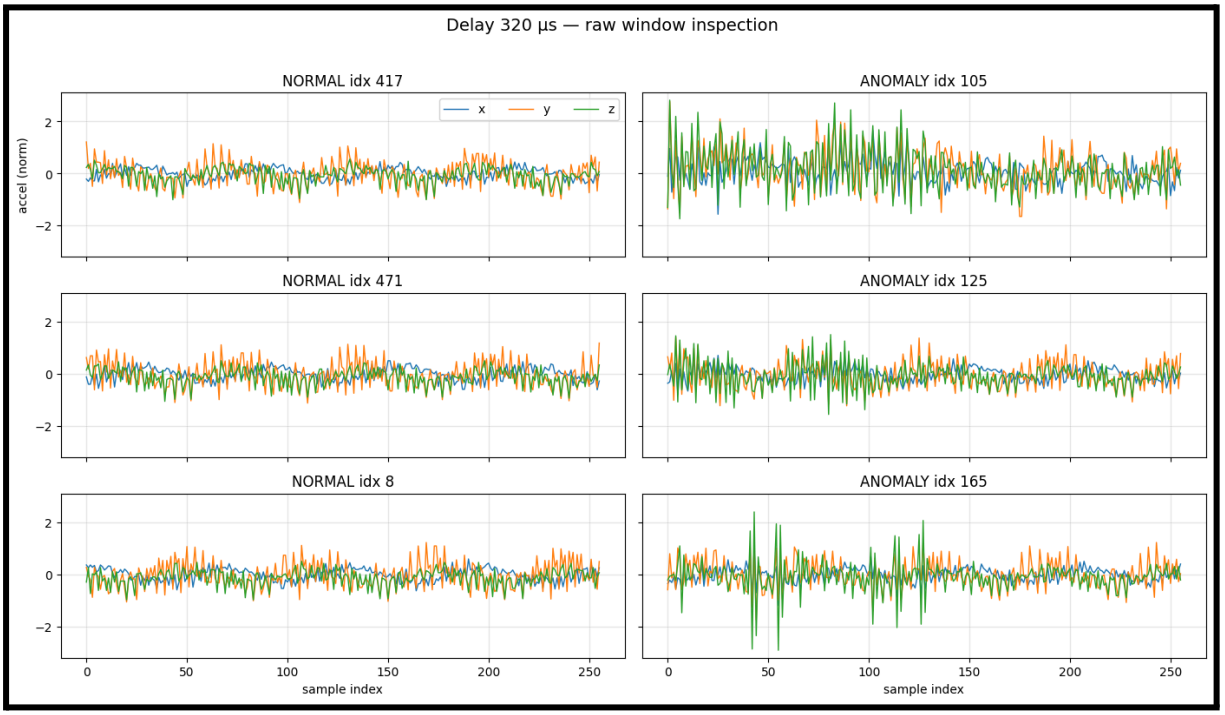


Figure 6.3. Example input windows for 320 µs step delay showing normal operation (left) and augmented anomalous operation (right) across the normalised x, y, z axes.

6.1.2 Quantisation

The quantisation results for the multi-speed model were particularly interesting. When comparing the float32 model outputs with the int8 quantised TensorFlow Lite implementation, the difference between predictions was practically negligible.

The example probability outputs printed:

```
float:  [1.3608177e-13 5.9990885e-14 4.9004366e-13 2.0311921e-13
4.6415606e-13]
tflite: [0. 0. 0. 0. 0.]
mean abs diff: 8.98e-13
```

For normal operating conditions, the float model produced probabilities in the order of 10^{-13}. These values are effectively zero from a classification perspective. Due to the limited resolution of the int8 quantised model, such extremely small values are rounded to 0.

The mean absolute difference between the float and quantised outputs was 8.98^{-13}, confirming that the quantisation step introduced practically no degradation. This demonstrates that the learned decision boundary is extremely well separated for normal conditions, and that model confidence remains the same under the quantised, low precision deployment model.

6.1.3 Model Deployment - LED verification

For deployment testing, the stride was reduced down to 32 samples. With a 200 Hz sampling rate, this resulted in inference updates every 0.16 seconds. This change introduced overlapping windows while significantly reducing the inference latency.

When operating at one of the trained speeds (180µs), the LED consistently indicated normal operation. When the motor was set to 100 µs (previously identified as a resonance band), the LED immediately indicated anomalous behaviour. This confirms that the deployed embedded model behaves within expectations.

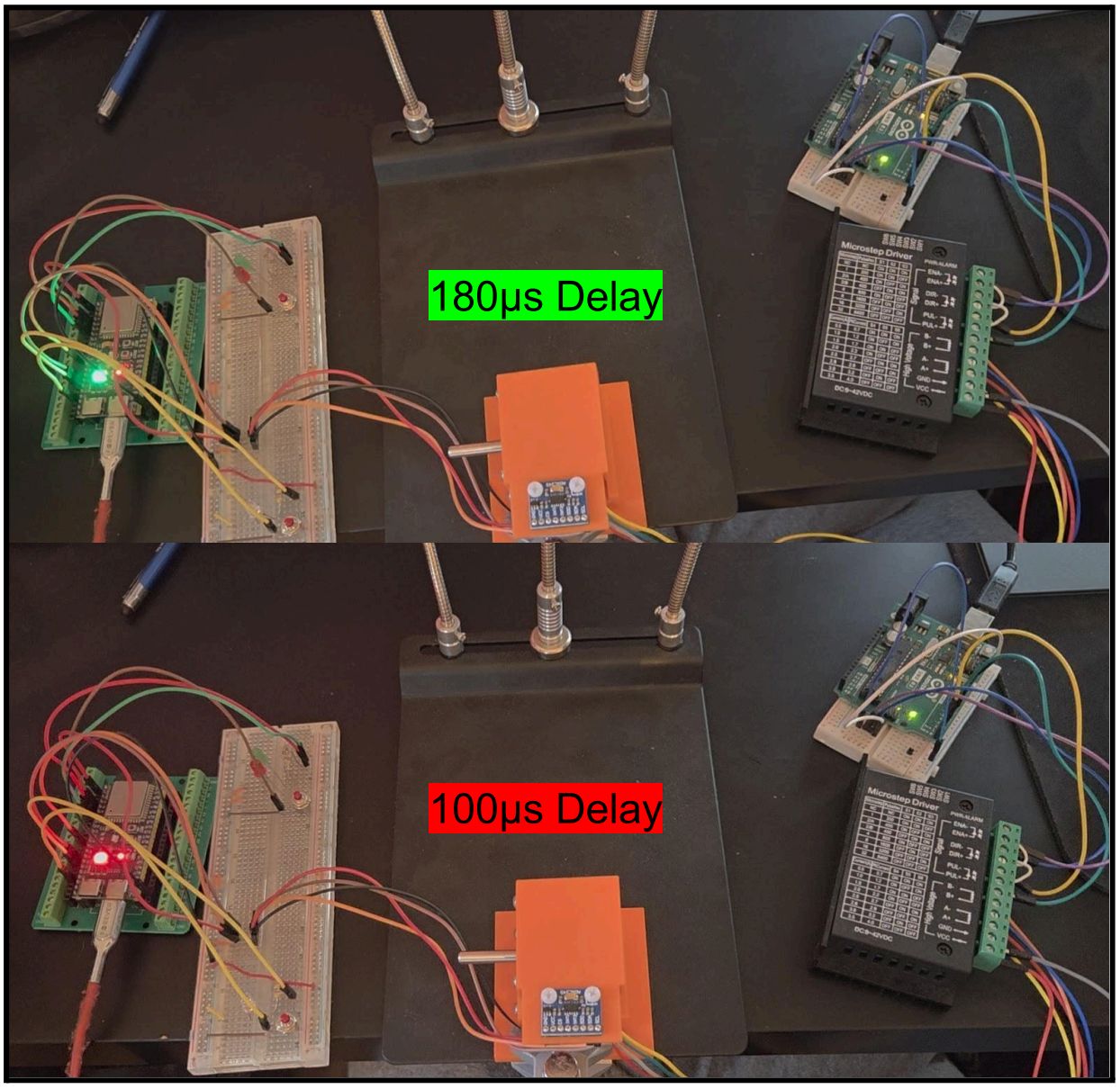


Figure 6.4. Full rig operating at 180µs (top) and 100µs (bottom)

6.2 Robustness Across Operating Conditions

The serial monitor was recorded to gain deeper insight into the probability behaviour across the different operating conditions.

For speeds within the **training distribution** (180µs, 240µs, 320µs), the anomaly probability consistently reported:

- prob = 0.000000
- anomaly = 0

This demonstrates extremely high confidence in normal classification across the trained operating regime.

For a speed **outside of the training regime** (500µs/11.7 RPM) the typical values for the probabilities increased slightly:

- 0.007813
- 0.019531
- 0.050781
- 0.125000 (maximum observed)

However, all the values recorded remained below the anomaly threshold and the anomaly flag remained at 0. This indicated that although the signal characteristics differ from the training data, the model does not falsely classify them as faults. The probability value increases due to a small amount of uncertainty, not misclassification.

For a **high-speed condition** (60µs/156.25RPM), which lies significantly outside the training range, the predicted probability values increased even more:

- 0.125
- 0.273
- 0.500 (maximum observed)
- 0.050

Although these values were noticeably higher than those observed within the training range, no anomaly flag was triggered. All inference probabilities remained below the decision threshold and the classification output remained at anomaly = 0.

This behaviour indicates that while extreme high-speed operation introduces vibration characteristics that differ from the training data, the model does not incorrectly classify them as faulty. The absence of false positives at 60μs demonstrates the model's strong generalisation beyond the trained speed range. This insight suggests that the model is not only associating “high energy” or “high frequency” vibration with anomalous behaviour. Instead, it appears to respond to distortion patterns.

At the **resonance frequency** (100μs), the model consistently produces anomaly flags:

- 0.875
- 0.949
- 0.980
- 0.992
- 0.996

With anomaly flag = 1 in all cases.

This confirms that resonance induced vibration is strongly detected as anomalous. Since this resonance condition was not explicitly included in the trained data, the model successfully identified severe mechanical excitation as abnormal behaviour. This aligns with the project goals since severe mechanical excitation is an anomalous event that needs to be addressed in industrial environments.

With the **coil disconnected**, the model consistently produces anomaly flags:

- 0.996
- 0.992
- 0.996
- 0.996
- 0.996
- 0.949

With anomaly flag = 1 in all cases.

To verify the plausibility of the electrical anomaly, data was collected over a short period of time with the motor running normally and then with the coil disconnected. The anomaly induced by disconnecting the coil is clearly observable within the recorded signal. This behaviour is expected, since the loss of one phase significantly alters the motor’s electromagnetic balance and therefore its mechanical operation. This resulting change in vibration confirms that disconnecting the coil provides a valid and repeatable method for inducing an electrical fault condition to test the model.

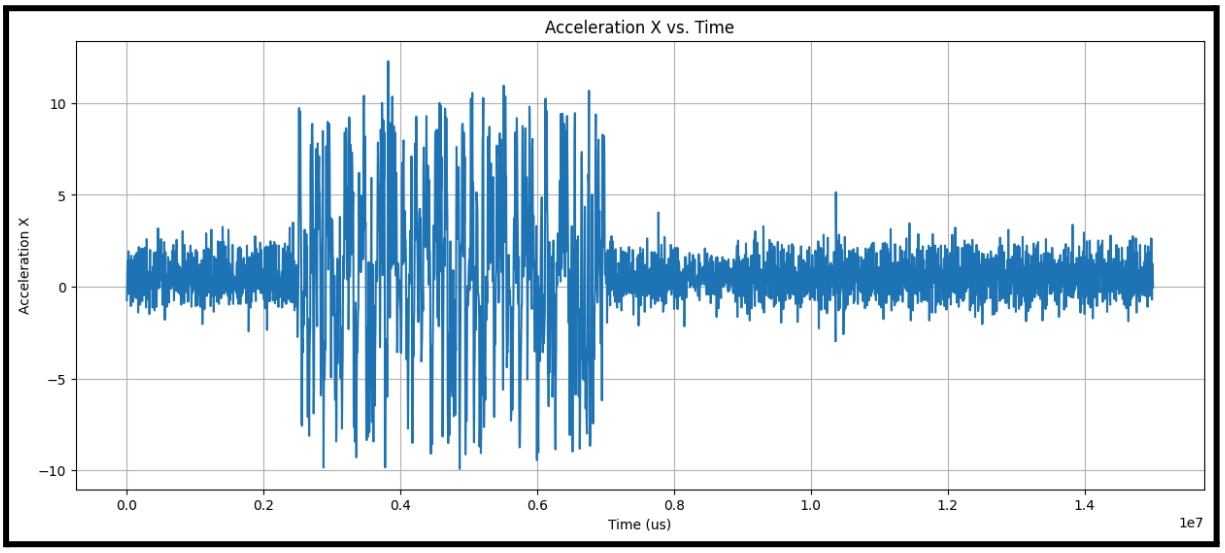


Figure 6.5. X-axis acceleration over time with electrical fault induced.

6.2.1 False Positives

At **80µs delay**, results were mixed. While many windows were correctly classified as normal, several windows exceeded the anomaly threshold:

- 0.875 (anomaly = 1)
- 0.980 (anomaly = 1)

This indicates some false positives at this operating speed. Although 80µs corresponds to a higher mechanical speed than those used in the training data, the behaviour observed suggests that the issue is not purely speed related. Notably, the model performed more consistently at 60µs, despite that representing an even higher rotational speed. This implies that the elevated anomaly probabilities at 80µs are likely linked to the resonance band rather than the increased speed. Importantly, the false positive rate remained low relative to the total number of windows evaluated.

6.2.2 Discussion of Results

Overall, the experimental results demonstrate that the embedded anomaly detection system is robust across a wide range of operating conditions. Within the trained speeds, the model maintained extremely low anomaly probabilities. This indicates that the model has high stability in normal classification cases. When exposed to motor speeds outside of the training data, the probability values increased slightly and in a controlled manner, rather than producing misclassifications. This sort of behaviour suggests that the model is capable of recognising deviations from learned vibration patterns while still being resilient against false positives.

Importantly, severe mechanical excitation associated with structural resonance and electrical imbalance faults were consistently detected, even though these faults were not explicitly included during training. In the case of the induced electrical fault, anomaly probabilities jumped to a range of 94.9%-99.6%, with the anomaly flag active in all observed windows. This behaviour indicates that the CNN has learned generalisable representations of normal and abnormal vibration behaviour, rather than relying on amplitude or frequency thresholds. The mixed results observed near the resonance band further highlight that anomaly detection is influenced by structural dynamics and not purely by rotational speed.

These findings support the design decisions made in Version 2C of the project. The improved generalisation observed in the final system validates the use of physically inspired anomaly injections. Collectively, the results demonstrate that the final embedded architecture can maintain reliable anomaly detection under varying operating speeds. This is sufficient evidence to prove that the system is suitable for practical PdM monitoring applications.

6.3 IoT Results

The first image shows the motor operating under normal conditions. The anomaly probability value reported by the model in that particular snapshot is 0.007812. As expected, both the anomaly_now and alarm_latched values are 0, indicating that the system is operating normally and that no alarm is present. The MQTT alarm topic is therefore low, which is reflected by the greyed out out light on the monitoring dashboard.

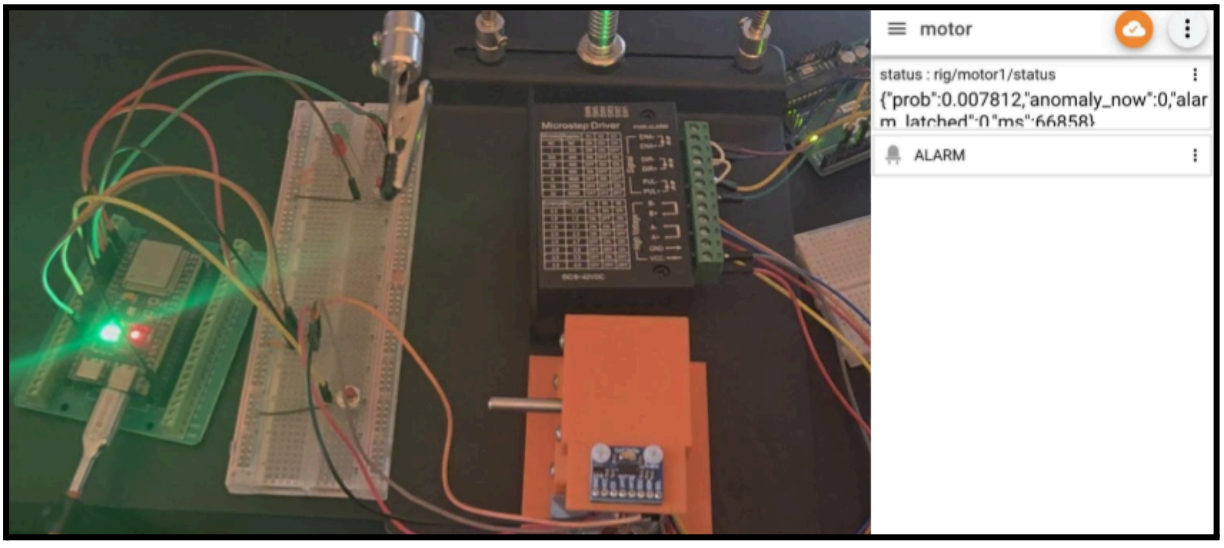


Figure 6.6. Normal operation

The second image shows the motor operating at its resonance frequency. In this case the anomaly probability increases significantly, reaching values as high as 0.996. As a result both the anomaly_now and alarm_latched states are 1. The MQTT alarm topic is indicated on the monitoring dashboard with the red alarm indicator.

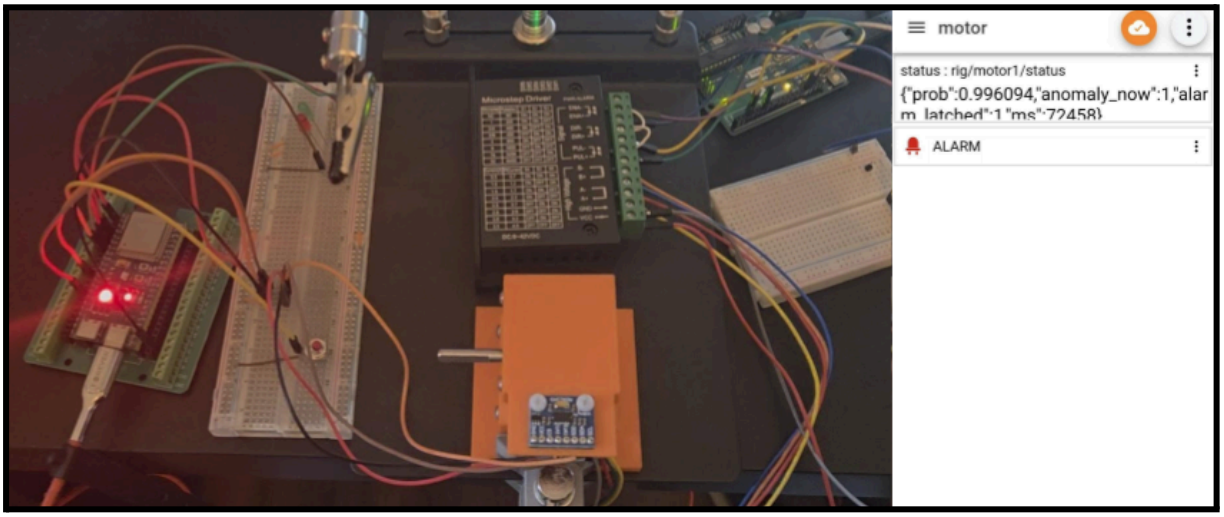


Figure 6.7. Resonance anomaly detection

The third image demonstrates the latching mechanism. In this scenario, the motor has returned to normal operation and the anomaly probability has decreased to 0.003906. Consequently, the anomaly state returns to 0. However, the alarm_latched state remains high, meaning that the alarm must be manually reset before it drops.

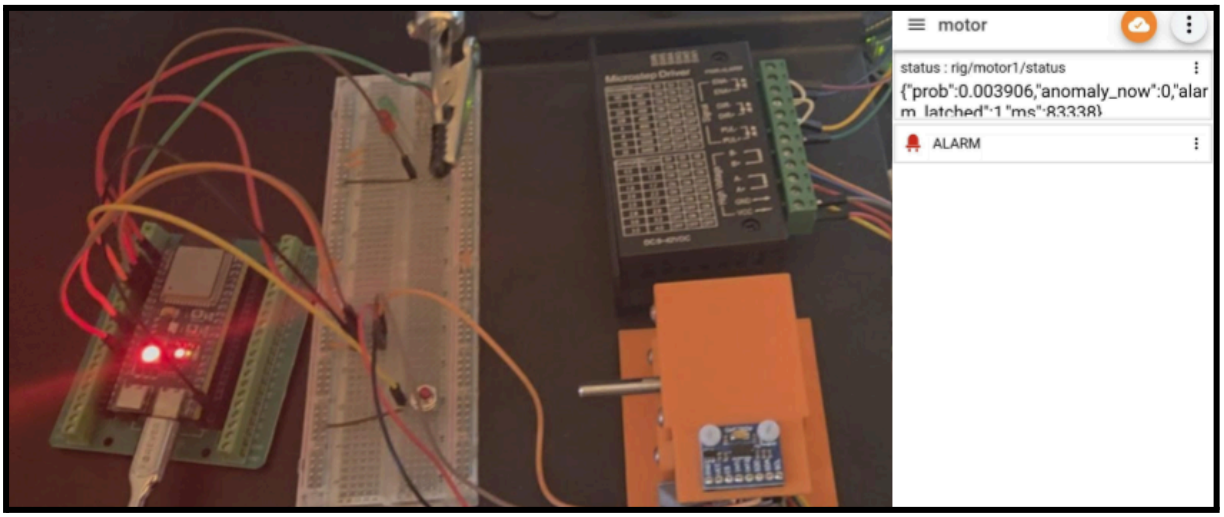


Figure 6.8. Latched alarm state.

The final image shows the system after the alarm has been manually reset using the physical button. Since the system is operating normally, the latch is successfully cleared. Both state values return to 0 and the MQTT alarm topic returns to the low state.

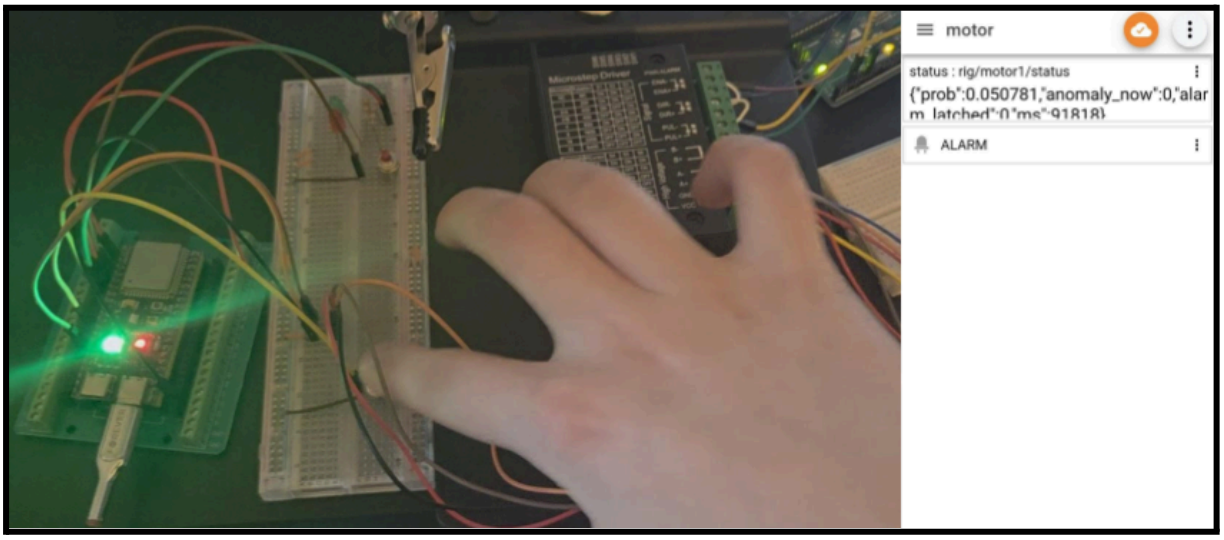


Figure 6.9. Alarm reset and return to normal operation.

The IoT implementation demonstrated that anomaly information could be transmitted reliably while maintaining extremely low communication overhead. During experimental testing, the MQTT status message containing the anomaly probability, anomaly state, latched alarm state, and timestamp was measured at 63 bytes per transmission. Since the maximum package size that can be sent using MQTT is 256 mb, the status message size used here is considered small for IoT and confirms that the system imposes minimal bandwidth requirements [59].

This implementation of the PdM system demonstrates several of the theoretical advantages discussed earlier. Firstly, the low message size associated with MQTT communication limits network transmission to only essential diagnostic information. This reduces energy usage and improves the scalability of the monitoring system when multiple devices are deployed. Secondly, this communication method reduces potential risks of exposure. Since no raw data is transmitted, all detailed operational information remains local to the embedded device. Finally, the implementation of a latched alarm mechanism reflects a more realistic industrial scenario, ensuring that anomaly events remain visible until they are acknowledged, fixed and reset by an operator.

6.4 Embedded Deployment Discussion

6.4.1 Model Deployment

The final quantised CNN was successfully deployed on the ESP32-S3 embedded device using MicroTFLite. The compressed model size was **145 kB**, which is significantly smaller than the available 16 MB flash memory. This confirms that the model can easily be stored on the device alongside MQTT communication, other firmware and logging functionality.

During runtime, the Tensor Arena allocation required **11,572 bytes** (11.6 kB), which is well within the available 512 kB RAM of the device. This demonstrates that real-time deep learning inference can be performed on low cost microcontrollers. Importantly, this relatively small memory footprint leaves sufficient headroom for future system extensions, such as multi sensor functionality or higher level prognostic capabilities.

Inference was performed every 32 new samples, corresponding to approximately **0.16 seconds** at the 200 Hz sampling rate. This configuration was trial and tested, where a stride of 32 samples provided the best balance between detection responsiveness and computational overhead. The signal acquisition loop remained stable throughout extended tests exceeding over an hour, indicating that the sensing, inference and network communication did not introduce timing jitter or temporal instability.

During live operation, anomaly detections triggered both the NeoPixel LED and the MQTT alarm transmission with minimal latency (visually instant). Network connectivity remained stable during continuous operations, and probability updates were reliably transmitted to the online dashboard. Overall, the entire sensing, inference, communication pipeline can operate concurrently without compromising real-time performance.

6.4.2 Practical Deployment

While the embedded PdM system demonstrated reliable and consistent performance in controlled conditions, practical development considerations were identified.

Sensor mounting was observed to influence vibration signal characteristics. Changes in mounting stiffness or placement can alter the measured vibration response, potentially requiring recalibration or retraining. This highlights the importance of consistent installation procedures in real industrial environments.

The data collection process required to train the multi speed model was relatively fast. Approximately 15 minutes of vibration recordings per operating speed was sufficient to construct the training dataset. The augmentation pipeline required less than one minute per dataset to generate synthetic anomalous samples. Therefore, the entire model preparation process could be completed in under one hour. This suggests that similar monitoring systems could be rapidly developed and deployed in small scale settings where historical data is unavailable.

However, long term operational effects such as gradual wear, temperature variation and sensor drifts were not evaluated within the scope of this project. These factors may influence the vibration behaviour over extended periods and could necessitate model retraining or adaptive threshold adjustment in real world applications.

6.5 Limitations of Final System

Despite the reliable anomaly detection performance under the testing conditions, several limitations need to be considered.

Firstly, the experimental validation of the model was conducted using a single stepper motor running at a somewhat low torque output. While this allowed controlled investigation of vibration behaviour and repeatable anomaly generation, the results may not fully represent the variability present across different motor types, mounting conditions or industrial noise. Variations in mechanical coupling, structural resonance and load conditions could influence vibration behaviour and therefore affect the models performance. In addition, larger industrial motors typically produce higher back EMF during operation. In practical applications, such interference could degrade accelerometer signal quality, potentially impacting the reliability of the data acquisition pipeline. This highlights the need for careful compatibility considerations and power supply isolation in future system implementations.

Secondly, the duration of the monitoring experiments was limited. The longest uninterrupted operational test was approximately one hour. Although the system was stable and deterministic sampling was maintained, long term effects were not evaluated. The factors associated with long term runs may include slow changes in vibration characteristics and could require model retaining or adaptive threshold tuning.

Another limitation concerns the diversity of anomaly signatures included during training. While physically inspired anomaly injections were successfully demonstrated to improve detection robustness, the range of generated faults was limited to bearing-like impulsive disturbances. Real industrial faults may include complex degradation mechanisms such as misalignment, imbalance evolution or lack of lubrication. The augmentation framework provides a flexible and easily implementable foundation for introducing additional fault types, and further introducing a broader range of failure modes would strengthen the systems robustness.

The anomaly decision threshold was selected during experimental observations. While a threshold of 0.8 provided a suitable balance between sensitivity and false positive avoidance, optimal threshold selection may vary depending on the machine, operating conditions and risk tolerance. Future implementations could include adaptive thresholding strategies or confidence based alarm logic to improve flexibility.

Despite these limitations, the overall findings provide strong evidence that the proposed system represents a practical and scalable foundation for embedded PdM applications. The results directly address all research questions, and establish the technical basis for the broader evaluation of engineering contributions and future development directions in the following chapter.

Chapter 7: Conclusions & Recommendations

7.1 Conclusions

The main conclusions from this research are summarised as follows:

- Vibration data suitable for machine learning based anomaly detection can be reliably collected using a low cost accelerometer operating at 200 Hz. The developed data gathering pipeline provided stable sampling behaviour and sufficient signal quality for feature extraction and deep learning model training.
- Anomaly detection using both classical and deep learning methods is feasible on resource limited embedded devices. The successful deployment of a quantised CNN on the ESP32-S3 demonstrates that real time vibration analysis can be performed without reliance on cloud computing.
- The trained 1D-CNN model was capable of detecting anomalous vibrations patterns directly from raw signals generated by a running motor. Severe mechanical excitation from resonance conditions and electrically induced imbalance faults were consistently detected as anomalous, even when such conditions were not explicitly included within the training dataset.
- The investigation into data scarcity mitigation methods revealed that simple signal lengthening through signal duplication did not improve robustness. In contrast, physically inspired anomaly augmentation techniques enriched the training dataset and improved model generalisation.
- The final integrated PdM architecture demonstrated robust performance across multiple motor speeds and previously unseen operating speeds. Controlled increases in anomaly probabilities were observed when the motor was operating outside the training regime but no false triggering occurred. This indicates that the model learned a generalised representation of normal and abnormal vibration behaviour.
- Real-time embedded inference, deterministic signal acquisition and stable MQTT communication were successfully achieved within the memory constraints of the ESP32-S3. The resulting system validates the feasibility of a fully independent Edge AI monitoring device.
- The implementation of a latched alarm and lightweight IoT communication improves practical usability by ensuring that anomalous events remain visible to human operators.

The figure below details the objectives and achievements of each model version.



Figure 7.1. Objectives and Achievements of each model iteration.

7.2 Recommendations for Future Work

Based on the review of literature, findings and limitations identified in this research, the following recommendations are proposed for future work:

- Validation of the proposed final monitoring architecture across a wider range of motors, load conditions and industrial environments to assess generalisation performance.
- Integration of additional physically informed fault types to represent a broader range of fault mechanisms such as misalignment or imbalance.
- Investigation of adaptive thresholding alarm logic to improve detection flexibility.
- Long-term monitoring studies to evaluate system performance under gradual wear conditions, sensor drift and environmental variability.
- Exploration of multi-sensor anomaly detection including current and temperature measurements to improve diagnostic accuracy.
- Development of larger IoT architecture enabling fleet level monitoring, model updates and predictive analytics while maintaining local inference capabilities.
- Extension of system towards prognostic functionality, enabling estimation of remaining useful life (RUL) rather than binary anomaly detection.

References

[1] ‘ABB survey reveals unplanned downtime costs $125,000 per hour’, Zurich, Switzerland, Oct. 11, 2023. [Online]. Available: https://new.abb.com/news/detail/107660/abb-survey-reveals-unplanned-downtime-costs-125000-per-hour

[2] G. Immerman, ‘The Real Cost of Downtime in Manufacturing’. Accessed: Nov. 11, 2025. [Online]. Available: https://www.machinemetrics.com/blog/the-real-cost-of-downtime-in-manufacturing

[3] ‘Accreditation Process | Engineers Ireland’. Accessed: Mar. 14, 2026. [Online]. Available: https://www.engineersireland.ie/Professionals/Membership/Become-a-member/Accredited-third-level-courses/Accreditation-criteria-procedure-and-training

[4] Siemens AG, ‘The True Cost of Downtime 2024’. Digital Industries, 2024. [Online]. Available: https://assets.new.siemens.com/siemens/assets/api/uuid:1b43afb5-2d07-47f7-9eb7-893fe7d0bc59/TCOD-2024_original.pdf

[5] D. Thomas and B. Weiss, ‘Maintenance Costs and Advanced Maintenance Techniques in Manufacturing Machinery: Survey and Analysis’, *Int. J. Progn. Health Manag.*, vol. 12, no. 1, p. 10.36001/ijphm.2021.v12i1.2883, 2021, doi: 10.36001/ijphm.2021.v12i1.2883.

[6] M. Jasiulewicz - Kaczmarek and A. Gola, ‘Maintenance 4.0 Technologies for Sustainable Manufacturing - an Overview’, *IFAC-Pap.*, vol. 52, no. 10, pp. 91–96, Jan. 2019, doi: 10.1016/j.ifacol.2019.10.005.

[7] P. Vejjanugraha and W. Pawasarn, ‘Technician Utilization-Based Analysis of Mean Time to Detect (MTTD) and Mean Time to Repair (MTTR) for a Strategic Approach in Failure Corrective Maintenance System (FCMS)’, in *2025 20th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, Nov. 2025, pp. 1–5. doi: 10.1109/iSAI-NLP66160.2025.11320480.

[8] M. H. Abidi, M. K. Mohammed, and H. Alkhalefah, ‘Predictive Maintenance Planning for Industry 4.0 Using Machine Learning for Sustainable Manufacturing’, *Sustainability*, vol. 14, no. 6, p. 3387, Jan. 2022, doi: 10.3390/su14063387.

[9] M. Achouch *et al.*, ‘On Predictive Maintenance in Industry 4.0: Overview, Models, and Challenges’, *Appl. Sci.*, vol. 12, no. 16, p. 8081, Jan. 2022, doi: 10.3390/app12168081.

[10] J. M. Hunde *et al.*, ‘Data-driven and physics-based modeling approaches and their integration in building digital twins: A systematic review’, *J. Build. Eng.*, vol. 114, p. 114214, Nov. 2025, doi: 10.1016/j.jobe.2025.114214.

[11] J. Lee, B. Bagheri, and H.-A. Kao, ‘A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems’, *Manuf. Lett.*, vol. 3, pp. 18–23, Jan. 2015, doi: 10.1016/j.mfglet.2014.12.001.

[12] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. da P. Francisco, J. P. Basto, and S. G. S. Alcalá, ‘A systematic literature review of machine learning methods applied to predictive maintenance’, *Comput. Ind. Eng.*, vol. 137, p. 106024, Nov. 2019, doi: 10.1016/j.cie.2019.106024.

[13] X.-S. Si, W. Wang, C.-H. Hu, and D.-H. Zhou, ‘Remaining useful life estimation – A review on the statistical data driven approaches’, *Eur. J. Oper. Res.*, vol. 213, no. 1, pp. 1–14, Aug. 2011, doi: 10.1016/j.ejor.2010.11.018.

[14] A. K. S. Jardine, D. Lin, and D. Banjevic, ‘A review on machinery diagnostics and prognostics implementing condition-based maintenance’, *Mech. Syst. Signal Process.*, vol. 20, no. 7, pp. 1483–1510, Oct. 2006, doi: 10.1016/j.ymssp.2005.09.012.

[15] W. Zhang, D. Yang, and H. Wang, ‘Data-Driven Methods for Predictive Maintenance of Industrial Equipment: A Survey’, *IEEE Syst. J.*, vol. 13, no. 3, pp. 2213–2227, Sep. 2019, doi: 10.1109/JSYST.2019.2905565.

[16] R. K. Mobley, *An introduction to predictive maintenance*, 2nd ed. in Plant Engineering. Amsterdam ; Butterworth-Heinemann, 2002.

[17] W. T. Thomson and M. Fenger, ‘Current Signature Analysis to Detect Induction Motor Faults’, *IEEE Ind. Appl. Mag.*, vol. 7, no. 4, pp. 26–34, Jul. 2001, doi: 10.1109/2943.930988.

[18] Y. Lei, J. Lin, Z. He, and M. J. Zuo, ‘A review on empirical mode decomposition in fault diagnosis of rotating machinery’, *Mech. Syst. Signal Process.*, vol. 35, no. 1, pp. 108–126, Feb. 2013, doi: 10.1016/j.ymssp.2012.09.015.

[19] A. Widodo and B.-S. Yang, ‘Support vector machine in machine condition monitoring and fault diagnosis’, *Mech. Syst. Signal Process.*, vol. 21, no. 6, pp. 2560–2574, Aug. 2007, doi: 10.1016/j.ymssp.2006.12.007.

[20] O. Barzilay and V. L. Brailovsky, ‘On domain knowledge and feature selection using a support vector machine’, *Pattern Recognit. Lett.*, vol. 20, no. 5, pp. 475–484, May 1999, doi: 10.1016/S0167-8655(99)00014-8.

[21] A. Heng, S. Zhang, A. C. C. Tan, and J. Mathew, ‘Rotating machinery prognostics: State of the art, challenges and opportunities’, *Mech. Syst. Signal Process.*, vol. 23, no. 3, pp. 724–739, Apr. 2009, doi: 10.1016/j.ymssp.2008.06.009.

[22] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, and R. X. Gao, ‘Deep learning and its applications to machine health monitoring’, *Mech. Syst. Signal Process.*, vol. 115, pp. 213–237, Jan. 2019, doi: 10.1016/j.ymssp.2018.05.050.

[23] Y. LeCun, Y. Bengio, and G. Hinton, ‘Deep learning’, *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015, doi: 10.1038/nature14539.

[24] J. Wu, ‘Introduction to Convolutional Neural Networks’.

[25] A. Graves, ‘Generating Sequences With Recurrent Neural Networks’, Jun. 05, 2014, *arXiv*: arXiv:1308.0850. doi: 10.48550/arXiv.1308.0850.

[26] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, ‘Long Short Term Memory Networks for Anomaly Detection in Time Series’, *Comput. Intell.*, 2015.

[27] R. Zhao, R. Yan, J. Wang, and K. Mao, ‘Learning to Monitor Machine Health with Convolutional Bi-Directional LSTM Networks’, *Sensors*, vol. 17, no. 2, p. 273, Feb. 2017, doi: 10.3390/s17020273.

[28] R. Chalapathy, A. Menon, and S. Chawla, ‘Anomaly Detection using One-Class Neural Networks’, *ArXiv*, Feb. 2018, Accessed: Feb. 22, 2026. [Online]. Available: https://www.semanticscholar.org/paper/67b9c2b376a01d8757dc6d704be450d1c46c4ced

[29] K. Zhang *et al.*, ‘Self-Supervised Learning for Time Series Analysis: Taxonomy, Progress, and Prospects’, *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 10, pp. 6775–6794, Oct. 2024, doi: 10.1109/TPAMI.2024.3387317.

[30] L. Alzubaidi *et al.*, ‘A survey on deep learning tools dealing with data scarcity: definitions, challenges, solutions, tips, and applications’, *J. Big Data*, vol. 10, no. 1, p. 46, Apr. 2023, doi: 10.1186/s40537-023-00727-2.

[31] ‘(PDF) Edge AI for Predictive Maintenance in Industrial IoT Environments’, ResearchGate. Accessed: Nov. 11, 2025. [Online]. Available: https://www.researchgate.net/publication/391384065_Edge_AI_for_Predictive_Maintenance_in_Industrial_IoT_Environments

[32] C. C. Walani and W. Doorsamy, ‘Edge vs. Cloud: Empirical Insights into Data-Driven Condition Monitoring’, *Big Data Cogn. Comput.*, vol. 9, no. 5, p. 121, May 2025, doi: 10.3390/bdcc9050121.

[33] K. Weiss, T. M. Khoshgoftaar, and D. Wang, ‘A survey of transfer learning’, *J. Big Data*, vol. 3, no. 1, p. 9, May 2016, doi: 10.1186/s40537-016-0043-6.

[34] O. Vermesan, F. Wotawa, M. D. Nava, and B. Debaillie, *Industrial Artificial Intelligence Technologies and Applications*. River Publishers, 2022. Accessed: Nov. 11, 2025. [Online]. Available: https://riverpublishers.com/book_details.php?book_id=1024

[35] N. N. Alajlan and D. M. Ibrahim, ‘TinyML: Enabling of Inference Deep Learning Models on Ultra-Low-Power IoT Edge Devices for AI Applications’, *Micromachines*, vol. 13, no. 6, p. 851, Jun. 2022, doi: 10.3390/mi13060851.

[36] A. Soro, G. Brunner, S. Tanner, and R. Wattenhofer, ‘Recognition and Repetition Counting for Complex Physical Exercises with Deep Learning’, *Sensors*, vol. 19, no. 3, p. 714, Jan. 2019, doi: 10.3390/s19030714.

[37] ‘Microsoft Word - 20230130_Code of Ethics (2023) 2023 01 30 A’. Accessed: Mar. 11, 2026. [Online]. Available: https://www.engineersireland.ie/LinkClick.aspx?fileticket=0L3VSJJG_xY%3D&portalid=0&resourceView=1

[38] B. D. Mittelstadt, P. Allo, M. Taddeo, S. Wachter, and L. Floridi, ‘The ethics of algorithms: Mapping the debate’, *Big Data Soc.*, vol. 3, no. 2, p. 2053951716679679, Dec. 2016, doi: 10.1177/2053951716679679.

[39] J. Forge, ‘A Note on the Definition of “Dual Use”’, *Sci. Eng. Ethics*, vol. 16, no. 1, pp. 111–118, Mar. 2010, doi: 10.1007/s11948-009-9159-9.

[40] P. Lin, K. Abney, and G. A. Bekey, *Robot Ethics: The Ethical and Social Implications of Robotics*. MIT Press, 2014.

[41] ‘Data centres & networks’, IEA. Accessed: Mar. 16, 2026. [Online]. Available: https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks

[42] ‘The Sustainable Development Goals, explained’, Concern Worldwide. Accessed: Mar. 16, 2026. [Online]. Available: https://www.concern.net/news/sustainable-development-goals-explained

[43] S. Al-Rabiaah, ‘The “Stuxnet” Virus of 2010 As an Example of A “APT” and Its “Recent” Variances’, in *2018 21st Saudi Computer Society National Computer Conference (NCC)*, Apr. 2018, pp. 1–5. doi: 10.1109/NCG.2018.8593143.

[44] ‘ADXL345 (Rev. G)’. Accessed: Mar. 02, 2026. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/adxl345.pdf

[45] N. Mofi, A. Hassan, A. Alsaffar, and S. A. Aldossari, ‘IoT Earthquake Detection Using Localization and GSM Networks’, in *2024 8th International Symposium on Innovative Approaches in Smart Technologies (ISAS)*, Dec. 2024, pp. 1–4. doi: 10.1109/ISAS64331.2024.10845624.

[46] A. Intizar, ‘EEN1083 Data Analysis and Machine Learning I - “Linear Regression with Gradient Descent (Colab)’, Dublin City University, School of Electronic Engineering, 2025.

[47] Y. Bai, ‘RELU-Function and Derived Function Review’, *SHS Web Conf.*, vol. 144, p. 02006, 2022, doi: 10.1051/shsconf/202214402006.

[48] ‘Affine Quantization ’:, Gen AI Guide. Accessed: Feb. 25, 2026. [Online]. Available: https://example.org/170.quantization/10.affine/index.html

[49] B. Iglewicz and D. C. Hoaglin, *Volume 16: How to Detect and Handle Outliers*, vol. 16. Quality Press, 1993.

[50] ‘MDI Case Study: Inner Race Bearing Defect | CTC’. Accessed: Feb. 26, 2026. [Online]. Available: https://www.ctconline.com/blog-archive/mdi-case-study-inner-race-bearing-defect/

[51] D.-J. Jwo, I.-H. Wu, and Y. Chang, ‘Windowing Design and Performance Assessment for Mitigation of Spectrum Leakage’, *E3S Web Conf.*, vol. 94, p. 03001, 2019, doi: 10.1051/e3sconf/20199403001.

[52] ‘numpy.hanning — NumPy v2.5.dev0 Manual’. Accessed: Feb. 26, 2026. [Online]. Available: https://numpy.org/devdocs/reference/generated/numpy.hanning.html

[53] ‘butter — SciPy v1.17.0 Manual’. Accessed: Feb. 26, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html

[54] ‘filtfilt — SciPy v1.17.0 Manual’. Accessed: Feb. 26, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.filtfilt.html

[55] ‘hilbert — SciPy v1.17.0 Manual’. Accessed: Feb. 26, 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.hilbert.html

[56] J. Antoni, D. Abboud, and G. Xin, ‘Cyclostationarity in Condition Monitoring: 10 years after’.

[57] J. Antoni, ‘Cyclostationarity by examples’, *Mech. Syst. Signal Process.*, vol. 23, no. 4, pp. 987–1036, May 2009, doi: 10.1016/j.ymssp.2008.10.010.

[58] N. Sawalhi and R. B. Randall, ‘Simulating gear and bearing interactions in the presence of faults’, *Mech. Syst. Signal Process.*, vol. 22, no. 8, pp. 1924–1951, Nov. 2008, doi: 10.1016/j.ymssp.2007.12.001.

[59] N. Naik, ‘Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP’, in *2017 IEEE International Systems Engineering Symposium (ISSE)*, Oct. 2017, pp. 1–7. doi: 10.1109/SysEng.2017.8088251.

[60] ‘mosquitto_passwd man page’, Eclipse Mosquitto. Accessed: Mar. 11, 2026. [Online]. Available: https://mosquitto.org/man/mosquitto_passwd-1.html

[61] ‘SPIFFS fast enough for a one second data logger? - ESP32 Forum’. Accessed: Feb. 26, 2026. [Online]. Available: https://esp32.com/viewtopic.php?t=5070

Appendix A - SPIFFS Data Logging

The first iteration began as a functionality testing prototype, where data logging was coupled directly to the servo motion logic. The ADXL345 was sampled whenever the servo position was updated and not at a fixed rate. Timestamps were computed using millis() - startTime, where millis() is a built-in timing function that returns the number of milliseconds elapsed since the microcontroller was powered or reset. startTime was a referenced value captured during the systems initialisation to approximate a near zero-time origin for the data logging experiment. Consequently, all timestamps in the dataset were expressed with a resolution in the milliseconds and represented *elapsed time relative to the start of execution*, rather than being tied to a deterministic sampling schedule.

Each accelerometer sample was appended to a CSV file stored on SPIFFS. SPIFFS is the ESP32's internal flash-based file system designed to work as a lightweight, persistent storage layer on embedded devices. For every logged sample, the file was opened in append mode, a single line of comma-separated data was written, and the file buffer was flushed to flash memory. While SPIFFS enables non-volatile storage without the need for external hardware, flash write operations are relatively slow and introduce variable latency due to internal buffering [61]. In parallel to writing to SPIFFS, Wi-Fi connectivity was enabled, and a basic HTTP server endpoint was implemented to allow the CSV file to be downloaded at the end of a logging session.

Although this implementation successfully demonstrated end-to-end data acquisition, it exposed a fundamental limitation inherent to event-driven sampling. Since data logging was triggered by servo motion rather than a timing reference, the effective sampling interval was dependent on multiple variables, including accelerometer read time, SPIFFS latency, Wi-Fi stack servicing, and blocking delays used to control the servo. These sources of variation resulted in irregular and non-uniform time steps between consecutive samples. The first couple of lines from the dataset can be seen below in **Table A.1**, where the timestamps in the first column are shown in milliseconds and the second column shows the difference in time in between samples.

Irregular sampling is problematic for vibration analysis and machine-learning based modelling. When sampling intervals vary, windows of equal sample length correspond to unequal durations in real time, causing misalignment in time-based features and degrading the models performance. As a result, the data produced by this initial implementation was unsuitable for reliable vibration analysis or training ML/DL models. This motivated a transition to a deterministic, time-driven sampling strategy in subsequent iterations.

Table A.1. *Sample of the dataset from iteration 1 of the data logging program.*

Time	Δtime	log_val	x	y	z
104	-	1	0.549172	1.176798	10.08124
288	184	2	1.098345	1.333704	10.39505
399	111	3	0.666852	1.412158	10.00278
510	111	4	0.627626	1.372931	10.00278
621	111	5	0.666852	1.490611	10.04201
734	113	6	0.666852	1.412158	10.00278
844	110	7	0.627626	1.412158	10.00278
955	111	8	0.666852	1.451384	10.00278
1154	199	9	0.627626	1.451384	10.00278

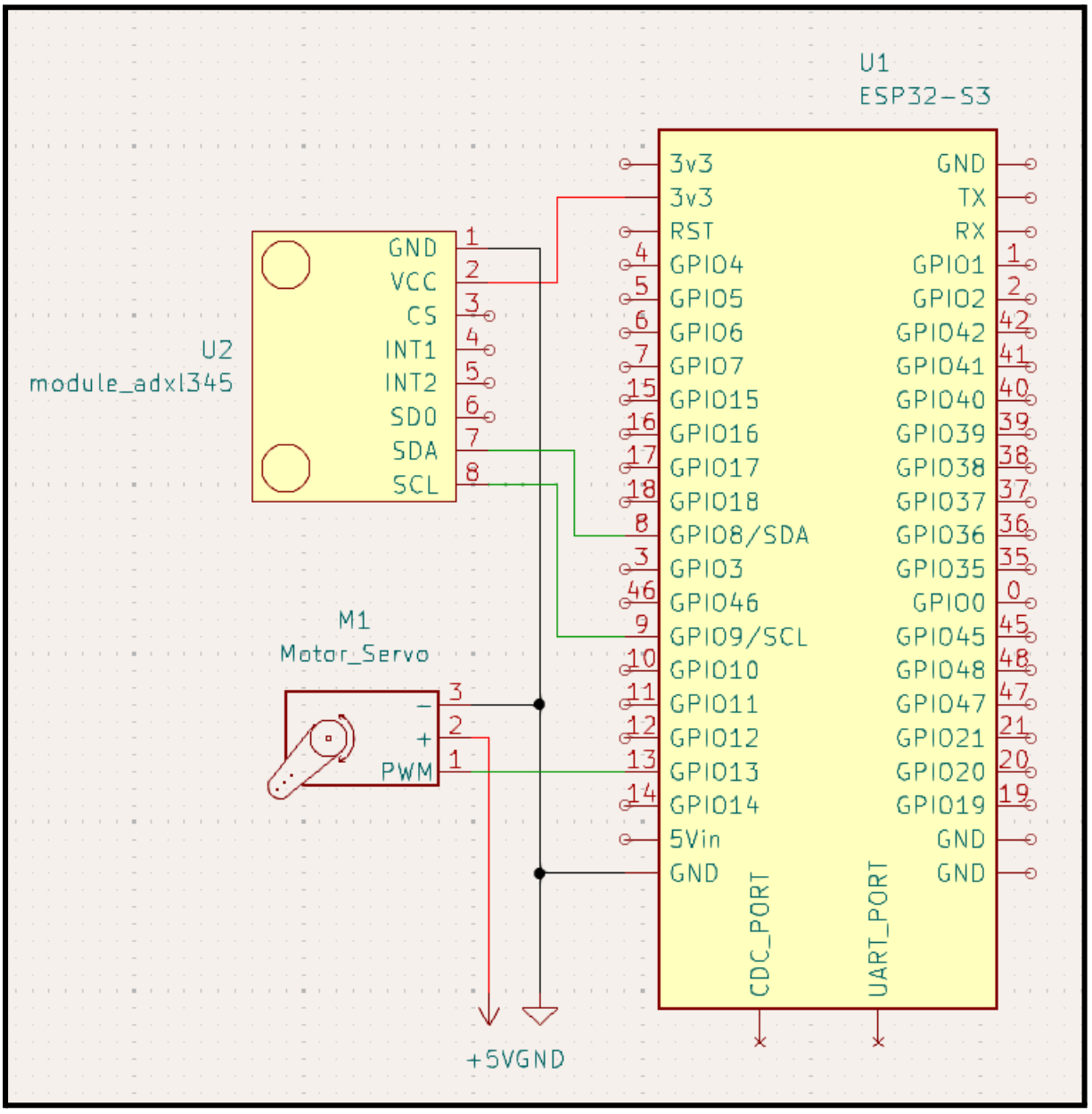


Figure A.1 Iteration 1 Circuit Design

In the second iteration of the data logging system, the design shifted from servo-driven logging to time-driven logging by introducing a fixed rate scheduler based on `micros()`. A target sampling rate of 200 Hz was defined explicitly, and sampling was triggered when the microsecond timer crossed a scheduled `next_sample_us` threshold. This implementation immediately improved the validity of the data because sampling was no longer dependent on an event. The use of microsecond-resolution timestamps also made the recorded data more appropriate for future windowing and frequency-domain processing. This iteration still performed per-sample file operations (open, append, close) and handled HTTP server requests continuously within the main loop, meaning that the filesystem overhead and network activity could still introduce jitter into the sampling schedule.

Iteration three addressed the remaining sources of timing variability by treating data acquisition as the real-time priority and delegating storage and network tasks to lower frequency operations. Instead of writing every sample immediately, CSV lines were first written into a RAM buffer and flushed to SPIFFS in batches (e.g, every 100 samples), dramatically reducing the frequency of computationally expensive filesystem calls. In parallel, HTTP server handling was decoupled from the main acquisition loop and executed at a fixed polling interval rate rather than on every single loop iteration. By limiting how frequently the web server could process incoming requests, the amount of time spent servicing network traffic during any given sampling period was set. This ensured that network related interrupts and processing could not repeatedly take action before the sampling logic. This further improved the consistency of accelerometer data readings and it helped reduce interference between data acquisition and auxiliary tasks.

This iteration of the program also introduced explicit “lateness” handling. At each scheduled sampling instance, the current microsecond timestamp was compared against the expected `next_sample_us` value. If the system was detected to be running behind a schedule, due to processing delays, the scheduler would resynchronise by advancing the next sampling deadline relative to the current time, rather than repeatedly attempting to service missed samples. This approach was implemented to prevent the accumulation of timing drift and avoid bursts of back-to-back samples that would otherwise distort the temporal structure of the recorded vibration data. The listing can be seen below in **Listing A.1**

Collectively, these modifications reduced I/O timing jitter and constrained the impact of non-deterministic operations on the sampling loop. As a result, the recorded datasets exhibited sampling behaviour at the intended 200 Hz specification. Despite these improvements, prolonged operation (13 minutes) of the buffered SPIFFS-based logger revealed a further limitation. As logging continued, the internal filesystem housekeeping and memory allocation overheads increased, causing SPIFFS write operations to become progressively slower and more variable. This behaviour introduced pressure onto the sampling loop, where the time spent managing buffered writes began to exceed the available slack within the 5,000 µs sampling period. As a result, the system began to miss more sampling instants and, in extended runs the number of dropped samples exceeded the number of samples successfully recorded. This observation highlighted a major constraint of using flash-based filesystems for high-rate, long-duration data logging, motivating a shift into the subsequent iteration.

***Listing A.1**. Lateness Handling implementation.*

```
unsigned long now_us = micros();

if (next_sample_us == 0) {
  next_sample_us = now_us + SAMPLE_PERIOD_US;
}
```

```
if ((int32_t)(now_us - next_sample_us) >= 0) {
  unsigned long lateness = now_us - next_sample_us;
  if (lateness > SAMPLE_PERIOD_US) {
    next_sample_us = now_us + SAMPLE_PERIOD_US;
  } else {
    next_sample_us += SAMPLE_PERIOD_US;
  }
```

Appendix B - Python Logger

Listing B.1. *Python Logger Code*

```
import serial
import time
from datetime import datetime

PORT = "COM7"          # CDC port (COM7 on PC)
BAUD = 115200
OUTFILE = f"accel_log_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"
#File name

print(f"[INFO] Opening {PORT} ...")
ser = serial.Serial(PORT, BAUD, timeout=1)

print(f"[INFO] Writing to {OUTFILE} ...")
lines = 0
last_report = time.time()

with open(OUTFILE, "w", buffering=1) as f:  # line-buffered
    # write a header if not sending using esp32
    # f.write("Time_us,log_val,x,y,z\n")G

    while True:
        raw = ser.readline()
        if not raw:
            if time.time() - last_report > 2:
                print("[WARN] Press Button to begin logging")
                last_report = time.time()
            continue

        try:
            text = raw.decode("utf-8", errors="ignore").strip()
        except Exception:
            continue

        #Skip status lines that start with '#'
        if not text or text.startswith("#"):
            continue

        f.write(text + "\n")
        lines += 1
        #Force periodic flush so file size grows visibly
        if lines % 200 == 0:   #~1 second at 200 Hz
            f.flush()
            print(f"[INFO] Logged {lines} lines")
```

Appendix C - Augmented Anomaly Generation

Listing C.1 Ring resonator function used to create a decaying sinusoid.

```
def ring_resonator(imp, fs, f0_hz=120.0, decay=0.015):
    n = len(imp)
    t = np.arange(n) / fs
    h = np.exp(-t/(decay + 1e-9)) * np.sin(2*np.pi*f0_hz*t)
    y = np.convolve(imp, h, mode="same")
    return y
```

Listing C.2 Bearing-like fault injection.

```
def inject_bearing_like(x_win, fs, fr_hz=None, k_mult_range=(3.0, 10.0),
snr_db_range=(-6, 6)):
    x = x_win.copy()
    n = x.shape[0]
    t = np.arange(n) / fs

    if fr_hz is None:
        fr_hz = estimate_fr_hz(x[:,2], fs)  #uses Z axis, same as data
lengthening

    #STEP 1, random repetition frequency ff
    k = np.random.uniform(*k_mult_range)#random k values inbetween
3->10.
    ff = max(0.5, k * fr_hz)  #fault repetition frequency ff

    #Step 2, timing jitter
    period = fs / ff
    idx = []
    cur = 0.0
    while cur < n:
        jitter = np.random.uniform(-0.03, 0.03) * period  #+/- 3% jitter
        ii = int(round(cur + jitter))
        if 0 <= ii < n:
            idx.append(ii)
        cur += period

    #Step 3, amplitude modulation.
    imp = np.zeros(n)
    if len(idx) > 0:
        fr = fr_hz
        am = 0.6 + 0.4*np.sin(2*np.pi*fr*t + np.random.uniform(0,
2*np.pi)) #Amplitude from theory section
        for ii in idx:
            imp[ii] += (1.0 + 0.3*np.random.randn()) * am[ii] #a_n from
```

```
theory

    #Step 4: Resonance Shaping
    f0 = np.random.uniform(40.0, min(0.45*fs, 250.0))
    decay = np.random.uniform(0.01, 0.05)
    fault_1ch = ring_resonator(imp, fs, f0_hz=f0, decay=decay)

    #Step5: Tria axial mixing
    mix = np.random.uniform(0.3, 1.0, size=(3,)) #axes
    fault = fault_1ch[:, None] * mix[None, :]

    #Step 6: SNR scaling
    sig_rms = np.sqrt(np.mean((x - x.mean(axis=0))**2)) #signal rms
    fault_rms = np.sqrt(np.mean(fault**2)) + 1e-9       #fault rms
    snr_db = np.random.uniform(*snr_db_range)
    target_fault_rms = sig_rms * (10.0 ** (-snr_db/20.0))
    fault *= (target_fault_rms / fault_rms)

    return x + fault #Return window with bearing like fault
```

Appendix D - FYP Risk Assessment

Hazards	Is the Hazard present? Y/N	What is the risk?	Likelihood	Severity	Risk Rating (L x S)	Risk Rating -prior to having controls in place H= high M=Medium L= Low	Controls - Controls in place to reduce risks	Is the control in place? Y/N	Likelihood	Severity	Risk Rating (L x S)	L/M/H	Control measure implementation - Person/Office/Unit /Dept. responsible?
Risk of Component Burning Out	Y	Risk of electronic components (resistors, capacitors, regulators) burning out due to incorrect current/voltage calculations. Risk of electrical fire, electrical shock, smoke or toxic fumes	3	1	3	L	Ensure calculations are correct before enabling power	Y	2	1	2	L	Franciszek Fraszczyk
							Select componence with sufficient tolerance rating	Y					Franciszek Fraszczyk
							Ensure fuses are operational	Y					Franciszek Fraszczyk
							Ensure RCD and properly rated PSU are used	Y					Franciszek Fraszczyk
							Verify Circuit design	Y					Franciszek Fraszczyk
ESP32 (Or Equivalent) Malfunction	Y	Risk of microcontroller failure from overcurrent draw by connected peripherals (servo motor, sensors, accelerometer) Risk of fire, electrical shock, smoke, toxic fumes, power surge. thermal damage. Risk of damage (USB port short circuit) to other devices, such as computer or laptop	3	2	6	M	Verify Current draw against datasheet specifications	Y	1	2	2	L	Franciszek Fraszczyk
							Ensure firmware shuts down properly	Y					Franciszek Fraszczyk
							Use external power supply for motors	Y					Franciszek Fraszczyk
							Use USB voltage regulator if needed	Y					Franciszek Fraszczyk
							Use current limiting resistors on GPIO pins	Y					Franciszek Fraszczyk
Motor Malfunction	Y	Risk of motor overheating or running beyond its designed limits. Risk of fire, electric shock, smoke and toxic fumes. Risk of mechanical injury such as a pinch or minor abrasion	2	2	4	M	Operate withing specified torque, voltage and current ratings	Y	1	1	1	L	Franciszek Fraszczyk
							Physically secure the motor to avoid unwanted movement	Y					Franciszek Fraszczyk
							Disconnect power entirely when making mechanical adjustments	Y					Franciszek Fraszczyk
							Keep moving parts enclosed	Y					Franciszek Fraszczyk
Health safety while programming	Y	Risk of posture damage and eye strain during extended programming sessions.	3	1	3	L	Ensure correct posture is adopted whilst programming/wiring	Y	1	1	1	L	Franciszek Fraszczyk
							Take short breaks every 45-60 minutes	Y					Franciszek Fraszczyk
							Use adjustable chair and moitor to maintain ergonomic workstation posture	Y					Franciszek Fraszczyk
3D Printing	Y	Risk of burns from heating bed/nozzle, exposure to toxic fumes from heated filament.	3	2	6	M	Operate printer in a well ventilated area	Y	1	2	2	L	Franciszek Fraszczyk
							Do not touch nozzel or bed until cooled	Y					Franciszek Fraszczyk
							Do not interact with the plate or piece while machine is operating	Y					Franciszek Fraszczyk
Electrical Shocks	Y	Risk of electroic shocks when handling live circuits, power supply and microcontroller	3	3	9	M	Verify all wiring connections before enabling power	Y	1	2	2	L	Franciszek Fraszczyk
							Ensure all equipment is properly grounded	Y					Franciszek Fraszczyk
							Use low voltage DC systems where possible (<12V)	Y					Franciszek Fraszczyk
							Keep workstation clear of any hazards (hot liquids, food, sharp objects)	Y					Franciszek Fraszczyk
Software Failure	Y	Fault in code may cause unintended motor actuation, overheating or sensor misreadings	2	2	4	M	Test algorithms (Build before upload)	Y	1	1	1	L	Franciszek Fraszczyk
							Implement emergency stop	Y					Franciszek Fraszczyk

Figure D.1. FYP Risk Assessment

Appendix E - Gantt Charts

Task	Stage	Duration	07/09/2025 Week 1	14/09/2025 Week 2	21/09/2025 Week 3	28/09/2025 Week 4	05/10/2025 Week 5	12/10/2025 Week 6	19/05/2025 Week 7	26/05/2025 Week 8	02/11/2025 Week 9	09/11/2025 Week 10	16/11/2025 Week 11	23/11/2025 Week 12
1. Project Planning, Documentation & Research	Completed	Project Long												
1.1 Initial Background Research	Completed	4 Weeks												
1.2 Initial Meeting with Supervisor	Completed	1 Week												
1.2.1 Possible Approaches & Scope Definition	Completed	2 Weeks												
1.2.2 Literature Review	Completed	7 Weeks												
1.2.3 Hazard Assessment	Completed	5 Weeks												
1.2.4 Status Report	Completed	3 Weeks												
2. Data Gathering	Completed	4 Weeks												
2.1 Physical System	Completed	2 Weeks												
2.2 Data Analysis	Completed	Project Long												
3. Prelimiary ML/DL Model Develpoment	Completed	6 Weeks												
3.1 LSTM Model 1	Completed	2 Weeks												
3.2 LSTM Model 2	Completed	3 Weeks												
3.3 LSTM Model 3	Completed	3 Weeks												

Figure E.1. Semester 1 Gantt Chart

Task	Stage	Duration	12/01/2026 Week 19	19/01/2026 Week 20	26/01/2026 Week 21	02/02/2026 Week 22	09/02/2026 Week 23	16/02/2026 Week 24	23/02/2026 Week 25	02/03/2026 Week 26	09/03/2026 Week 27	16/03/2026 Week 28
4. Functional ML/DL Model Develpoment	Completed	10 Weeks										
4.1 Model	Completed	3 Weeks										
4.1.1 Construct a model capable of detection anomalies	Completed	2 Weeks										
4.1.2 Introduce temperature sensor	Removed	2 Weeks										
4.1.3 Measure Accuracy	Completed	1 Week										
4.1.4 Compare against other methods	Completed	1 Week										
4.2 Real time Inference	Completed	3 Weeks										
4.2.0 Model Investigation on microcontroller	Completed	3 Weeks										
4.2.1 Quantise and prune model	Completed	1 Week										
4.2.2 Covert to LiteRT (or equivalent)	Completed	1 Week										
4.2.3 Run real time inference on embedded system	Completed	1 Week										
4.2.4 Measure accuracy and latency	Completed	1 Week										
4.3 Industrial Motor	Completed	5 Weeks										
4.3.1 Modify Model to use on motor	Completed	2 Weeks										
4.3.2 Quantise, prune & convert model	Completed	1 Week										
4.3.3 Run real time inference on embedded system	Completed	1 Week										
4.3.4 Measure Accuracy and Latency	Completed	2 Weeks										

Figure E.2. Semester 2 Gantt Chart

Appendix F - Experimental Rig

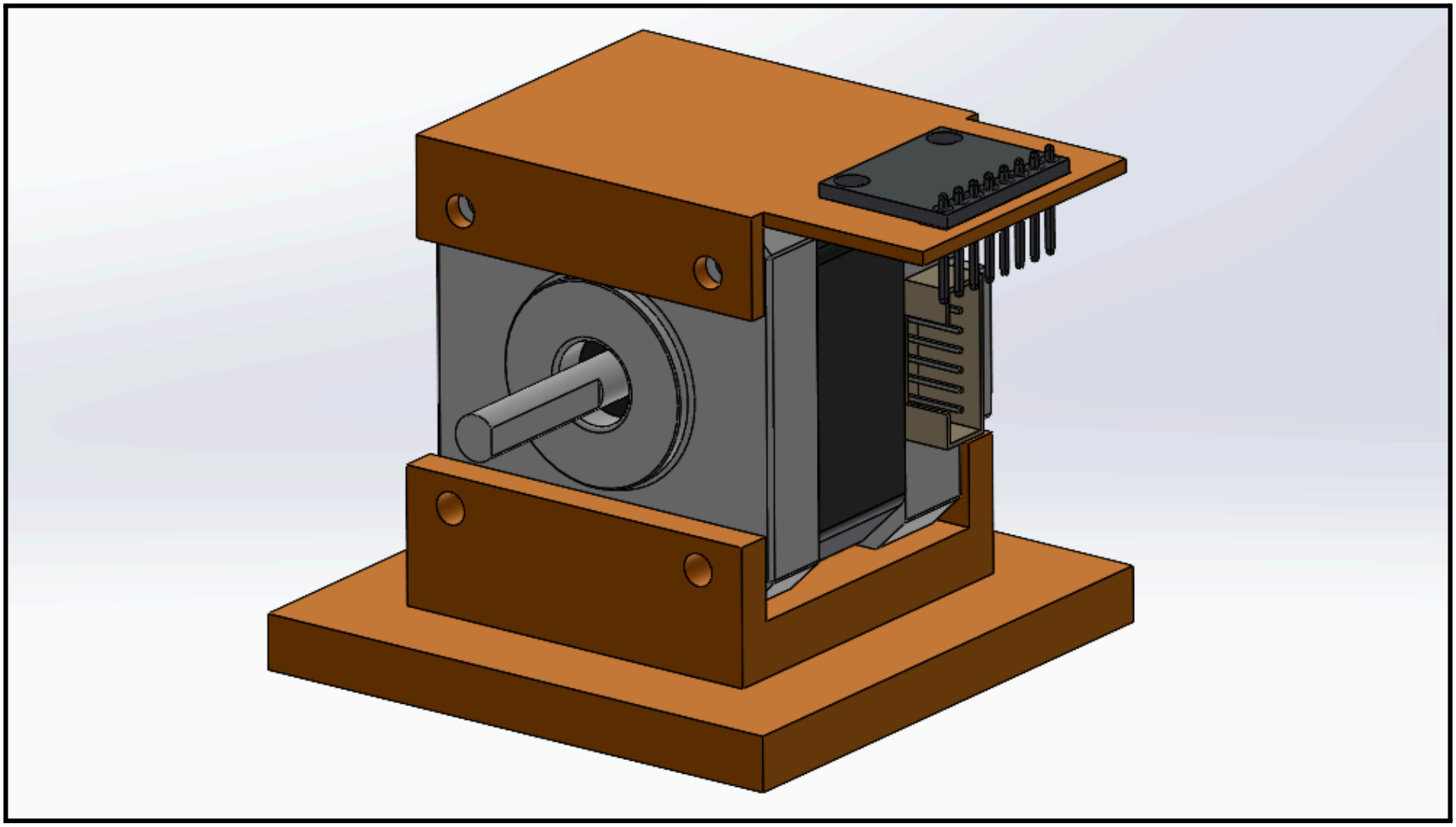


Figure F.1. SOLIDWORKS Assembly of Experimental Rig

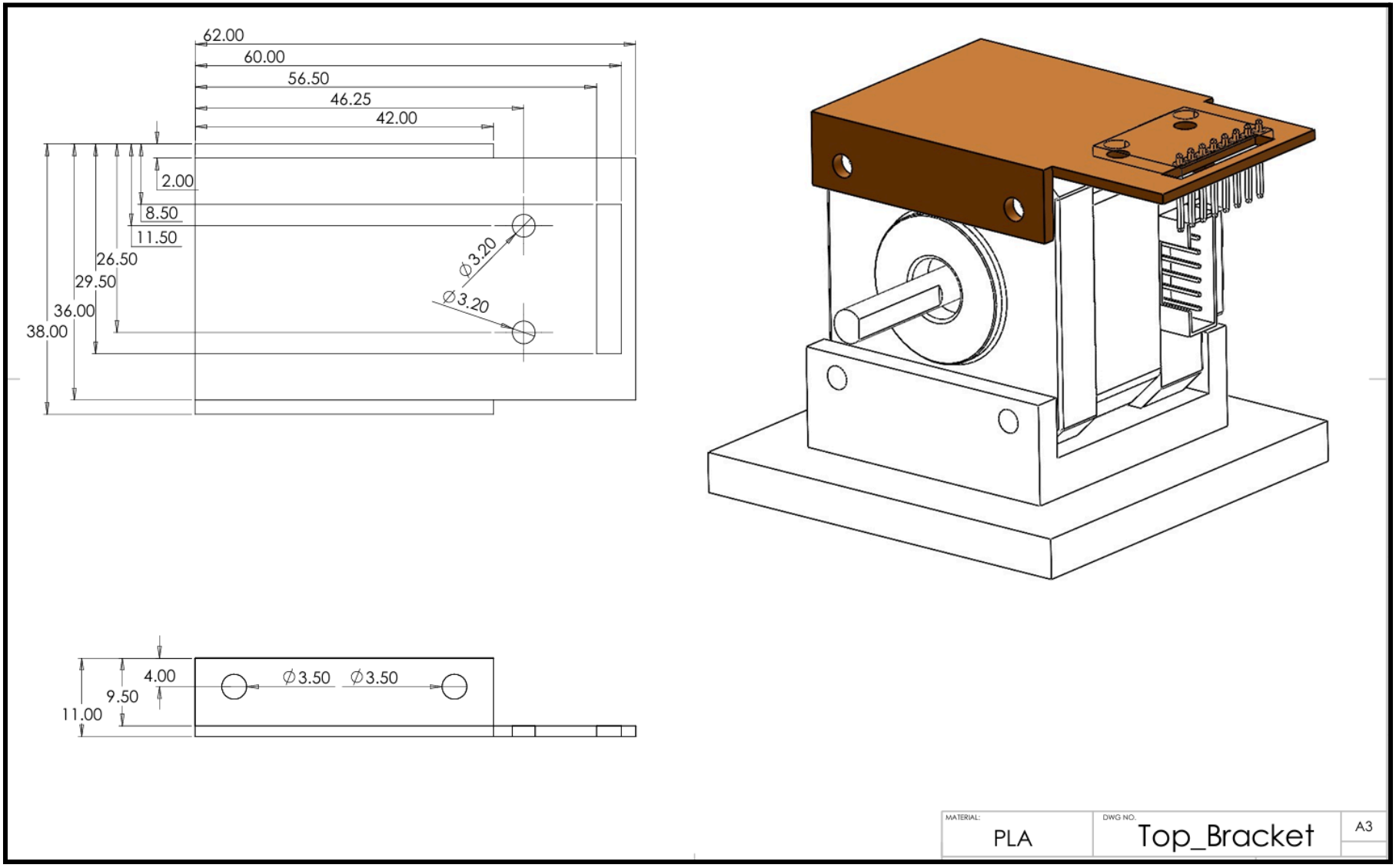


Figure F.2. SOLIDWORKS Drawing of Top Bracket

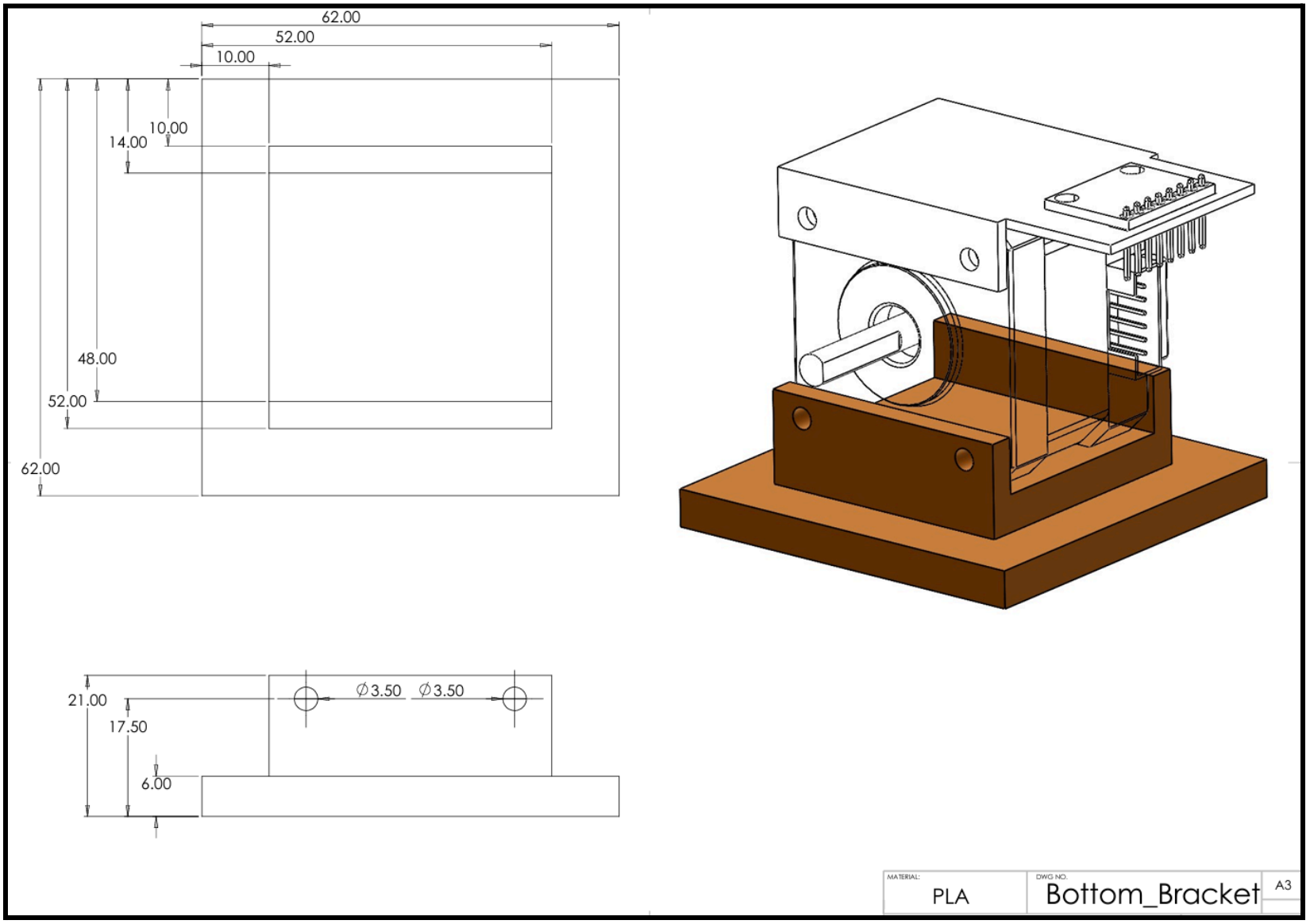


Figure F.3. SOLIDWORKS Drawing of Bottom Bracket

Appendix G - Platform Selection

Strengths vs Weaknesses

Table G.1. *Table detailing the strengths and weaknesses of different programming platforms.*

Platform	Strengths	Weaknesses
Arduino IDE [1]	● Easy to setup and deploy ● Lots of extensive Examples for boards like ESP32 ● Libraries available for anything, Wi-Fi, Sensors, motors, MQTT, etc. ● Cross-platform support	● Minimal debugging support ● Poor code/file management options ● Less powerful for complex projects
Platform IO (VS Code) [7]	● Built in library manager ● Version Control ● Dependency Management ● Multi-file management ● Supports deep debugging ● Can build multiple boards	● Steeper learning curve ● Requires heavy software ie. VScode
Espressif's ESP-IDF [2] [3]	● Official framework from Espressif (For the ESP boards specifically) ● Direct access to all ESP features ● Power management for ESP boards ● Built-in logging/Debugging tools	● Steep learning curve ● Less "plug&play" than arduino or PlatformIO, fail fast method is not as easy ● Documentation is very overwhelming and feels outdated
MicroPython [6]	● Python code directly, much faster than C/C++ ● Interactive read–eval–print loop (REPL) ● Good for rapid IoT prototyping	● Slower performance than C/C++ ● Limited access to low-level features ● Smaller ecosystem (less libraries, examples, community engagement)
Eclipse IDE with ESP-IDF [4] [5]	● Good GUI project management ● Links to Java/C++ ● Strong profiling integration (measure of space, time, complexity of particular instructions)	● Resource heavy ● Less community engagement ● Complex setup

Decision Matrix

Table G.2. *Decision Matrix detailing the chosen platform.*

Criteria	Weight (1-10)	Arduino IDE	Platform IO	Espressif	MicroPython	Eclipse IDE
Ease of Installation	5	10	7	6	8	5
Ease of use	9	10	9	5	8	4
Debugging Support	8	4	8	10	3	7
File Management	3	5	8	8	5	6
Cross-Platform Support	5	7	9	8	8	7
Libraries	8	9	9	7	5	6
Programming Language familiarity	10	8	10	6	6	4
Board Support	9	7	9	5	7	7
Performance Speed	7	6	7	10	4	6
Access to low-level feature	10	4	7	9	3	6
Resource Allocation	10	5	7	10	4	6
		569	691	640	452	483

Selection & Reasoning

- Arduino IDE scores high for its ease of use and installation. There is also a large library ecosystem and a community that is active with support. The Arduino IDE is slightly weaker when it comes to debugging, low-level access, performance and resource control than some of the other choices. This claim is strongly supported by the paper ‘Usability and Performance Analysis’ (2024), where there are tables verifying its inefficiency while still highlighting its ease of use. [16]

- PlatformIO is the optimal choice for this project as it has generally strong marks across all categories. It is a versatile system which has good cross-platform support, good file management across all areas, and better debugging capabilities than the Arduino IDE. With better debugging capabilities, it will be easier to track ML inference failures and progress the project at a faster pace. The ecosystem for PlatformIO is ever expanding and has a strong and growing community with tutorials and various examples. Alongside all these positives to the features, familiarity with the platform and its coding style is another strong asset. [8] [9] [10] [16]

- Espressif/ESP-IDF is by far the most efficient when it comes to performance, low level access, resource allocations and debugging. For example, the ‘Usability and Performance Analysis’ paper (2024) shows that although the Arduino framework facilitates faster development, ESP-IDF yields better energy efficiency and performance on resource constrained devices. This IDE seems ideal performance wise but lacks several critical criteria. Given the nature of this project, the board might change from Espressif hardware due to several different reasons, this would make it no longer applicable as it does not support boards like the RP2040 (Raspberry Pi Pico), which could be a viable choice. [10] [11] [12] [16]

- MicroPython is good in scenarios where fast prototyping is far more important than optimisation and speed. Python as a programming language allows the programmer to code complex functions at a much faster pace at the cost of low level features and resource management. This claim is supported by the work of Plauska et al. (2023) , who in Performance Evaluation of C/C++, MicroPython, Rust and TinyGo on ESP32 found that MicroPython implementations are many times slower that C/C++ in signal processing, which is relevant to this project. Forum threads further support this claim and often compare its speed as subpar when put against C/C++ frameworks. [13] [15]

- Eclipse is a strongly suited IDE for embedded system projects with very specific needs. It has good debugging features and given its Java related background, it is ideal for any projects which require a fully developed GUI. This IDE has the steepest learning curve and is somewhat cumbersome in comparison to the other options, as well as not being recommended on various forums and amongst hobbyists.. [10] [14]

Board Selection

Criteria

- Sufficient RAM / Flash to host a machine learning model,
- Wireless (Wi-Fi) connectivity,
- Good and reliable IO to interface sensors,
- Low-power mode to maximise efficiency,
- Support for edge frameworks (eg. TensorFlow Lite for Microcontrollers, now LiteRT),
- Capability to act as a ‘master node’ for scalability

CPU / MHz

Context

MHz refers to *millions of cycles per second*, this is a quantified measure of how fast the microcontroller executes instructions on its CPU. This parameter is critical in this project because a higher clock frequency directly reduces inference latency - the time taken by the local machine learning model to make a decision. For example if an anomaly occurs, the microcontroller needs to identify and process the event in a fraction of a second in order to be useful in a predictive maintenance project. Another reason to consider the clock speed is that different sensors sample at different rates, and a higher MHz ensures that the processor can manage interrupts while still handling data without missing samples or aliasing. Finally, networking tasks such as the Wi-Fi stack also use CPU cycles. If the CPU is too slow, the combination of the various background tasks could interfere with one another and inevitably block the ML model from working as required. While the paper “Interference-Aware Schedulability Analysis And Task Model for Embedded Systems” (Aceituno et al., 2022) does not specifically detail Wi-Fi stacks as a source of interference, it justifies treating networking overloads as an interference, since they rely on CPU processing time. [17]

Minimum Requirements

Papers like “A Review on TinyML: State-of-The-Art and Prospects” (Ray et al., 2022) and “Understanding TinyML Inference on Resource-Constrained Devices.” (2025) both detail how the majority of microcontrollers that are used in conjunction with TinyML are heavily resource constrained. Despite the low clock speed on some of these boards, TinyML still manages to perform as expected. The first paper reports that an 8-bit model optimised with CMSIS-NN can perform inference in under 20 milliseconds on a Cortex-M4 microcontroller, providing a general reference for expected processing requirements. In addition to running TinyML, the processor must also be capable of handling MQTT communications, which introduces additional CPU load due to network stack interference.

While TinyML can run on sub-100 MHz MCUs (Ray 2022), for this project’s combined workload (TinyML + MQTT + Wi-Fi), 240 MHz is a safer lower bound. [18] [19]

Architecture

Context

The architecture of a microcontroller encompasses the internal hardware design of the chip, including the CPU, memory, peripherals, communication interfaces and I/O pins. The architecture influences the microcontroller's computational power, efficiency and software robustness.

For a project focused on TinyML and MQTT, the architecture of the microcontroller is a crucial matter. It must support efficient ML inference, seamless wireless communication and scalability for industrial applications.

Possible Options

ARM Cortex-M

The ARM Cortex-M architecture is designed and licensed to several chip manufacturers, including Microchip, Arduino, STMicroelectronics, etc. The ARM architecture consists of 32-bit CPUs which are optimised for low-power embedded systems, providing excellent performance per watt. Boards based on this architecture are widely used in industry, and have a mature ecosystem, including full support for LiteRT which facilitates TinyML applications. While boards that support this architecture are more costly than some alternatives, they are an excellent choice for predictive maintenance projects. [20] [21]

Tensilica Xtensa LX6

The Tensilica Xtensa LX6 architecture was developed by Cadence and customised by Espressif specifically for their ESP32 series of microcontrollers. This architecture is built up of a dual-core 32-bit RISC processor, designed for wireless applications. Boards based on this architecture excel in IoT applications that require both robust wireless connectivity and computational performance. Additionally, the Xtensa ecosystem is supported by a large open-source community. The main limitation to this architecture type is that it is primarily restricted to Espressif chips, making it less common in industry applications. [22]

RISC-V

RISC-V is an open source “Instruction Set Architecture” (ISA), which allows for custom processor designs. This open architecture type greatly reduces the cost and strongly encourages innovation, resulting in a rapidly growing ecosystem. While it is a promising architecture for future projects, it is not yet as mature or widely adopted as ARM or Tensilica, this makes it a poor choice when it comes to specifically wanting to use TinyML as any software support may be limited.

Decision

Based on the project requirements for TinyML inference, real-time processing and MQTT, any board based on either the ARM Cortex-M or Tensilica Xtensa LX6 architecture is the most suitable choice. Both architecture types provide the necessary computational performance, energy efficiency, and community support to ensure a reliable and expandable operation.

RAM/SRAM

Context

RAM (Random Access Memory), specifically SRAM (Static RAM) in microcontrollers is the amount of memory available on-chip which is used for storing working data. The data includes sensor inputs through I/O pins, intermediate activations in the ML model, buffers, stack space for function calls, runtime data and wireless communication.

When a microcontroller runs a TinyML model using LiteRT, the available memory (SRAM) is used to temporarily store several different categories of data. First, it holds the model **weights and constants** which are the values learned during training. Some microcontrollers store this information in the flash memory, but during the model's inference, LiteRT loads certain parts of the data into the SRAM for faster access. Second, the system requires memory for the **input/output tensors**. These tensors represent the sensor readings going into the model and the predictions coming out. Third, the inference engine itself (like LiteRT) needs a block of memory (known as the '**tensor arena**'). This preallocated block of memory is where all the intermediate results, activations and temporary calculations for each model layer are stored while the model evaluates the final prediction. Without enough SRAM allocated to the tensor arena, inference cannot run because the engine will not be working with all the tensors and the engine will inevitably make wrong predictions. [24] [25]

Additionally, when using MQTT or other networking stacks, RAM is used for buffers when sending/receiving MQTT packets, the devices connection state, data, etc. [26]

These cumulative reasons detail why RAM/SRAM is important when selecting a microcontroller board. If the available SRAM is too small, several errors could occur, for example, the model may not fit entirely, leading to crashes and failure, or there could be serious slowdowns which would defeat the purpose of the device.

Minimum Requirements

Given the scale of this project, which includes monitoring a small motor, multiple sensors (vibration, temperature, current), performing TinyML anomaly detection, maintaining MQTT communication and running continuously, the minimum amount of SRAM required can be estimated as follows. Research such as MCUNetV2 (Lin et al., 2021) demonstrate that vision oriented CNN-style TinyML models can be optimised to run within ~32 kB of SRAM when carefully compressed and quantised. However, this represents an ideal lower bounder under a highly optimised and constrained system. On the networking side of the project, Espressif documentation indicated that MQTT tasks typically require about 60 kB for program memory, with an additional ~4kB consumed per client (Lee, 2025). This shows that simultaneous communication is a non-trivial overhead which needs to be accounted for, alongside ML workloads. [27] [28]

Finally, relevant entire system level evaluations, like the ones reported in TinyML reviews (Alajlan & Ibrahim, 2022) note that microcontrollers which run both ML inference tasks and network tasks such as MQTT require different amounts of SRAM. There were entire models that ran effectively on a microcontroller with just 138kB of SRAM, while other models ran on devices with 340kB of SRAM. Based on these findings, a safe lower band for this project is at least 256kB of SRAM, with a 512kB recommended to ensure headroom for both ML inference and reliable MQTT communication. [27] [28] [29] [30]

Flash Memory

Context

Flash memory (program flash, nonvolatile memory) is where the microcontroller stores its firmware, program instructions and constant data. Unlike SRAM, the flash memory retains data even when powered off. In relation to this project, the flash memory will store critical information relating to the TinyML model's weights and constants. It will also store sensor drivers, the MQTT client, the network stack and all logic which seamlessly connects all aspects of the project together. The size of the flash limits how large a model or how many features/layers can be included.

Minimum Requirements

In MCUNetV2 (Lin et al., 2021), the authors present experiments with various different microcontrollers, ones with 1 MB flash and others with 2 MB. The paper details that the model size must be balanced against the flash size. In "How to Manage Tiny Machine Learning at Scale – An Industrial Perspective" (Ren, H. and Runkler, T. et al., 2022) the researchers had their specific model run with a flash size of as little as 531 kB, *"Suppose we trained a TinyML model for a motion classification task using accelerometer and gyroscope data. From the parsing result of the model, its minimum RAM and Flash requirements are 116 and 531 kb, respectively."*. Considering the firmware size, model size, space for logs, calibration and future expandability a practical minimum flash size would be 1MB, with 2-4MB being a safer target to allow room for enhancements. The increased flash size would also allow for future firmware updates to be made without having to replace the board or purchase any additional components. [28] [30]

Summary Table

Table G.3. *Table of boards*

Board	CPU (MHz / Core)	Architecture	RAM / SRAM	Flash	Wireless	Suitability Note
Arduino Uno (ATmega328P)	16 MHz (8-bit, single-core)	AVR 8-bit RISC	2 KB SRAM	32 KB	None	❌ Not suitable
Arduino Mega 2560	16 MHz (8-bit, single-core)	AVR 8-bit RISC	8 KB SRAM	256 KB	None	❌ Not suitable
Raspberry Pi Pico (RP2040)	Dual-core 133 MHz	ARM Cortex-M0+	264 KB SRAM	2 MB	None	❌ Not suitable
Adafruit Feather STM32F405	Cortex-M4 @ 168 MHz (single-core)	ARM Cortex-M4	192 KB SRAM	1 MB	None	❌ Not suitable
ESP8266 (ESP-12E module)	80–160 MHz (single-core)	Tensilica LX106	160 KB SRAM	4 MB	Wi-Fi	❌ Borderline
Raspberry Pi Pico W	Dual-core 133 MHz	ARM Cortex-M0+	264 KB SRAM	2 MB	Wi-Fi + BT	❌ Borderline
STM32F746 Discovery	Cortex-M7 @ 216 MHz (single-core)	ARM Cortex-M7	320 KB SRAM	1 MB	add-on needed	✅ Suitable
Seeed XIAO ESP32-C3	RISC-V @ 160 MHz (single-core)	RISC-V	400 KB SRAM	4 MB	Wi-Fi + BT	✅ Suitable
ESP32-WROOM 32	Dual-core 240 MHz	Tensilica Xtensa LX6	~520 KB SRAM	4 MB	Wi-Fi + BT	✅ Perfect fit
ESP32-S3	Dual-core 240 MHz	Tensilica Xtensa LX7	512 KB SRAM	8–16 MB	Wi-Fi + BT	✅ Very suitable
Arduino Portenta H7	Dual-core Cortex-M7 @ 480 MHz + Cortex-M4 @ 240 MHz	ARM Cortex-M7/M4	8 MB SRAM	16 MB	Wi-Fi + BT	✅ Suitable but expensive
Raspberry Pi Zero 2 W	Quad-core 1 GHz	ARM Cortex-A53	512 MB LPDDR2 RAM	microSD card	Wi-Fi + BT	⚠ Overkill
NVIDIA Jetson Nano	Quad-core Cortex-A57 @ 1.43 GHz + 128-core GPU	ARM Cortex-A57 + GPU (SoC)	4 GB RAM	microSD / eMMC	Wi-Fi	⚠ Massive overkill

Table G.4. *Legend detailing colour patterns in Table C*

CPU	RAM/SRAM	Flash Memory	Explanation
<120Mhz	<128 kB	>1MB	Not suitable - too weak
<240Mhz	<256 kB	>2MB	Borderline - usable with limitations
~240-480Mhz	256-512kB	2-4MB	Suitable - directly matches project needs
>480Mhz	>1 MB	<4MB	Overkill - too much for this project

References

[1] ARDUINO, “Software.” Accessed: Sept. 2025. [Online]. Available: https://www.arduino.cc/en/software/

[2] ESPRESSIF, “ESP-IDF Programming Guide - ESP32.” Accessed: 2025. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html

[3] Espressif Systems, ESP32-WROOM-32D & ESP32-WROOM-32U Datasheet, 2025.

[4] Espressif, “IDF Eclipse Plugin.” Accessed: 2019. [Online]. Available: https://github.com/espressif/idf-eclipse-plugin/blob/master/README.md

[5] C. Guindon, “Eclipse IDE | The Eclipse Foundation.” Accessed: 2025. [Online]. Available: https://www.eclipseide.org/

[6] MicroPython, “MicroPython – Python for Microcontrollers.” Accessed: 2025. [Online]. Available: https://micropython.org/

[7] PlatformIO, “Professional Collaborative Platform for Embedded Development.” Accessed: 2025. [Online]. Available: https://docs.platformio.org/en/latest/

[8] S. Santos, “Getting Started with VS Code and PlatformIO IDE for ESP32 and ESP8266.” Accessed: Sept. 23, 2025. [Online]. Available: https://randomnerdtutorials.com/vs-code-platformio-ide-esp32-esp8266-arduino

[9] X. Decuyper, “Why Use PlatformIO Instead of Arduino IDE.” Accessed: 2025. [Online]. Available: https://simplyexplained.com/courses/programming-esp32-with-arduino/platformio-vs-arduino-ide

[10] PlatformIO, “So PlatformIO and Arduino Yes, ESP-IDF No?” Accessed: Sept. 23, 2025. [Online]. Available: https://community.platformio.org/t/so-platformio-and-arduino-yes-esp-idf-no/15289/3

[11] Reddit, “Looking at the ESP-IDF: Is It Worth It?” Accessed: Sept. 23, 2025. [Online]. Available: https://www.reddit.com/r/esp32/comments/1c5z8uf/looking_at_the_espidf_im_wondering_if_its_worth/

[12] PlatformIO, “Espressif IoT Development Framework — PlatformIO Documentation.” Accessed: Sept. 23, 2025. [Online]. Available: https://docs.platformio.org/en/latest/frameworks/espidf.html

[13] Arduino Forum, “ESP32 – Arduino or MicroPython? + Arduino PlatformIO Problems.” Accessed: Sept. 23, 2025. [Online]. Available: https://forum.arduino.cc/t/esp32-arduino-or-micropython-arduino-platformio-problems/614674

[14] Arduino Forum, “Choosing a New IDE for ESP32.” Accessed: Sept. 23, 2025. [Online]. Available: https://forum.arduino.cc/t/choosing-a-new-i-d-e-for-esp32/1164656/14

[15] I. Plauska, A. Liutkevičius, and A. Janavičiūtė, “Performance Evaluation of C/C++, MicroPython, Rust and TinyGo Programming Languages on ESP32 Microcontroller,” Electronics,

vol. 12, no. 1, p. 143, 2022, doi: 10.3390/electronics12010143.

[16] E. Scaffi, A. Bonneau, M.-F. Le, and F. Mieyeville, “Usability and Performance Analysis of Embedded Development Environment for On-device Learning,” arXiv, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2404.07948

[17] J. M. Aceituno et al., “Interference-Aware Schedulability Analysis and Task Allocation for Multicore Hard Real-Time Systems,” Electronics, vol. 11, no. 9, p. 1313, Apr. 2022, doi: 10.3390/electronics11091313.

[18] P. P. Ray, “A Review on TinyML: State-of-the-Art and Prospects,” J. King Saud Univ. – Comput. Inf. Sci., vol. 34, no. 4, pp. 1595–1623, Apr. 2022, doi: 10.1016/j.jksuci.2021.11.019.

[19] Embedded Explorer, “Understanding TinyML Inference on Resource-Constrained Devices.” Accessed: Sept. 24, 2025. [Online]. Available: https://embeddedexplorer.com/understanding-tinyml-inference-on-resource-constrained-devices/

[20] Arduino, “Arduino Hardware.” Accessed: 2025. [Online]. Available: https://www.arduino.cc/en/hardware/

[21] TensorFlow, “Integrating Arm Virtual Hardware with TensorFlow Lite Micro CI Infrastructure.” Accessed: Sept. 25, 2025. [Online]. Available: https://blog.tensorflow.org/2022/10/integrating-arm-virtual-hardware-with-tensorflow-lite-micro-continuous-integration-infrastructure.html

[22] M. Z. H. Zim, “TinyML: Analysis of Xtensa LX6 Microprocessor for Neural Network Applications by ESP32 SoC,” arXiv, 2021. [Online]. Available: https://arxiv.org/pdf/2106.10652

[23] Google AI for Developers, “LiteRT for Microcontrollers.” Accessed: 2025. [Online]. Available: https://ai.google.dev/edge/litert/microcontrollers/overview

[24] GoogleSource, “TensorFlow Lite Micro – Memory Management.” Accessed: Sept. 28, 2025. [Online]. Available: https://android.googlesource.com/platform/external/tensorflow/+/a444526fa7a096ead6096b7e36c8da4371d6380b/tensorflow/lite/micro/docs/memory_management.md

[25] Google AI for Developers, “Get Started with LiteRT.” Accessed: 2025. [Online]. Available: https://ai.google.dev/edge/litert/inference

[26] T. H. Team, “HiveMQ – TLS and MQTT: How Is the Performance Affected?” Accessed: 2025. [Online]. Available: https://www.hivemq.com/blog/how-does-tls-affect-mqtt-performance/

[27] J. Lee, “Lightweight MQTT Broker for ESP32: Mosquitto Ported to ESP-IDF.” Accessed: Sept. 28, 2025. [Online]. Available: https://developer.espressif.com/blog/2025/05/esp-idf-mosquitto-port/

[28] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, “MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning,” arXiv, 2021, doi: 10.48550/arXiv.2110.15352.

[29] N. N. Alajlan and D. M. Ibrahim, “TinyML: Enabling Inference Deep Learning Models on Ultra-Low-Power IoT Edge Devices,” Micromachines, vol. 13, no. 6, p. 851, 2022, doi: 10.3390/mi13060851.

[30] H. Ren and T. Runkler, “How to Manage Tiny Machine Learning at Scale – An Industrial Perspective.” Accessed: Sept. 28, 2025. [Online]. Available: https://cms.tinyml.org/wp-content/uploads/talks2022/2202.09113.pdf